

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 3 — Programma, UI, Tools e Build

Logica Programma, UI Qt/PySide6, Design Atlas, Ingestion Pipeline, Tools Diagnostici,
Suite di Test, Rimediazione

Autore: Renan Augusto Macena

Agosto 2026

Ultimate CS2 Coach — Parte 3: Programma, UI, Tools e Build

Argomenti: Schema completo del database (three-tier storage, SQLite WAL, SQLAlchemy ORM), Regime di formazione e limiti di maturità (CALIBRATING/LEARNING/MATURE), Catalogo delle funzioni di perdita, Logica Completa del Programma (dal lancio al consiglio): Session Engine, Digester, Teacher, Hunter, Pulse; Interfaccia desktop Qt/PySide6 (15 schermate, ViewModels, Qt Signals); Pipeline di ingestione (demo e pro); Console di controllo unificata; Onboarding nuovo utente; Architettura di storage; Motore di playback e viewer tattico; Dati spaziali e mappe; Osservabilità e logging; Reporting; Quote e limiti; Tolleranza ai guasti; Viaggio completo dell'utente (4 flussi); Suite di strumenti (validazione e diagnostica); Test suite; Pre-commit hooks; Build, packaging, deployment; Migrazioni Alembic; HLTV sync service; RASP Guard; MatchVisualizer; File di configurazione runtime; Entry point root-level; Glossario.

Autore: Renan Augusto Macena

Questo documento è la continuazione della **Parte 2 — Servizi, Analisi e Database**, che copre i servizi di coaching, i motori di analisi, la conoscenza e il Database. La Parte 3 scende al livello di programma: come l'intero sistema si avvia, come i daemon si orchestrano, come l'interfaccia desktop rende visibile il coaching, come il codice è testato, validato e deployato.

Indice

Parte 3 — Programma, UI, Tools e Build (questo documento)

9. [Schema del database e ciclo di vita dei dati](#)
10. [Regime di formazione e limiti di maturità](#)
11. [Catalogo delle funzioni di perdita](#)
12. [Logica Completa del Programma — Dal Lancio al Consiglio](#)
 - 12.1 Avvio applicazione e bootstrap
 - 12.2 Session Engine e Quad-Daemon
 - 12.3 Daemon Digester e pipeline di ingestione
 - 12.4 Daemon Teacher e training orchestrator
 - 12.5 Interfaccia Desktop (Qt/PySide6, 15 schermate, ViewModels)
 - 12.6 Pipeline di Ingestione ([ingestion/](#))
 - 12.7 Console di Controllo Unificata ([backend/control/](#))
 - 12.8 Onboarding e Flusso Nuovo Utente
 - 12.9 Architettura di Storage ([backend/storage/](#))
 - 12.10 Motore di Playback e Viewer Tattico
 - 12.11 Dati Spaziali e Gestione Mappe
 - 12.12 Osservabilità e Logging
 - 12.13 Reporting e Visualizzazione

- 12.14 Gestione Quote e Limiti
 - 12.15 Tolleranza ai Guasti e Recupero
 - 12.16 Viaggio Completo dell'Utente — 4 Flussi Principali
 - 12.17 Suite di Strumenti — Validazione e Diagnostica (`tools/`)
 - 12.18 Architettura della Test Suite (`tests/`)
 - 12.19 Le Fasi di Rimediazione Sistemica
 - 12.19.1 La campagna di audit integrale (agosto 2026)
 - 12.20 Pre-commit Hooks e Quality Gates
 - 12.21 Build, Packaging e Deployment
 - 12.22 Sistema Migrazioni Alembic
 - 12.23 Orchestratore Ingestione Principale (`run_ingestion.py`)
 - 12.24 HLTV Sync Service e Background Daemon
 - 12.25 RASP Guard — Integrità Runtime del Codice
 - 12.26 MatchVisualizer — Rendering Avanzato
 - 12.27 File di Dati e Configurazione Runtime
 - 12.28 Entry Point Root-Level
- [Riepilogo Architetture](#)
 - [Mappa delle Interconnessioni tra le 3 Parti](#)
 - [Glossario Tecnico](#)

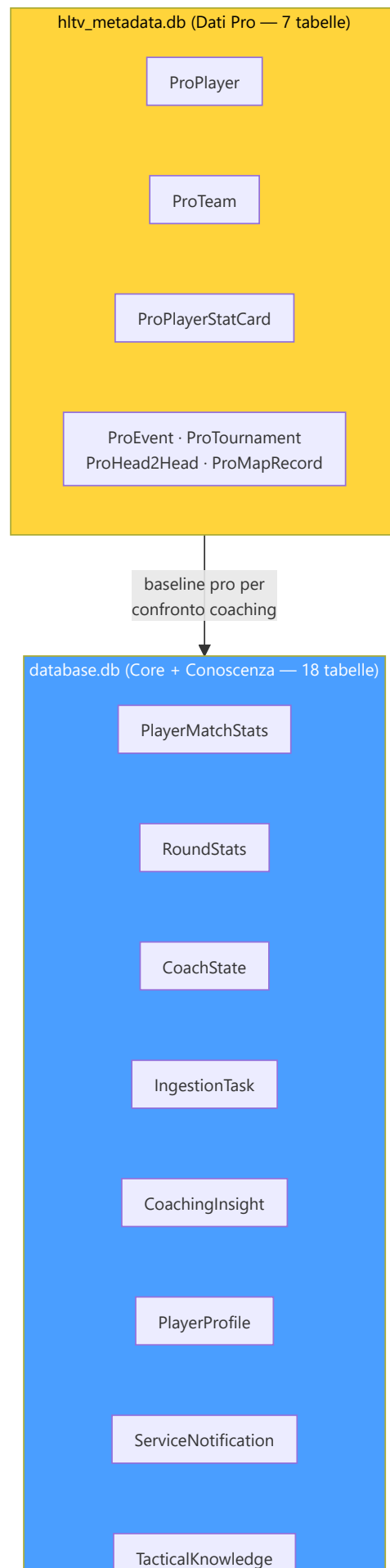
9. Schema del database e ciclo di vita dei dati

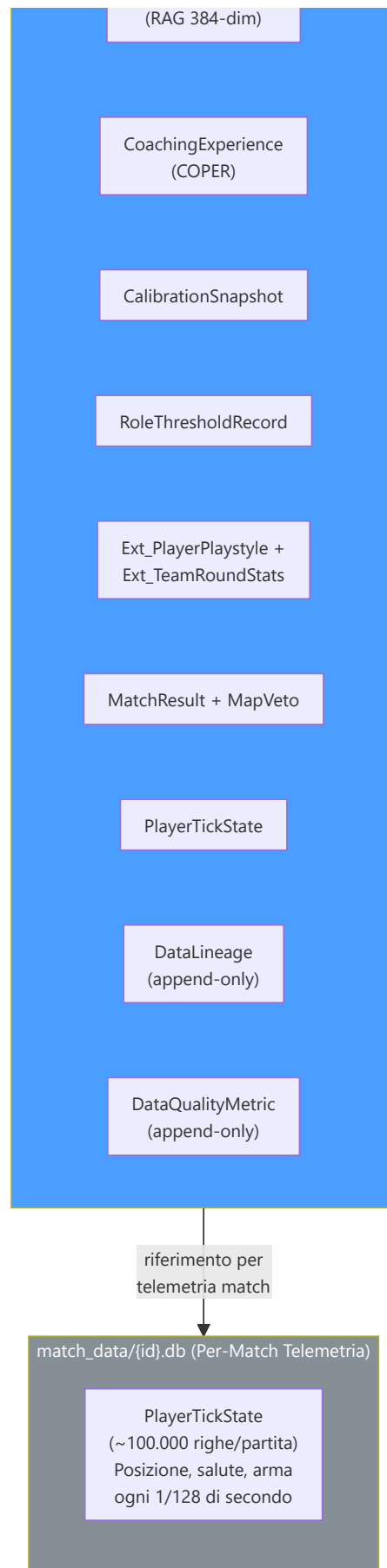
Il progetto utilizza **SQLModel** (Pydantic + SQLAlchemy) con SQLite (modalità WAL) e un'**architettura three-tier storage** specializzata. In totale **28 tabelle SQLModel** (`backend/storage/db_models.py` : 25 classi `table=True` ; `backend/storage/match_data_manager.py` : 3 tabelle per-match) distribuite su 3 tier di storage:

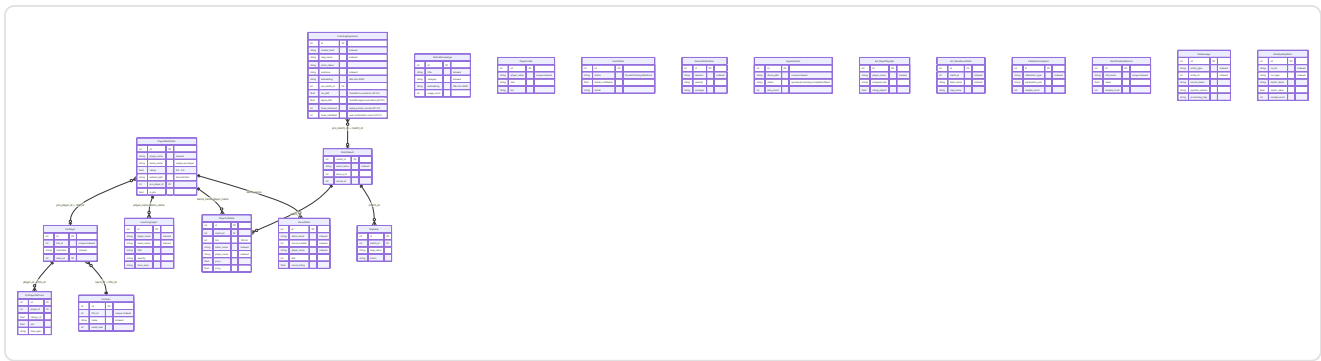
1. **database.db** — Database monolite principale dell'applicazione (**18 tabelle**, elencate esplicitamente in `database.py:_MONOLITH_TABLES`). Contiene tutte le tabelle core: statistiche giocatori (`PlayerMatchStats`), stato del coach (`CoachState`), task di ingestione (`IngestionTask`), insight di coaching (`CoachingInsight`), profili utente (`PlayerProfile`), notifiche di sistema (`ServiceNotification`), base RAG (`TacticalKnowledge`), banca esperienze COPER (`CoachingExperience`), risultati partite (`MatchResult` , `MapVeto`), calibrazioni (`CalibrationSnapshot`), soglie di ruolo (`RoleThresholdRecord`), tracciamento provenienza (`DataLineage` , `DataQualityMetric`), e tabelle estese per round team (`Ext_TeamRoundStats`) e stile di gioco (`Ext_PlayerPlaystyle`), oltre a `RoundStats` e `PlayerTickState` archiviale.
2. **hlty_metadata.db** — Database dei metadati professionali (**7 tabelle**, `database.py:_HLTV_TABLES`): profili dei giocatori pro (`ProPlayer` , `ProTeam`), schede statistiche (`ProPlayerStatCard`) e tabelle estese (`ProEvent` , `ProTournament` , `ProHead2Head` , `ProMapRecord`). Separato dal monolite perché viene scritto da un processo separato (HLTV sync service) per eliminare la contesa WAL con i daemon del session engine.
3. **match_data/{id}.db** — Database per-match di telemetria (**3 tabelle**, definite in `match_data_manager.py` : `MatchTickState` , `MatchEventState` , `MatchMetadata`). Ogni partita ha il proprio file SQLite dedicato (`match_{id}.db`) contenente i dati tick-per-tick. Gestito da `MatchDataManager` (cache LRU di engine, creazione a runtime della directory `match_data/`). Questa separazione risolve il problema dello "Telemetry Cliff" — evita che il database monolite cresca indefinitamente con dati ad alta frequenza.

Questa separazione a tre livelli garantisce che le operazioni di scrittura intensive del session engine (ingestione demo, addestramento ML → `database.db`) non contendano lock WAL con lo scraping HLTV in processo separato (`hltv_metadata.db`), e che la telemetria ad alta frequenza per-match non appesantisca il monolite (`match_data/{id}.db`).









Descrizione tabelle: **PlayerMatchStats** — statistiche aggregate per giocatore per partita (uccisioni, morti, ADR, rating HLTV 2.0, 25 feature normalizzate). **PlayerTickState** — telemetria tick-by-tick a 64/128 Hz: posizione (x,y,z), salute, armatura, angolo di visuale, arma corrente, economia, stato per ogni giocatore in ogni tick. **RoundStats** — statistiche isolate per-round: uccisioni, morti, danni, uccisioni noscope, assist flash, rating del round, utili per coaching granulare. **CoachingExperience** — record COPER: contesto del momento di coaching, consiglio dato, esito (positivo/negativo/neutro), efficacia numerica, embedding 384-dim per ricerca semantica. **CoachingInsight** — insight di coaching generati dal sistema e mostrati all'utente nell'UI. **TacticalKnowledge** — knowledge base RAG: strategie, posizionamenti, uso utility, indicizzati con embedding Sentence-BERT 384-dim, versione v3, 14 categorie. **RoleThresholdRecord** — soglie apprese per la classificazione dei 10 ruoli tattici, persistite tra riavvii. **CalibrationSnapshot** — timestamp e conteggio campioni di ogni auto-calibrazione del Bayesian death estimator. **Ext_PlayerPlaystyle** — metriche di stile di gioco da CSV esterni, usate per training di NeuralRoleHead. **ServiceNotification** — messaggi di errore/evento dai daemon in background, mostrati nell'UI tramite polling. **DataLineage** — audit trail append-only: traccia quale demo ha originato ogni entità e attraverso quale step del pipeline è passata. **DataQualityMetric** — metriche quantitative di qualità per esecuzione del pipeline (percentuale campioni scartati, tasso fallback zero-tensor).

Ciclo di vita dei dati:

Fase	Tabelle scritte	Volume
Inserimento demo	<code>PlayerMatchStats</code> , <code>PlayerTickState</code> , <code>MatchMetadata</code>	~100.000 tick/partita
Arricchimento round	<code>RoundStats</code> (isolamento per round, per giocatore)	~30 righe/partita (round × giocatori)
Scansione HLTV	<code>ProPlayer</code> , <code>ProTeam</code> , <code>ProPlayerStatCard</code>	~500 giocatori
Importazione CSV	Tabelle esterne tramite <code>csv_migrator.py</code>	~10.000 righe
Dati sullo stile di gioco	<code>Ext_PlayerPlaystyle</code> (da CSV per NeuralRoleHead)	~300+ giocatori
Progettazione delle feature	<code>PlayerMatchStats.dataset_split</code> aggiornato	Sul posto
Popolazione RAG	<code>TacticalKnowledge</code>	~200 articoli
Estrazione dell'esperienza	<code>CoachingExperience</code>	~1.000 per partita
Output di coaching	<code>CoachingInsight</code>	~5-20 per partita
Apprendimento delle soglie di ruolo	<code>RoleThresholdRecord</code>	9 soglie
Calibrazione delle convinzioni	<code>CalibrationSnapshot</code> (dopo il riaddestramento)	1 per ogni calibrazione eseguita
Telemetria di sistema	<code>CoachState</code> , <code>ServiceNotification</code> , <code>IngestionTask</code>	Continuo
Tracciamento provenienza	<code>DataLineage</code> (append-only per ogni entità processata)	~N righe per demo ingerita
Metriche qualità pipeline	<code>DataQualityMetric</code> (append-only per ogni esecuzione pipeline)	~5-10 metriche per run
Backup	Automatizzato tramite <code>BackupManager</code> (7 rotazioni giornaliere + 4 settimanali)	Copia completa del database

Indici e ottimizzazione query:

Le tabelle più interrogate hanno indici strategici per garantire query veloci:

Tabella	Indice	Colonne	Tipo di query ottimizzata
<code>PlayerMatchStats</code>	<code>idx_pms_player</code>	<code>player_name</code>	Ricerca per giocatore
<code>PlayerMatchStats</code>	<code>idx_pms_demo</code>	<code>demo_name</code>	Ricerca per partita
<code>PlayerMatchStats</code>	<code>idx_pms_processed</code>	<code>processed_at</code>	Ordinamento cronologico
<code>RoundStats</code>	<code>idx_rs_demo_round</code>	<code>demo_name</code> , <code>round_number</code>	Ricerca per round specifico
<code>IngestionTask</code>	<code>idx_it_status</code>	<code>status</code>	Coda di lavoro (status=queued)
<code>CoachingInsight</code>	<code>idx_ci_player</code>	<code>player_name</code>	Insight per giocatore
<code>TacticalKnowledge</code>	<code>idx_tk_category</code>	<code>category</code>	Ricerca RAG per categoria
<code>ProPlayer</code>	<code>idx_pp_name</code>	<code>nickname</code>	Ricerca pro per nome
<code>DataLineage</code>	<code>idx_dl_entity</code>	<code>entity_type</code> , <code>entity_id</code>	Tracciamento provenienza per entità
<code>DataQualityMetric</code>	<code>idx_dqm_run</code>	<code>run_id</code> , <code>run_type</code>	Metriche qualità per esecuzione

Vincoli di integrità:

Vincolo	Tabelle	Enforcement
<code>player_name</code> NOT NULL	PlayerMatchStats, RoundStats	A livello di schema
<code>demo_name</code> UNIQUE per player	PlayerMatchStats	Previene duplicati
<code>status</code> CHECK IN ('queued', 'processing', 'completed', 'failed')	IngestionTask	Enum enforcement
<code>dataset_split</code> CHECK IN ('train', 'val', 'test')	PlayerMatchStats	Split validity
Foreign key <code>demo_name</code>	RoundStats → PlayerMatchStats	Relazione round-partita

Dettaglio tabelle chiave:

PlayerMatchStats (32 campi) — la tabella più interrogata del sistema:

La tabella PlayerMatchStats contiene tutte le statistiche aggregate per giocatore per partita. È la "pagella" di ogni partita analizzata:

Campo	Tipo	Descrizione
<code>id</code>	Integer PK	Identificatore univoco
<code>player_name</code>	String	Nome del giocatore (indexed)
<code>demo_name</code>	String	Nome del file demo (unique per player)
<code>kills, deaths, assists</code>	Integer	KDA base
<code>adr</code>	Float	Average Damage per Round
<code>kast</code>	Float	Kill/Assist/Survive/Trade %
<code>headshot_percentage</code>	Float	HS%
<code>hltv_rating</code>	Float	Rating HLTV 2.0 calcolato
<code>dataset_split</code>	String	"train" / "val" / "test"
<code>is_pro</code>	Boolean	Flag giocatore professionista
<code>processed_at</code>	DateTime	Timestamp di elaborazione
<code>map_name</code>	String	Mappa giocata
...	...	20+ campi aggiuntivi per feature avanzate

TacticalKnowledge — la base RAG:

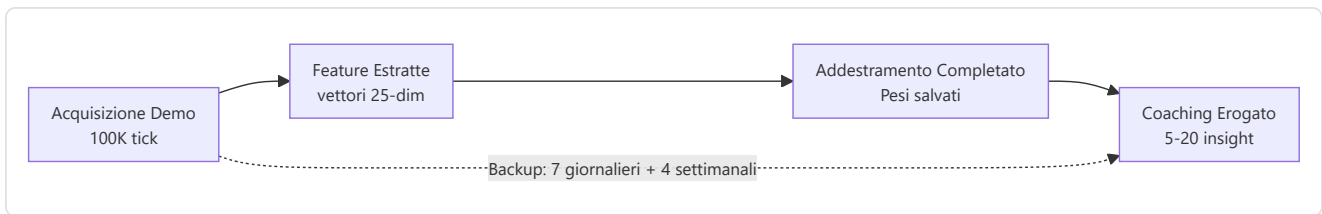
Campo	Tipo	Descrizione
<code>id</code>	Integer PK	Identificatore
<code>title</code>	String	Titolo del documento (indexed)
<code>description</code>	String	Descrizione del contenuto tattico
<code>category</code>	String	"positioning" / "economy" / "utility" / "aim" (indexed)
<code>map_name</code>	String	Mappa specifica (indexed, opzionale)
<code>situation</code>	String	Contesto situazionale: "T-side pistol round", "CT retake A site"
<code>pro_example</code>	String	Riferimento a demo pro (opzionale)
<code>embedding</code>	String	Vettore 384-dim JSON (sentence-transformers)
<code>created_at</code>	DateTime	Timestamp di creazione
<code>usage_count</code>	Integer	Contatore utilizzo

La ricerca semantica RAG funziona calcolando la **cosine similarity** tra l'embedding della query utente e gli embedding precomputati di ogni documento. I top-3 risultati con similarity > 0.5 vengono usati per arricchire il contesto di coaching. Il campo `situation` consente il filtraggio contestuale (es. "T-side pistol round") prima della ricerca semantica.

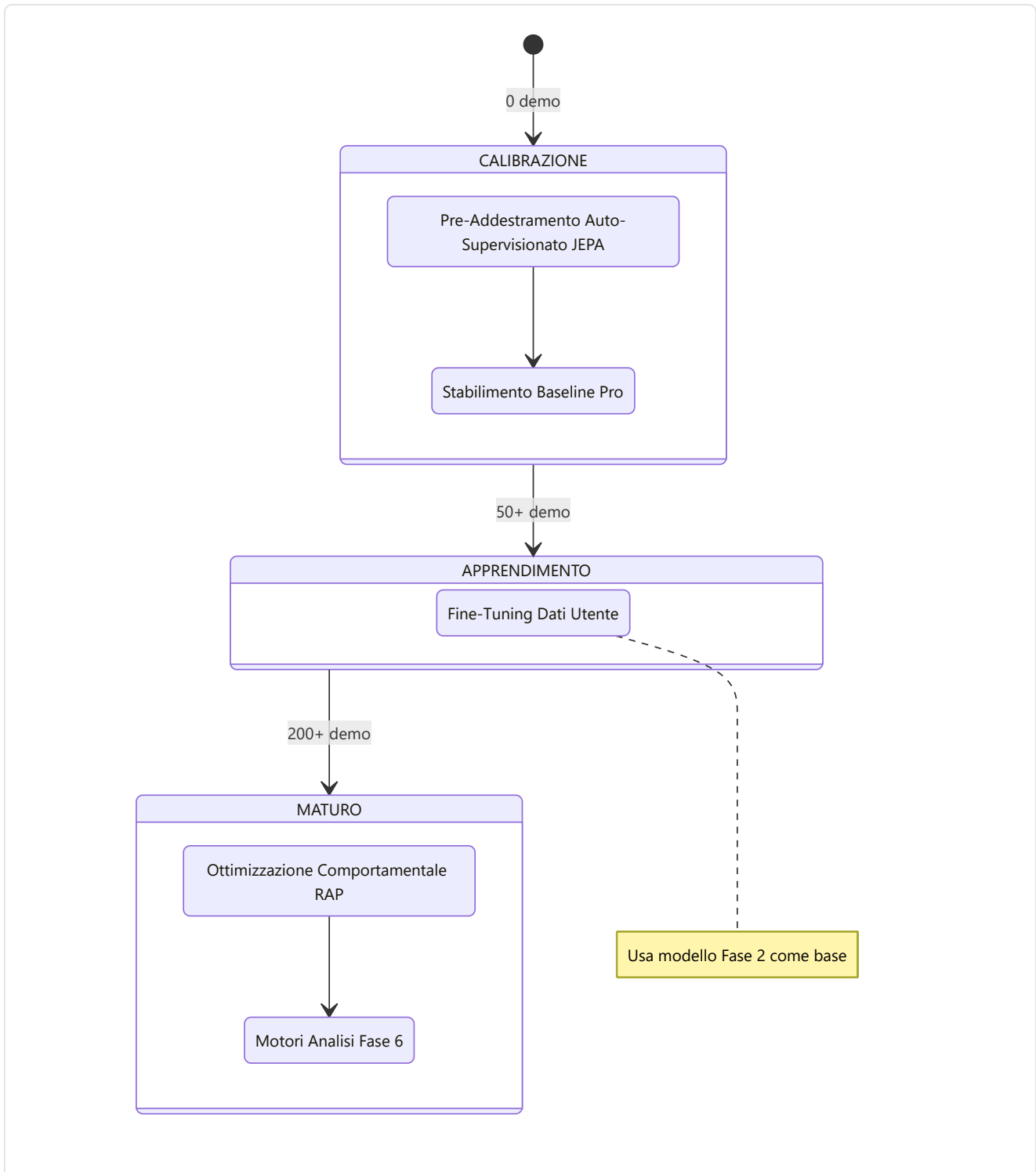
CoachingExperience — la banca COPER (22+ campi):

Campo	Tipo	Descrizione
<code>id</code>	Integer PK	Identificatore
<code>context_hash</code>	String	Hash dello stato di gioco per lookup rapido (indexed)
<code>map_name</code>	String	Mappa (indexed)
<code>round_phase</code>	String	"pistol" / "eco" / "full_buy" / "force"
<code>side</code>	String	"T" / "CT"
<code>position_area</code>	String	"A-site" / "Mid" / etc. (indexed, opzionale)
<code>game_state_json</code>	String	Snapshot completo del tick (max 16KB, validato con <code>field_validator</code>)
<code>action_taken</code>	String	"pushed" / "held_angle" / "rotated" / "used_utility" / etc.
<code>outcome</code>	String	"kill" / "death" / "trade" / "objective" / "survived" (indexed)
<code>delta_win_prob</code>	Float	Variazione della probabilità di vittoria da questa azione
<code>confidence</code>	Float	Affidabilità/generalizzabilità 0.0-1.0
<code>usage_count</code>	Integer	Quante volte recuperata per coaching
<code>pro_match_id</code>	Integer FK	Riferimento a MatchResult (ON DELETE SET NULL)
<code>pro_player_name</code>	String	Nome giocatore pro di riferimento (indexed, opzionale)
<code>embedding</code>	String	Vettore 384-dim JSON per ricerca semantica (opzionale)
<code>source_demo</code>	String	Demo di origine (opzionale)
<code>created_at</code>	DateTime	Timestamp di creazione
<code>outcome_validated</code>	Boolean	Se l'esito è stato validato
<code>effectiveness_score</code>	Float	Score -1.0 a 1.0
<code>follow_up_match_id</code>	Integer	Partita di follow-up per tracking
<code>times_advice_given</code>	Integer	Quante volte il consiglio è stato dato
<code>times_advice_followed</code>	Integer	Quante volte il consiglio è stato seguito
<code>last_feedback_at</code>	DateTime	Ultimo feedback ricevuto
<code>mu_skill</code>	Float	TrueSkill μ posterior — stima della qualità dell'esperienza (KT-01)
<code>sigma_skill</code>	Float	TrueSkill σ uncertainty — incertezza sulla qualità (KT-01)
<code>times_retrieved</code>	Integer	Contatore replay priority — quante volte recuperata (KT-01)
<code>times_validated</code>	Integer	Contatore conferme utente — quante volte validata (KT-01)

Il sistema COPER utilizza queste esperienze per **imparare dai propri consigli**: il campo `context_hash` consente il lookup rapido di situazioni simili, `outcome` e `delta_win_prob` misurano l'efficacia dell'azione, e il ciclo di feedback (`outcome_validated`, `times_advice_given/followed`, `effectiveness_score`) consente al sistema di prioritizzare consigli che hanno storicamente prodotto risultati positivi. Il campo `game_state_json` è limitato a 16KB per prevenire crescita incontrollata del database. I campi TrueSkill `mu_skill` / `sigma_skill` (KT-01) forniscono una stima bayesiana della qualità dell'esperienza, usata per la prioritizzazione del replay: $\text{confidence_score} = \text{mu_skill} - \kappa \times \text{sigma_skill}$. Il contatore `times_retrieved` implementa la replay priority, mentre `times_validated` traccia le conferme utente per il CRUD semantico.

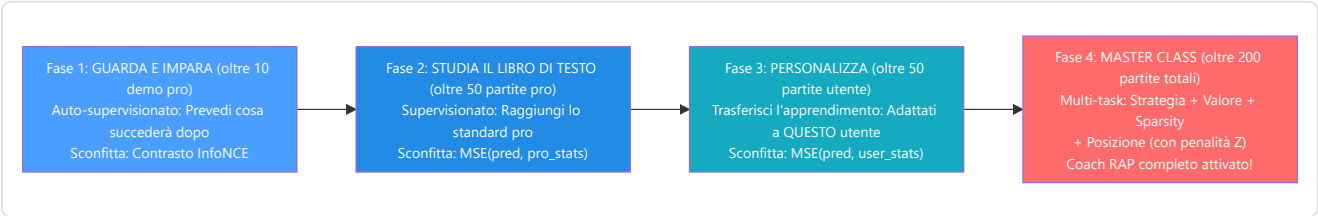


10. Regime di formazione e limiti di maturità



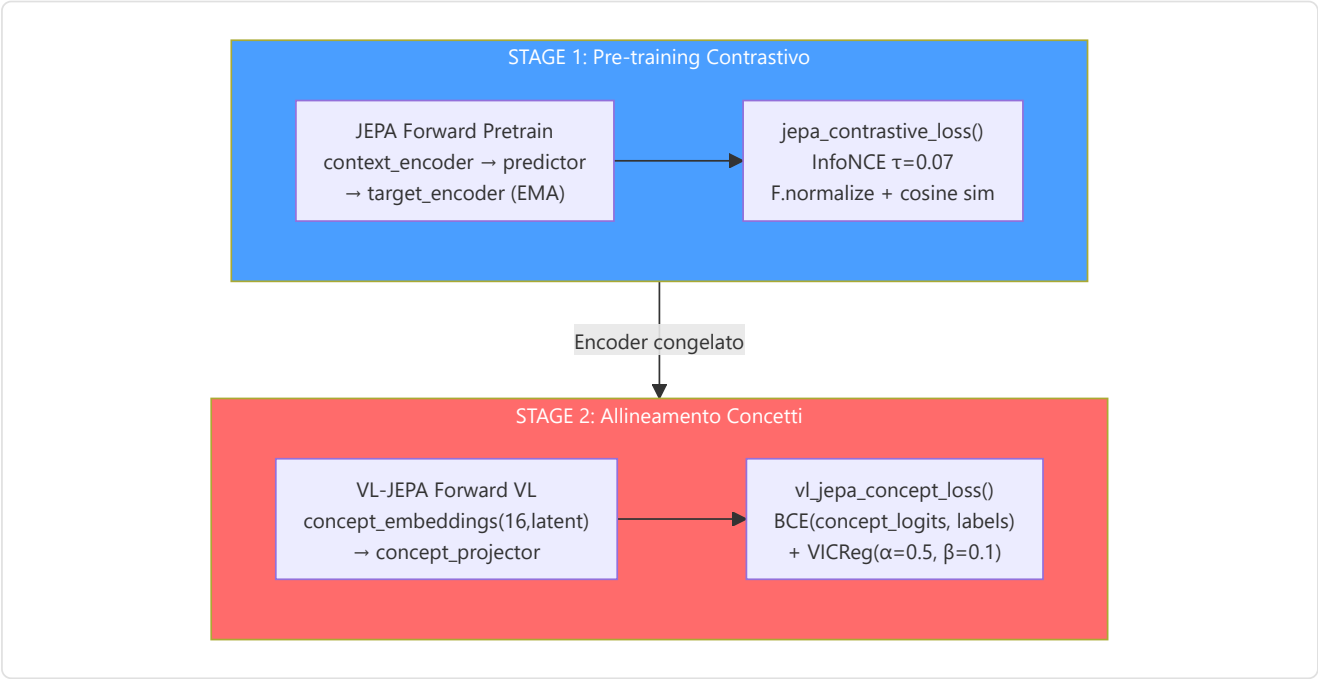
Requisiti dati per fase:

Fase	Dati minimi	Tipo di addestramento	Perdita primaria
1. Pre-addestramento JEPA	10 demo pro	Auto-supervisionato (InfoNCE)	Contrattivo con negativi in batch
2. Baseline pro	50 corrispondenze pro	Supervisionato	MSE(pred, pro_stats)
3. Ottimizzazione utente	50 corrispondenze utente	Supervisionato (trasferimento)	MSE(pred, user_stats)
4. Ottimizzazione RAP	200 corrispondenze totali	Multi-task	Strategia + Valore + Sparsità + Posizione



Protocollo VL-JEPA Two-Stage (Allineamento Concetti):

Quando il VL-JEPA è attivo, le Fasi 1-2 vengono estese con un **protocollo two-stage** che allinea le rappresentazioni latenti ai 16 coaching concepts:



Stage	Cosa si addestra	Cosa è congelato	Loss	Scopo
1	Context encoder, predictor	Target encoder (EMA)	InfoNCE ($\tau=0.07$)	Rappresentazioni latenti generali
2	Concept embeddings, concept projector, concept temperature	Encoder (opzionale fine-tuning)	BCE + VICReg diversity	Allineamento ai 16 coaching concepts

I 16 Coaching Concepts (tassonomia — da `COACHING_CONCEPTS` in `jepa_model.py`):

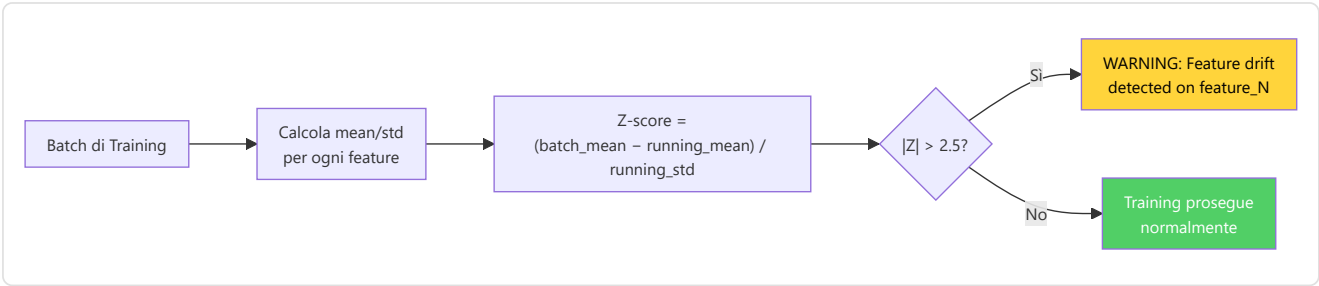
Indice	Concetto	Dimensione	Descrizione
0	positioning_aggressive	Posizionamento	Combattimenti ravvicinati, push degli angoli
1	positioning_passive	Posizionamento	Angoli lunghi, evita il contatto
2	positioning_exposed	Posizionamento	Posizione vulnerabile, alta probabilità di morte
3	utility_effective	Utility	Utilità con impatto significativo
4	utility_wasteful	Utility	Utilità inutilizzata o a basso impatto
5	economy_efficient	Decisione	Equipaggiamento allineato al tipo di round
6	economy_wasteful	Decisione	Force-buy sfavorevoli o morte con gear costoso
7	engagement_favorable	Ingaggio	Combattimenti con vantaggio HP/posizione/numeri
8	engagement_unfavorable	Ingaggio	Combattimenti in svantaggio numerico/HP
9	trade_responsive	Ingaggio	Trade kill rapidi, buon coordinamento
10	trade_isolated	Ingaggio	Morte senza trade, troppo isolato
11	rotation_fast	Decisione	Rotazione posizionale rapida dopo info
12	information_gathered	Decisione	Buona raccolta intel, nemici individuati
13	momentum_leveraged	Psicologia	Capitalizza hot streak con giocate sicure
14	clutch_composed	Psicologia	Decisioni calme in situazioni 1vN
15	aggression_calibrated	Psicologia	Aggressività appropriata alla situazione

AdamW + CosineAnnealing (JEPA Trainer):

Iperparametro	Valore	Scopo
Optimizer	AdamW	Weight decay separato dai gradienti
Learning rate	1e-4 (default; layer concept a lr×0.05)	Tasso di apprendimento iniziale
Weight decay	0.01	Regolarizzazione L2
Scheduler	SequentialLR: warmup lineare (5%) → CosineAnnealingLR	Warmup + decadimento coseno del LR
EMA decay	0.996 base, schedulazione coseno → 1.0	Target encoder momentum update
Gradient clip	1.0	Prevenzione gradient explosion
AMP + accumulo	GradScaler (CUDA) + 4 step di accumulo	Efficienza su GPU con poca VRAM

DriftMonitor (z_threshold=2.5):

Il **DriftMonitor** integrato nel JEPA Trainer monitora il **drift delle feature** durante il training. Se lo Z-score di una feature supera 2.5, emette un warning che indica possibile distribuzione shifting:



Trigger di riaddestramento: Il demone Teacher monitora la crescita del numero di demo professionali; attiva il riaddestramento quando `count ≥ last_count × 1,10`.

JEPAPretrainDataset (`jepa_train.py`):

Il dataset di pre-training JEPA utilizza **finestre temporali** per creare coppie contesto-target:

Parametro	Valore	Scopo
<code>context_len</code>	10 tick	Lunghezza finestra di contesto (input)
<code>target_len</code>	10 tick	Lunghezza finestra target (da predire)
<code>seed</code>	42	RNG dedicato per finestre riproducibili
Batch size (pretrain standalone)	16	Numero di coppie per batch
Vincoli sequenza	min 20 / max 500 tick	<code>_MIN_TICKS_FOR_SEQUENCE</code> / <code>_MAX_TICKS_PER_SEQUENCE</code>

11. Catalogo delle funzioni di perdita

Modello	Nome della perdita	Formula	Scopo
JEPA	InfoNCE Contrastive	$-\log(\exp(\text{sim}(\text{pred}, \text{target})/\tau) / \sum \exp(\text{sim}(\text{pred}, \text{neg}_i)/\tau))$, $\tau=0.07$, <code>F.normalize</code> prima della similarità del coseno	Allineamento delle previsioni di contesto con gli embedding del target
JEPA	Ottimizzazione	<code>MSE(coaching_head(Z_ctx), y_true)</code>	Punteggio di coaching supervisionato
AdvancedCoachNN	Supervisionato	<code>MSELoss(MoE_output, y_true)</code>	Allenamento a livello di partita
RAP	Strategia	<code>MSELoss(advice_probs, target_strat)</code>	Raccomandazione tattica corretta
RAP	Valore	$0,5 \times \text{MSE}(V(s), \text{true_advantage})$	Stima accurata del vantaggio
RAP	Sparsità	$\text{Entropia}(\text{gate_probs}) \times \text{context_gate_l1_weight} (1e-4)$	Specializzazione esperta (routing deciso)
RAP	Posizione	$\text{MSE}(xy) + 2 \times \text{MSE}(z)$	Posizionamento ottimale con penalità sull'asse Z
WinProb	Previsione	<code>BCELoss(pred, risultato)</code> (l'output è già passato per Sigmoid)	Previsione dell'esito del round
NeuralRoleHead	KL-Divergence	<code>KLDivLoss(log_softmax(pred), target)</code> con smoothing delle etichette $\epsilon=0,02$	Corrispondenza della distribuzione di probabilità del ruolo
VL-JEPA	Allineamento dei concetti	<code>BCE(concept_logits, concept_labels) + VICReg(concept_diversity)</code>	Fondamenti del concetto di linguaggio visivo

Dettaglio: InfoNCE Contrastive Loss (JEPA)

L'InfoNCE è la loss principale del pre-training JEPA. Il suo scopo è allineare le predizioni del contesto con gli embedding del target, respingendo contemporaneamente i negativi (altri campioni nel batch):

$$L_{\text{InfoNCE}} = -\log\left(\frac{\exp(\text{sim}(\text{pred}, \text{target}^*))}{\tau} / \sum_i \frac{\exp(\text{sim}(\text{pred}, \text{target}_i))}{\tau}\right)$$

Componente	Valore	Ruolo
<code>sim()</code>	Cosine similarity dopo <code>F.normalize</code>	Misura di similarità [-1, +1]
<code>τ</code> (temperature)	0.07	Sharpness della distribuzione — valori bassi = più selettivi
<code>target+</code>	L'embedding target corretto per questo contesto	Il "positivo" — la risposta corretta
<code>target_i</code>	Tutti gli embedding nel batch	Negativi in-batch — le risposte sbagliate
Batch size	32	Numero di negativi = batch_size - 1 = 31

Dettaglio: VL-JEPA Concept Loss (VICReg Components)

La loss di allineamento concetti del VL-JEPA combina due componenti:

```
L_concept = BCE(concept_logits, concept_labels) + α·VICReg_diversity
```

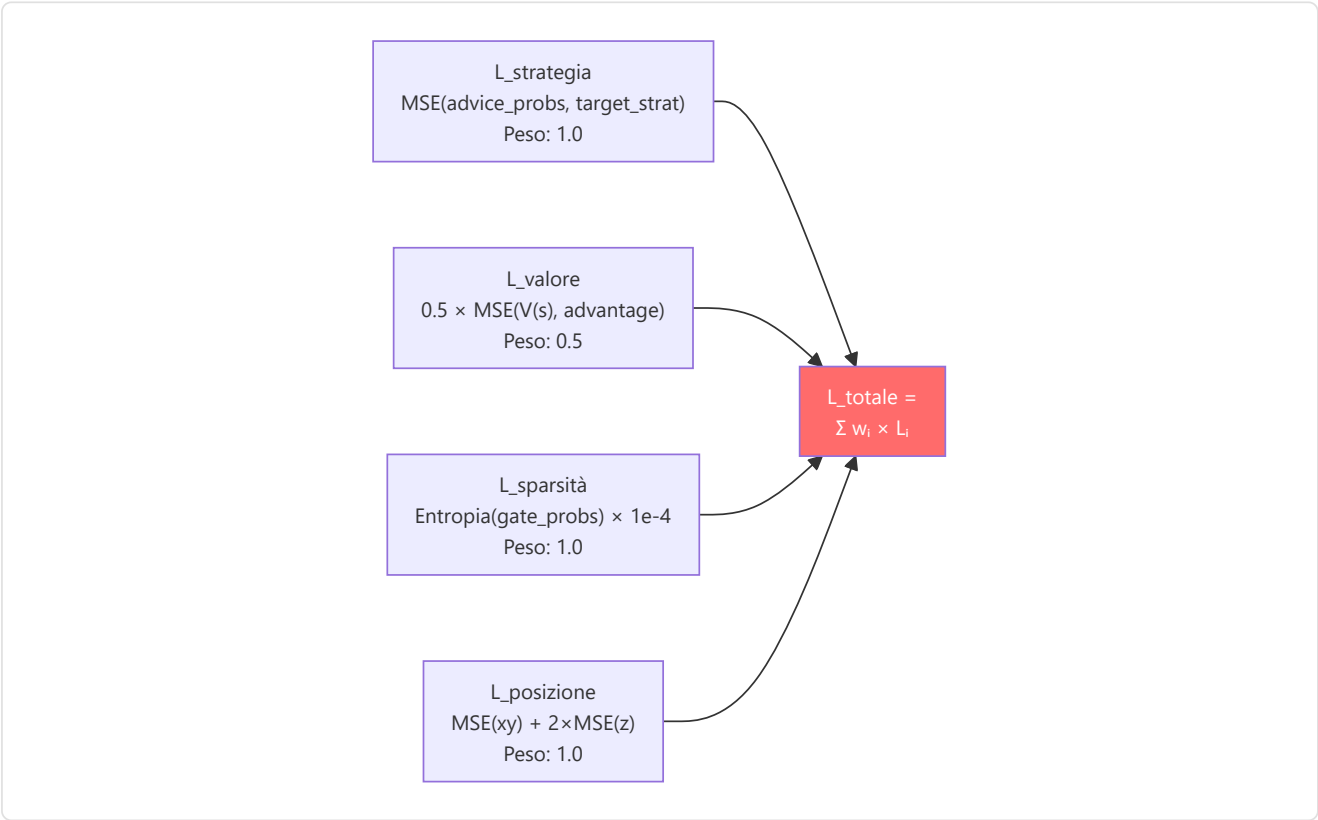
Nel dettaglio implementativo:

Termine	Formula	Peso	Scopo
Concept alignment	<code>BCE_with_logits(concept_logits, labels)</code>	$\alpha=0.5$	Allinea gli embedding ai concetti corretti
Diversity	<code>-std(L2_norm(concept_embeddings), dim=0).mean()</code>	$\beta=0.1$	Previene il collasso — le embedding di concetto devono restare distinte

Una regolarizzazione VICReg completa (`vicreg_regularization` , $\lambda_var=25.0$, $\lambda_cov=1.0$) è disponibile separatamente e viene aggiunta con peso 0.01 alla loss InfoNCE nel trainer.

Dettaglio: RAP Multi-Task Loss

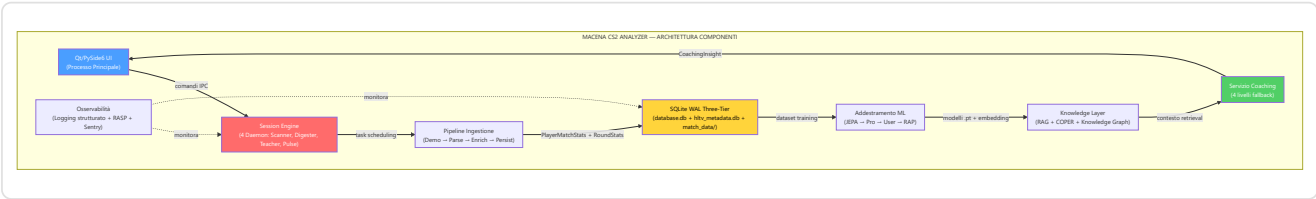
Il RAP Coach combina 4 loss in una loss totale pesata:



La penalità Z 2× nella loss di posizione riflette il fatto che in CS2 sbagliare il **piano** (sopra vs sotto in Nuke/Vertigo) è molto più grave che sbagliare la posizione orizzontale. Un errore di 100 unità sull'asse Z (piano sbagliato) è strategicamente catastrofico, mentre lo stesso errore su X/Y potrebbe essere irrilevante.

12. Logica Completa del Programma — Dal Lancio al Consiglio

Questo capitolo documenta la **logica completa** di Macena CS2 Analyzer, dal momento in cui l'utente lancia l'applicazione fino a quando riceve i consigli di coaching. A differenza dei capitoli precedenti che si concentrano sui sottosistemi AI, qui viene spiegato come **ogni componente del programma** lavora insieme: l'interfaccia desktop, l'architettura quad-daemon, la pipeline di ingestione, il sistema di storage, il playback tattico, l'osservabilità e il ciclo di vita dell'applicazione.



12.1 Punto di Ingresso e Sequenza di Avvio

Il sistema dispone di **un entry point principale** (Qt) e tre entry point di utilità:

#	Entry Point	Comando	Ruolo
1	Qt (primario)	<code>python -m Programma_CS2_RENAN.apps.qt_app.app</code> (via <code>launch.sh</code>)	UI desktop PySide6
2	Console operatore	<code>python console.py</code> (root, TUI Rich + CLI)	Controllo di sistema
3	Headless Validator	<code>python tools/headless_validator.py</code> (root)	Validazione CI/CD
4	HLTV Sync	<code>python -m Programma_CS2_RENAN.hltv_sync_service</code>	Scraping dati pro

12.1.1 Entry Point Qt (Primario) — `apps/qt_app/app.py`

File: `Programma_CS2_RENAN/apps/qt_app/app.py`

La sequenza di avvio Qt è basata su **QApplication** e **signal/slot**:

- High-DPI setup** — Abilita scaling automatico per display ad alta densità
- QApplication** — Crea l'istanza applicazione Qt con gestione args
- Tema e font** — Registra i 3 temi (CS2, CSGO, CS1.6) via QSS e i font personalizzati
- MainWindow** — Costruisce `QMainWindow` con sidebar + `QStackedWidget` (15 schermate)
- Signal wiring** — Connette i segnali Qt tra sidebar, schermate e backend
- First-run gate** — Se `SETUP_COMPLETED=False`, mostra il wizard; altrimenti la home
- Backend console boot** — Lancia il Session Engine come subprocess
- Window show** — Mostra la finestra e avvia il loop eventi Qt
- CoachState polling** — Timer Qt per aggiornamento periodico dello stato

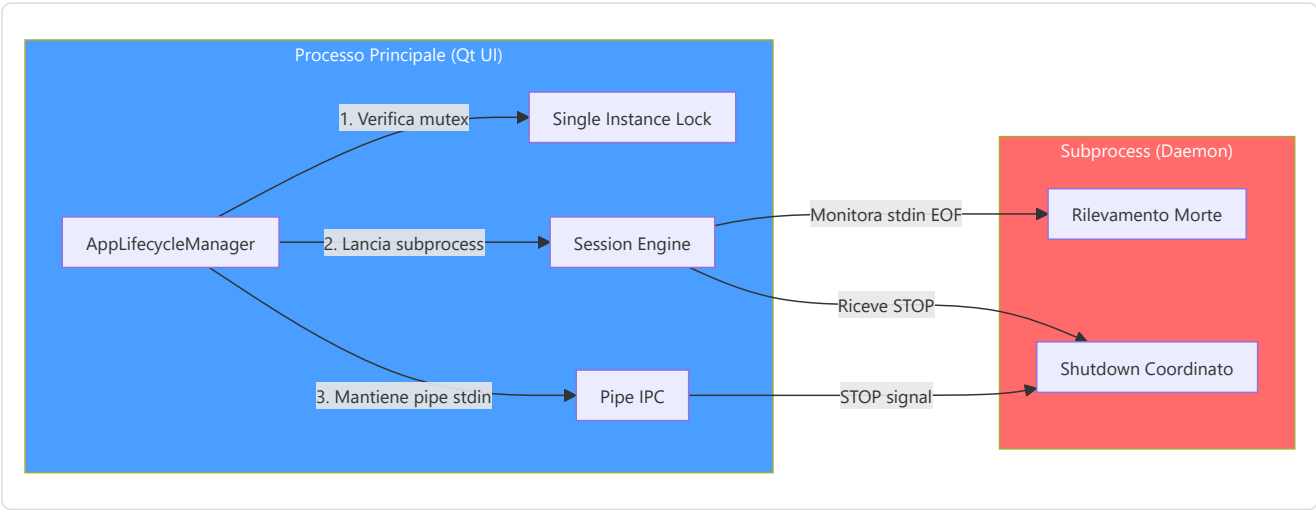
12.2 Gestione del Ciclo di Vita (`lifecycle.py`)

File: `Programma_CS2_RENAN/core/lifecycle.py`

L' `AppLifecycleManager` è un **Singleton** che gestisce il ciclo di vita dell'intera applicazione: dalla garanzia che esista una sola istanza attiva, al lancio del subprocess daemon, fino allo shutdown coordinato.

Meccanismi principali:

Meccanismo	Implementazione	Scopo
Single Instance Lock	Windows Named Mutex / file lock su Linux	Impedisce istanze multiple (corruzione DB)
Lancio Daemon	<code>subprocess.Popen(session_engine.py)</code> con <code>PYTHONPATH</code>	Processo separato per lavoro pesante
Rilevamento Morte Genitore	Il daemon monitora EOF su <code>stdin</code>	Se il processo principale muore, il daemon si arresta
Shutdown Graceful	Invio "STOP" via stdin → daemon termina entro 5s	Nessuna perdita di dati o task zombie
Status Polling	UI interroga <code>CoachState</code> ogni 10s	Aggiornamento stato daemon senza IPC diretto
Error Recovery	Se daemon muore, errore registrato + <code>ServiceNotification</code>	Utente informato, nessun crash silenzioso
Keep-Alive	UI verifica heartbeat ogni 15s	Se heartbeat > 30s stale → warning "Daemon non risponde"

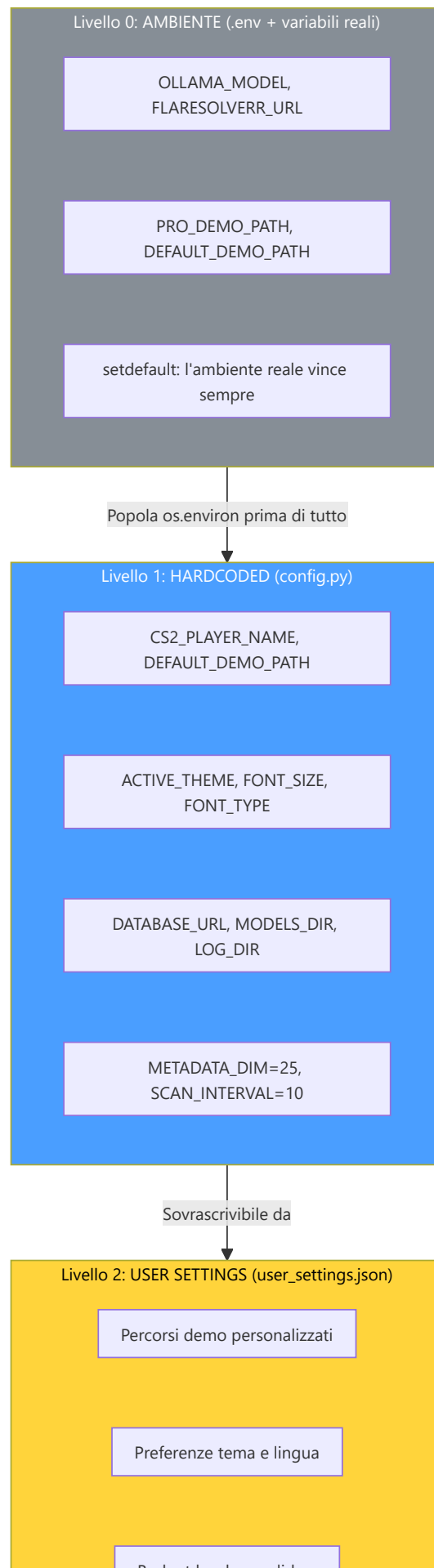


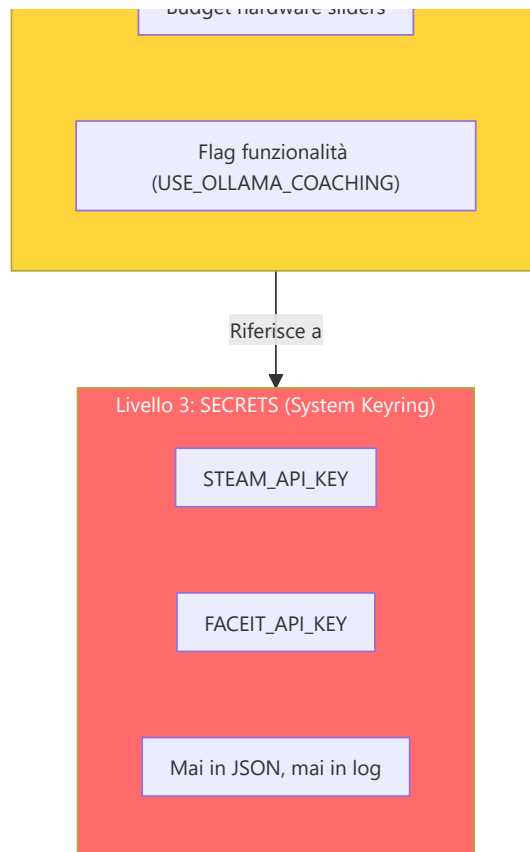
12.3 Sistema di Configurazione (`config.py`)

File: `Programma_CS2_RENAN/core/config.py`

Il sistema utilizza **quattro livelli di configurazione**, ciascuno con un diverso livello di persistenza e sicurezza:







Il livello 0 e la lezione che porta con sé. Fino all'agosto 2026 il file `.env` era una superficie di configurazione **documentata ma mai letta**: nessuna dipendenza lo interpretava e nessuno script lo caricava, quindi ogni override descritto nella documentazione era un'istruzione che non faceva niente — falliva in silenzio, che è il modo peggiore di fallire. Oggi `_load_dotenv_file()` in `core/config.py` lo analizza con la sola libreria standard, al momento dell'import del modulo, e lo fa con tre cautele deliberate:

- scrive con `os.environ.setdefault`, quindi **una variabile d'ambiente reale vince sempre** sul file — chi lancia il programma dentro un container o un job CI non viene scavalcato da un file sul disco;
- scarta le chiavi che non siano alfanumeriche più underscore, così una riga malformata non inietta nulla di strano nell'ambiente del processo;
- **non logga mai i valori**, perché `.env` può contenere chiavi API.

Se il file non esiste non succede nulla; se non è leggibile si ottiene un avviso, non un errore fatale. Sopra questo livello, quattro chiavi di percorso (`PRO_DEMO_PATH`, `DEFAULT_DEMO_PATH`, `BRAIN_DATA_ROOT`, `CUSTOM_STORAGE_PATH`) possono essere riscritte dall'ambiente anche quando `user_settings.json` contiene già un valore, e tre segreti (`STEAM_API_KEY`, `FACEIT_API_KEY`, `STORAGE_API_KEY`) arrivano dal portachiavi di sistema. `set_secret()` **restituisce False invece di sollevare un'eccezione** quando il portachiavi non è disponibile: su una macchina Linux senza backend installato, salvare le impostazioni non deve far cadere l'applicazione.

Costanti critiche del sistema:

Costante	Valore	File	Scopo
<code>METADATA_DIM</code>	25	<code>config.py</code>	Dimensione del feature vector — contratto unificato
<code>SCAN_INTERVAL</code>	10	<code>config.py</code>	Intervallo scansione Scanner (secondi)
<code>MAX_DEMOS_PER_MONTH</code>	10	<code>config.py</code>	Quota mensile upload demo
<code>MAX_TOTAL_DEMOS</code>	100	<code>config.py</code>	Limite totale demo a vita
<code>MIN_DEMOS_FOR_COACHING</code>	10	<code>config.py</code>	Soglia per coaching personalizzato completo
<code>TRADE_WINDOW_S</code>	3.0	<code>trade_kill_detector.py</code>	Finestra temporale trade kill in secondi (tick-rate aware: 192 tick a 64/s, 384 a 128/s)
<code>BASLINE_KPR</code>	0.679	<code>feature_engineering/rating.py</code>	Baseline HLTV 2.0 per KPR
<code>BASLINE_DPR_COMPLEMENT</code>	0.317	<code>feature_engineering/rating.py</code>	Baseline HLTV 2.0 per sopravvivenza
<code>FOV_DEGREES</code>	90	<code>core/constants.py</code>	Campo visivo simulato del giocatore
<code>MEMORY_DECAY_TAU_S</code>	2.5s	<code>core/constants.py</code>	Emivita memoria nemici ($\times$ tick_rate)
<code>CONFIDENCE_ROUNDS_CEILING</code>	300	<code>correction_engine.py</code>	Tetto round per confidenza massima
<code>SILENCE_THRESHOLD</code>	0.2	<code>explainability.py</code>	Soglia sotto la quale il silenzio è azione valida
<code>MIN_SAMPLES_FOR_VALIDITY</code>	30	<code>role_thresholds.py</code>	Campioni minimi per soglia ruolo valida
<code>HALF_LIFE_DAYS</code>	90	<code>pro_baseline.py</code>	Decadimento temporale dati pro
<code>Z_LEVEL_THRESHOLD</code>	200	<code>connect_map_context.py</code>	Soglia Z per classificazione piano

Architettura dei percorsi:

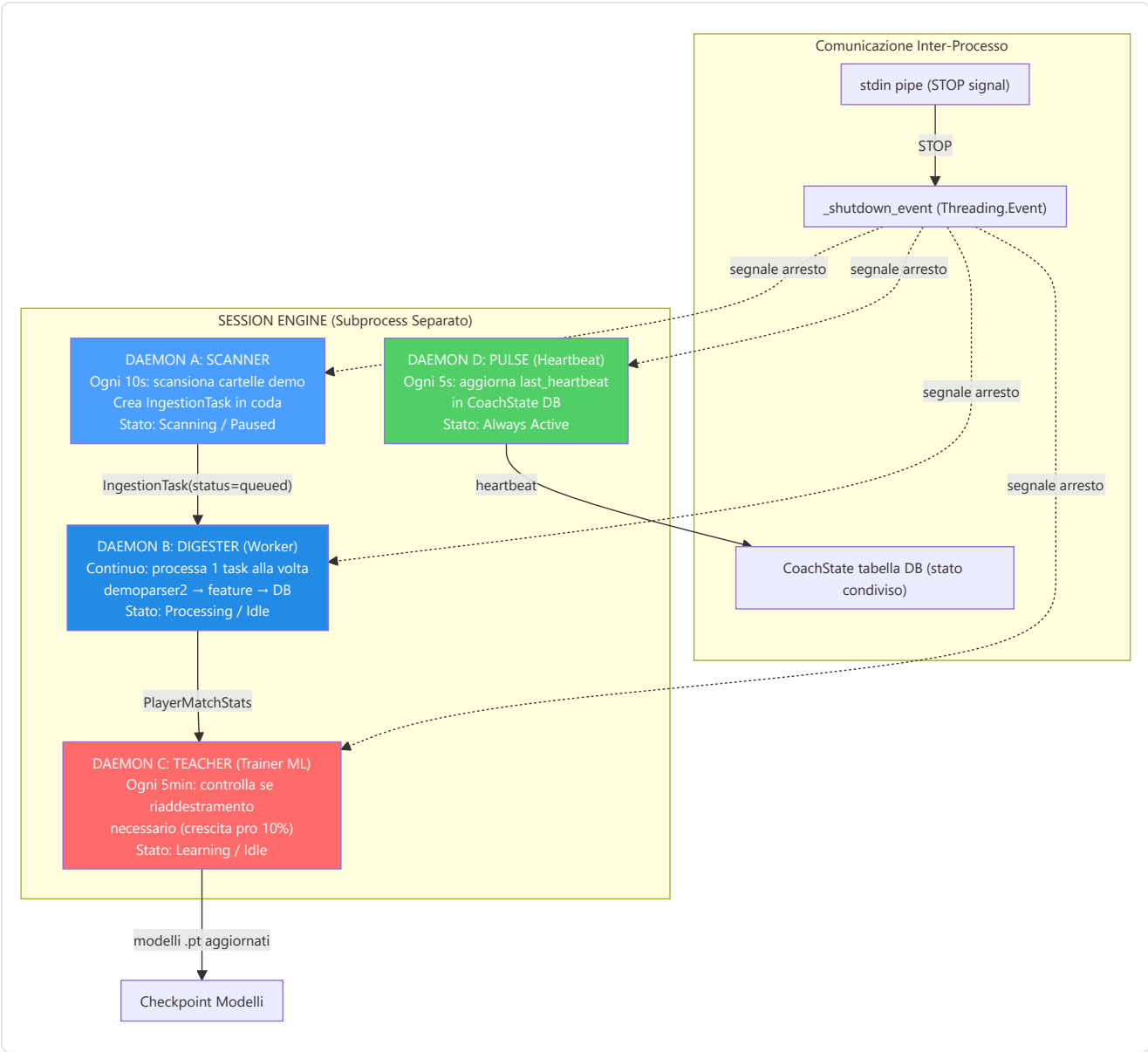
Il sistema gestisce percorsi con un'attenzione particolare alla **portabilità Windows/Linux**. Il cuore è `BRAIN_DATA_ROOT`: una directory configurabile dall'utente che contiene modelli, log e dati derivati. Se non esiste, il sistema ricade sulla cartella del progetto.

Percorso	Contenuto	Configurabile
<code>DATABASE_URL</code>	Database monolite principale (<code>database.db</code>)	No — sempre nella cartella del progetto
<code>BRAIN_DATA_ROOT</code>	Radice per dati derivati (modelli, log)	Sì — via <code>user_settings.json</code>
<code>MODELS_DIR</code>	Checkpoint dei modelli <code>.pt</code>	Derivato da <code>BRAIN_DATA_ROOT</code>
<code>LOG_DIR</code>	File di log dell'applicazione	Derivato da <code>BRAIN_DATA_ROOT</code>
<code>MATCH_DATA_PATH</code>	Database per-match (<code>match_XXXX.db</code>)	Derivato da <code>BRAIN_DATA_ROOT</code>
<code>DEFAULT_DEMO_PATH</code>	Cartella demo dell'utente	Sì — via UI Settings
<code>PRO_DEMO_PATH</code>	Cartella demo professionali	Sì — via UI Settings

12.4 Motore di Sessione — Architettura Quad-Daemon (`session_engine.py`)

File: `Programma_CS2_RENAN/core/session_engine.py`

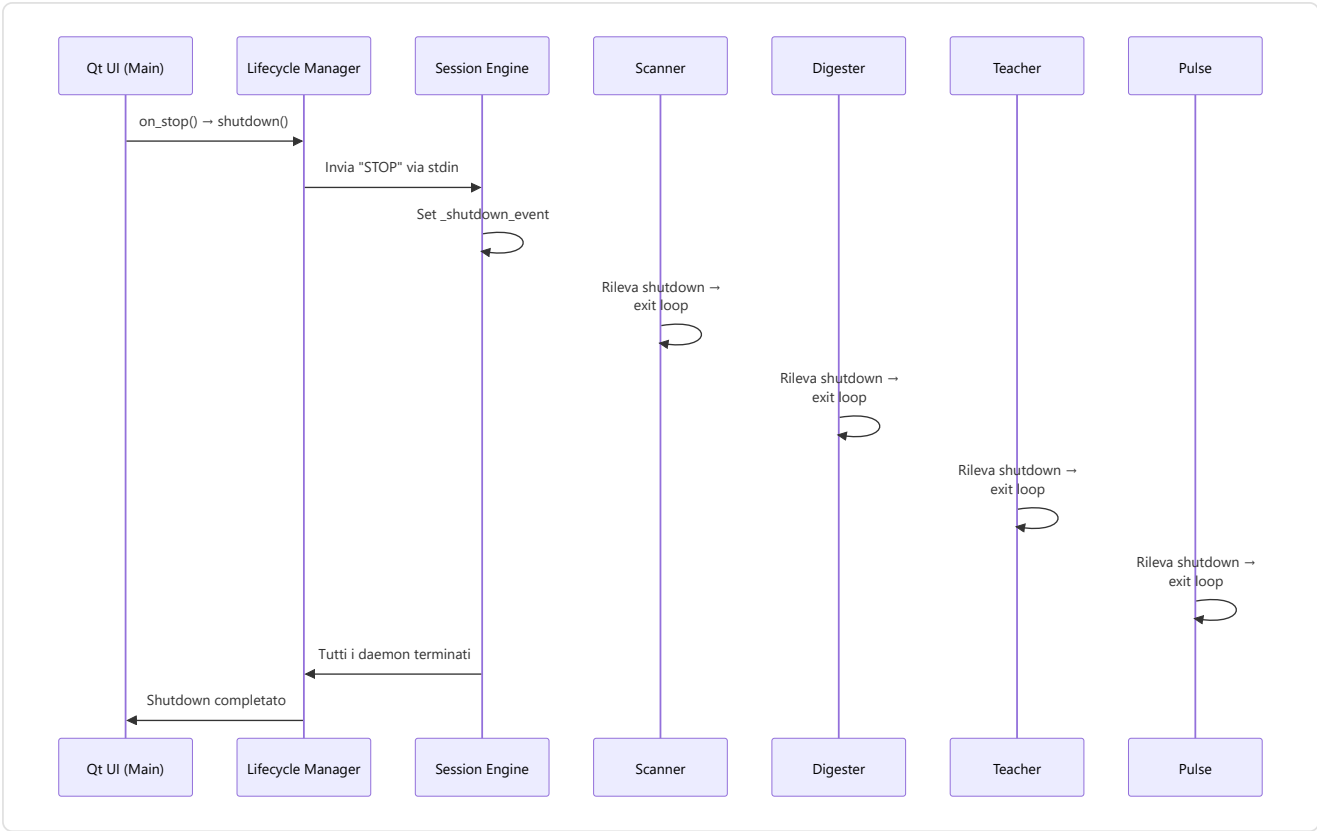
Il Session Engine è il **cuore pulsante** dell'automazione del sistema. Vive come subprocess separato e ospita **4 daemon thread** che lavorano in parallelo, ciascuno con una responsabilità ben definita. Questo design separa completamente il lavoro pesante (parsing demo, addestramento ML) dall'interfaccia utente, garantendo che la GUI Qt rimanga sempre reattiva.



Ciclo di vita di ogni daemon:

Daemon	Intervallo	Lavoro per ciclo	Trigger
Scanner	10 secondi	Scansiona cartelle pro e utente, crea <code>IngestionTask</code> per nuovi <code>.dem</code>	Sempre attivo (se stato = Scanning)
Digester	Continuo	Preleva 1 task dalla coda, esegue parsing completo	<code>_work_available_event</code> (segnalato da Scanner)
Teacher	300 secondi (5 min)	Controlla crescita sample pro; se $\geq 10\%$ → <code>run_full_cycle()</code>	<code>pro_count ≥ last_count × 1.10</code>
Pulse	5 secondi	Aggiorna <code>CoachState.last_heartbeat</code> nel database	Sempre attivo

Sequenza di shutdown:



Dettagli implementativi dei daemon:

Daemon	Metodo Principale	Loop Interno	Condizione di Uscita
Scanner	<code>_scanner_daemon_loop()</code>	<code>while not _shutdown_event.is_set() -> scan_all_paths() -> sleep(10)</code>	<code>_shutdown_event</code> set
Digester	<code>_digester_daemon_loop()</code>	<code>while not _shutdown_event.is_set() -> _work_available_event.wait(2) -> process_next_task()</code>	<code>_shutdown_event</code> set
Teacher	<code>_teacher_daemon_loop()</code>	<code>while not _shutdown_event.is_set() -> _check_retrain_needed() -> sleep(300)</code>	<code>_shutdown_event</code> set
Pulse	<code>_pulse_daemon_loop()</code>	<code>while not _shutdown_event.is_set() -> state_mgr.heartbeat() -> sleep(5)</code>	<code>_shutdown_event</code> set

Gestione errori nei daemon:

Ogni daemon è protetto da un `try/except` globale. Se un daemon crasha:

1. L'errore viene loggato con traceback completo
2. Lo `StateManager` registra l'errore (`set_error(daemon, message)`)
3. Una `ServiceNotification` viene creata per l'utente
4. Un **watchdog** del Session Engine controlla ogni 30 secondi i thread daemon e **riavvia automaticamente** quelli morti
5. Gli altri daemon continuano a funzionare indipendentemente

Zombie Task Cleanup: All'avvio, il Session Engine cerca task con `status="processing"` rimasti da un crash precedente e li resetta a `status="queued"` , consentendo il ripristino automatico senza perdita di dati.

Backup Automatico: All'avvio del Session Engine, `BackupManager.should_run_auto_backup()` verifica se è necessario un backup e, in caso affermativo, crea un checkpoint con etichetta `"startup_auto"` . Il backup segue una rotazione di 7 copie giornaliere + 4 settimanali.

12.5 Interfaccia Desktop

L'interfaccia desktop è costruita con **Qt/PySide6** seguendo il pattern **MVVM** (Model-View-ViewModel).

12.5.1 Interfaccia Qt — `apps/qt_app/`

Directory: `Programma_CS2_RENAN/apps/qt_app/` **File chiave:** `app.py`, `main_window.py`, `core/i18n_bridge.py`, `core/theme_engine.py`, `screens/`

L'interfaccia Qt è costruita con **PySide6 (Qt 6)** e utilizza un pattern **MVVM con Qt Signals/Slots**. La `MainWindow` (`QMainWindow`) è composta da una **sidebar di navigazione** e un `QStackedWidget` che ospita le 15 schermate.

Specifica	Dettaglio
Framework	PySide6 (Qt 6 per Python)
Pattern	MVVM con Qt Signals/Slots
Piattaforme	Windows, macOS, Linux
Risoluzione	Adattiva, High-DPI nativo
Temi	3: CS2 (arancione), CSGO (blu-grigio), CS1.6 (verde) — un unico template QSS parametrizzato da design token
i18n	3 lingue: EN, IT, PT — JSON + <code>QtLocalizationManager</code> , 572 chiavi per lingua
Grafici	QPainter puro ovunque: mappa tattica e tutti i widget grafici

Sistema i18n: Il `QtLocalizationManager` (`core/i18n_bridge.py`) carica file JSON per lingua (`en.json`, `it.json`, `pt.json`) e gestisce il cambio lingua a runtime tramite segnali Qt. Ogni stringa UI viene risolta dinamicamente tramite chiave di localizzazione.

Sistema temi: dall'agosto 2026 i tre temi non sono più tre fogli di stile. La cartella `apps/qt_app/themes/` contiene **un solo file**, `base.qss.template`, e il colore arriva da una pipeline in tre stadi:

- `design/tokens/design-tokens.json` è la fonte di verità dei colori. `tools/gen_design_tokens.py` lo compila in `core/design_tokens.py`, un modulo **generato** — da non modificare a mano — che espone tre istanze congelate di `DesignTokens`: `CS2_TOKENS`, `CSGO_TOKENS`, `CS16_TOKENS`.
- `core/qss_generator.py` esegue `Template(base.qss.template).safe_substitute(asdict(tokens))` e tiene in cache un foglio di stile già reso per ciascun nome di tema.
- `core/theme_engine.py` applica quel QSS e poi costruisce la `QPalette` **dalla stessa istanza di token**, così foglio di stile e palette non possono più divergere.

Il guadagno pratico è che aggiungere un tema significa aggiungere un blocco di token, non riscrivere un foglio di stile da centinaia di righe; e un colore corretto in un punto si propaga insieme a QSS, `QPalette` e `rating_color()`.

Tema	Accento	Superficie di base	Ispirazione
CS2 (default)	#FF6A00 arancione	#0B1628 blu notte	UI moderna di Counter-Strike 2
CSGO	#617D8C blu-grigio	#1A1C21 grigio quasi nero	Counter-Strike: Global Offensive
CS1.6	#4DB04F verde	#121A12 verde-nero	Counter-Strike 1.6 classico

Il relay di tema. Un `ThemeEngine` vive quanto un avvio dell'applicazione. I widget che si ricoloravano da soli non avevano quindi un oggetto stabile a cui iscriversi, e dopo un cambio tema restavano del colore vecchio. La soluzione è un **relay a livello di modulo**, `get_theme_relay()`: i widget con stile applicato per istanza — chip di filtro e di stato, card del roster, banner — si iscrivono al relay, che sopravvive ai singoli motori, e Qt scollega da solo i destinatari distrutti.

Web Views (`apps/qt_app/web/`):

L'interfaccia Qt include **3 web app React** (React 18 + TypeScript + Vite, workspace pnpm) renderizzate in `QWebEngineView` :

Web View	Directory	Descrizione
Coach Chat	<code>web/coach-chat/</code>	Interfaccia chat con il coach AI — formattazione rich text
Match Detail	<code>web/match-detail/</code>	Visualizzazione dettagliata di una partita con timeline e statistiche per round
Tactical Viewer	<code>web/tactical-viewer/</code>	Viewer tattico a layer (<code>MapCanvas</code> , <code>PlayerLayer</code> , <code>GhostLayer</code> , <code>HeatmapLayer</code> , <code>TrailsLayer</code> , <code>RoundTimeline</code> , <code>ControlBar</code>)

La comunicazione bidirezionale tra il backend Python e le web app passa per `QWebChannel` (`web/shared/qwebchannel.ts` + `bridge.ts` per app), esponendo metodi Python come API JavaScript. Questa architettura ibrida permette di utilizzare librerie di visualizzazione web mantenendo la logica di business nel backend Python.

15 schermate dell'interfaccia:

Schermata	Ruolo	Componenti chiave
Wizard	Prima configurazione	Nome giocatore, ruolo, percorsi cartelle demo
Home	Dashboard	Quota mensile (X/10), stato servizi (verde/rosso), fiducia credenze (0-1), task attivi, contatore partite processate
Coach	Insight coaching	Card colorate per severità, radar skill multi-dimensionale, trend storici, chat AI (Ollama/Claude), task attivi
Tactical Viewer	Riproduzione tattica	Mappa 2D con giocatori/granate/fantasma, timeline con marcatori eventi, sidebar giocatori CT/T, controlli velocità (0.25x→8x)
Settings	Personalizzazione	Tema (CS2/CSGO/CS1.6), font, dimensione testo, lingua, percorsi demo, wallpaper
Help	Supporto utente	Tutorial interattivo, FAQ, troubleshooting
Match History	Storico partite	Lista demo analizzate con filtri e ordinamento
Match Detail	Dettaglio partita	Statistiche dettagliate per una singola demo analizzata
Performance	Progressi	Radar skill a 5 assi, grafici di tendenza, confronti temporali
User Profile	Profilo utente	Bio, ruolo preferito, sincronizzazione Steam/FACEIT
Profile	Profilo pubblico	Visualizzazione profilo pubblico del giocatore
Steam Config	Configurazione Steam	Inserimento e validazione API key Steam
Pro Comparison	Confronto pro	Confronto statistiche utente con giocatori professionisti HLTV, benchmark prestazionale
Pro Player Detail	Dettaglio giocatore pro	Profilo individuale del giocatore professionista HLTV con statistiche complete della carriera
FACEIT Config	Configurazione FACEIT	Inserimento e validazione API key FACEIT

Widget personalizzati:

Widget	File	Funzione
<code>PlayerSidebar</code>	<code>tactical/player_sidebar.py</code>	Lista CT/T con icone ruolo, salute/armatura, arma corrente, denaro e stato vivo/morto (<code>_PlayerItem</code> , <code>_StatBar</code>)
<code>TacticalMapWidget</code>	<code>tactical/map_widget.py</code>	Canvas 2D con rendering multilivello: texture mappa → zone → heatmap → giocatori → granate → fantasma (QPainter)
<code>TimelineWidget</code>	<code>tactical/timeline_widget.py</code>	Scrubber orizzontale con numeri di tick, glifi evento (stella, rombo), drag-to-seeek e salto al doppio clic
<code>with_alpha</code>	<code>tactical/_paint_utils.py</code>	Helper condiviso di disegno: applica un canale alfa a un <code>QColor</code> senza duplicare la logica in ogni widget

Widget grafici (`widgets/charts/`):

QtCharts è stato **ritirato** per una ragione di licenza, non di gusto: è distribuito sotto GPLv3, incompatibile con la distribuzione di questo progetto. I grafici sono stati riscritti in QPainter puro conservando l'API pubblica, così le schermate che li usavano non hanno dovuto cambiare. Un test di gate (`tests/test_charts.py` , classe `TestQtChartsRetired`) fallisce se un solo riferimento a `QtCharts` o `QChart` rientra nel codice.

Tutti leggono i token dentro `paintEvent` , quindi si ridisegnano del colore giusto dopo un cambio tema.

Widget	File	Funzione
<code>RadarChart</code>	<code>radar_chart.py</code>	Radar delle abilità a N assi ($N \geq 3$), griglia a poligoni concentrici, una figura piena per serie — usato per sovrapporre utente e professionista
<code>RatingSparkline</code>	<code>rating_sparkline.py</code>	Andamento del rating con linee di riferimento HLTV tratteggiate a 0,90 / 1,00 / 1,10 e didascalie a bordo destro
<code>UtilityBarChart</code>	<code>utility_bar_chart.py</code>	Barre raggruppate utente-vs-pro (<code>set_rows</code>) oppure una barra per riga con scala a tacche (<code>set_single</code>)
<code>EconomyChart</code>	<code>economy_chart.py</code>	Valore di equipaggiamento per round, barre colorate dal lato di quel round; <code>set_half_marker()</code> traccia il cambio di metà
<code>MomentumChart</code>	<code>momentum_chart.py</code>	Barre dello scarto uccisioni-morti per round, normalizzate sullo scarto massimo
<code>MiniSparkline</code>	<code>mini_sparkline.py</code>	Forma di tendenza minima, senza assi né cornice, per stare dentro una card

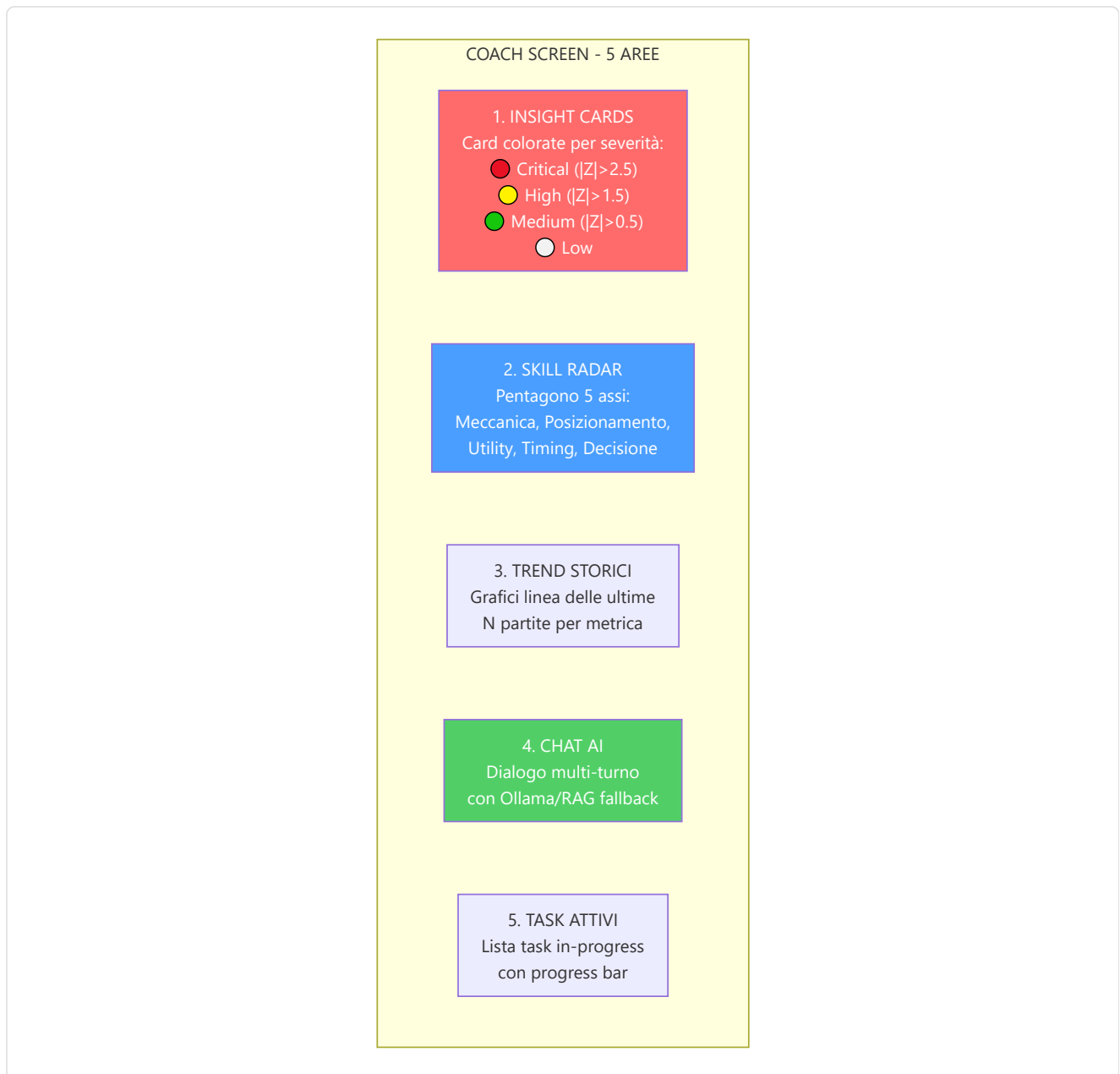
Libreria di componenti (`widgets/components/`):

26 componenti riusabili costruiti sull'atlante di design. I più caratterizzanti: `MapTile` (riquadro per mappa con barra di avanzamento e etichetta di accessibilità), `DeltaChip` (scarto rispetto a un riferimento — *il confronto è l'informazione*), `ProBadge` , `MetricBarRow` , `DbRecordCard` , `TipBox` , `NumberedStep` , `DriversList` , `MonoFooter` (riga di provenienza del dato: da quale tabella e da quale colonna arriva il numero mostrato).

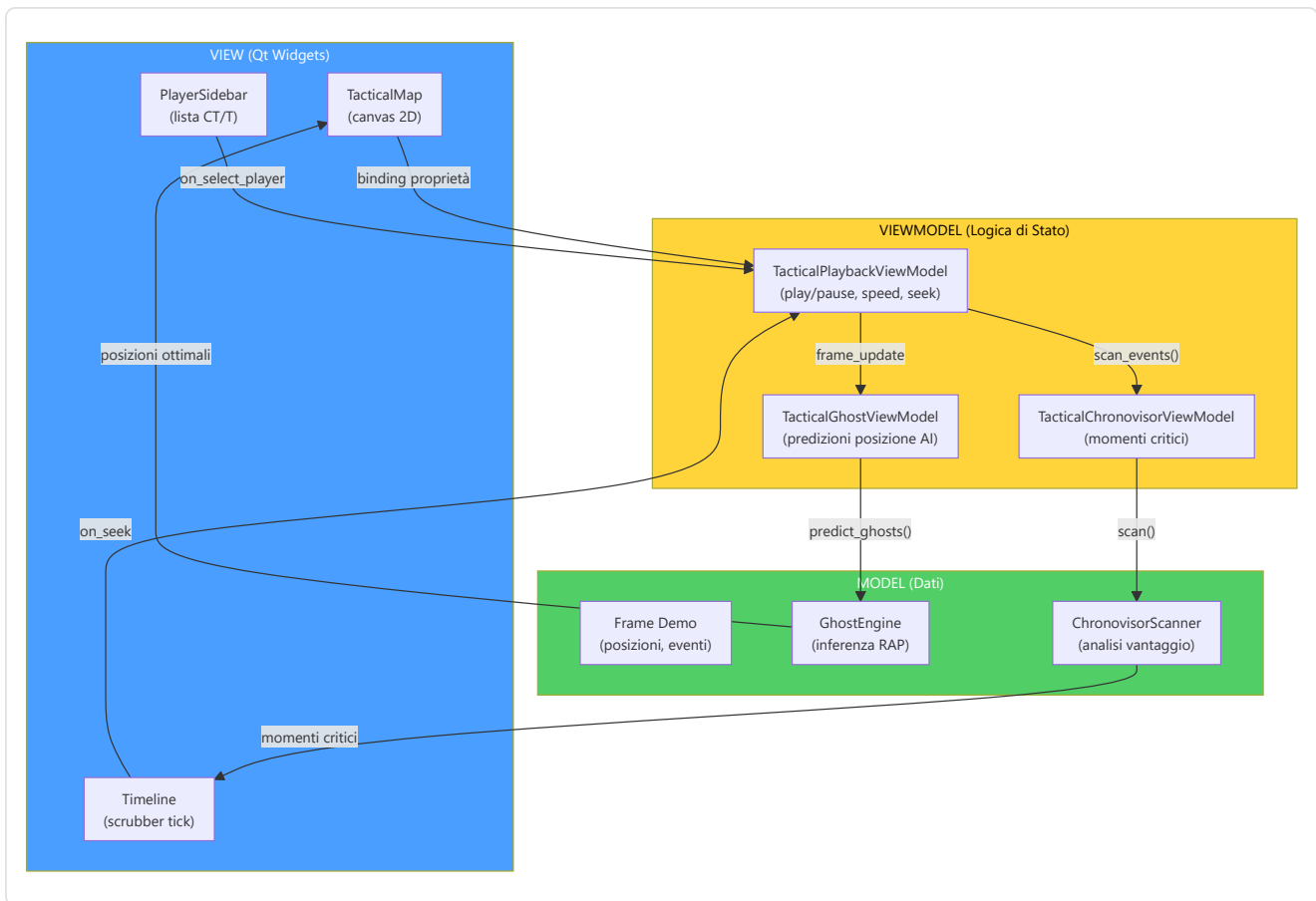
Il pannello di chat del coach (`widgets/coaching/chat_panel.py`) è deliberatamente **agnostico rispetto al ViewModel**: non parla mai direttamente con un VM, è una schermata a cablarlo (`panel.message_submitted` → `vm.send_message` , `vm.messages_changed` → `panel.add_message`). Così lo stesso pannello può servire contesti diversi senza sapere nulla di chi lo alimenta.

Coach Screen — Layout dettagliato:

La schermata Coach è la più complessa dell'applicazione, con 5 aree funzionali:



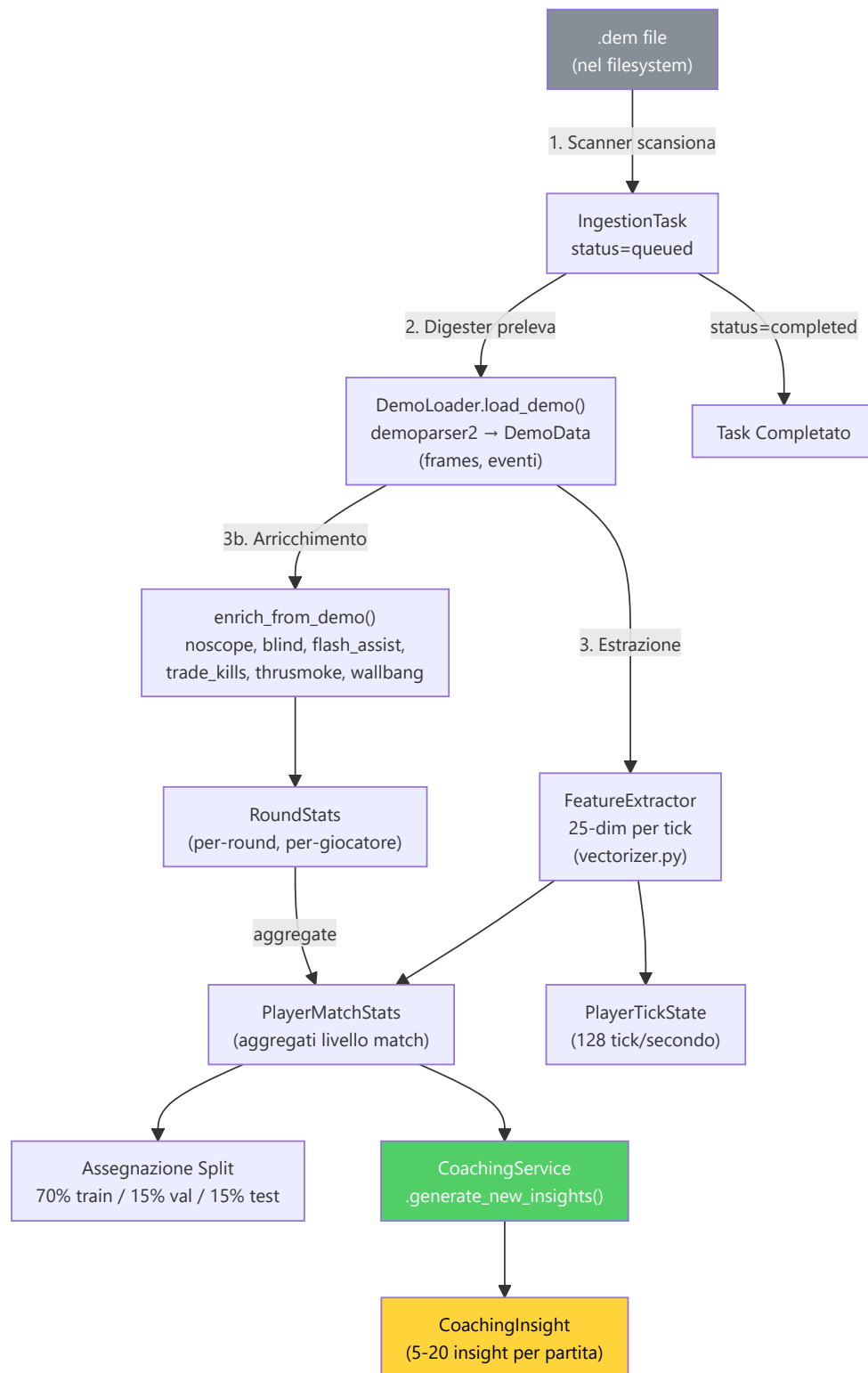
Pattern MVVM nel Tactical Viewer:



12.6 Pipeline di Ingestione (`ingestion/`)

Directory: `Programma_CS2_RENAN/ingestion/` **File chiave:** `demo_loader.py`, `steam_locator.py`, `integrity.py`, `registry/`, `pipelines/user_ingest.py`, `pipelines/json_tournament_ingestor.py`

La pipeline di ingestione è il **percorso completo** che un file `.dem` compie dal filesystem fino a diventare insight di coaching nel database. È orchestrata dal daemon Scanner (scoperta) e dal daemon Digester (elaborazione).



Chi prende in carico il task, e perché uno solo.

Il passo 2 del diagramma — *"il Digester preleva"* — nasconde un problema di concorrenza che è costato dati duplicati. Le superfici che possono avviare un'ingestione sono **sei**: la schermata Home, le impostazioni, il comando di ingest della console, `batch_ingest`, `ingest_pro_demos` e `run_worker`. Il percorso vecchio faceva una `SELECT` della coda e poi scriveva `status='processing'` senza condizioni: due runner avviati a poca distanza leggevano la **stessa fotografia** della coda, entrambi la ritenevano propria, e la stessa demo veniva analizzata due volte scrivendo statistiche duplicate.

La correzione è una sola istruzione SQL, e la sua forza sta nella clausola **WHERE** :

```
_sa_update(IngestionTask)
    .where(IngestionTask.id == task_id, IngestionTask.status == "queued")
    .values(status="processing", updated_at=...)
# rowcount == 1 → il task è nostro; rowcount == 0 → l'ha preso un altro
```

L' **UPDATE ... WHERE status='queued'** è atomico a livello di database: **esattamente un runner vince ogni task**, gli altri leggono **rowcount == 0** e passano oltre in silenzio. Non serve un lock applicativo, non serve coordinamento fra processi — la condizione di corsa viene eliminata invece che gestita, che è sempre la soluzione preferibile. Un test dedicato (**test_ingestion_atomic_claim.py**) verifica sia il claim esclusivo sia il fatto che un task già reclamato venga saltato.

Alla chiusura del cerchio ci pensa **run_worker.py** : se un task viene reclamato ma poi scartato, **_release_claim()** lo riporta a **queued** invece di lasciarlo bloccato in **processing** per sempre.

DemoLoader — Il parser del cuore della pipeline:

Il **DemoLoader** è il wrapper attorno a **demoparser2** (libreria Rust ad alte prestazioni) che trasforma un file **.dem** binario in strutture dati Python:

Fase di Parsing	Output	Dimensione tipica
1. Header parsing	Metadata (map, server, duration)	~100 bytes
2. Frame extraction	Lista di frame (posizione, salute, arma per ogni tick)	~100.000 frame/partita
3. Event extraction	Lista eventi (kill, death, bomb_plant, round_start, etc.)	~500-2.000 eventi/partita
4. Player summary	Statistiche aggregate per giocatore	~10 record

FeatureExtractor (**backend/processing/feature_engineering/vectorizer.py**):

Il FeatureExtractor è il componente che trasforma i dati grezzi del demo in **vettori numerici a 25 dimensioni** (**METADATA_DIM=25**) utilizzabili dalle reti neurali. **Importante:** queste sono feature **a livello di tick** (128 Hz), non statistiche aggregate a livello di partita. Ogni singolo frame di gioco produce un vettore 25-dim che cattura lo stato istantaneo del giocatore:

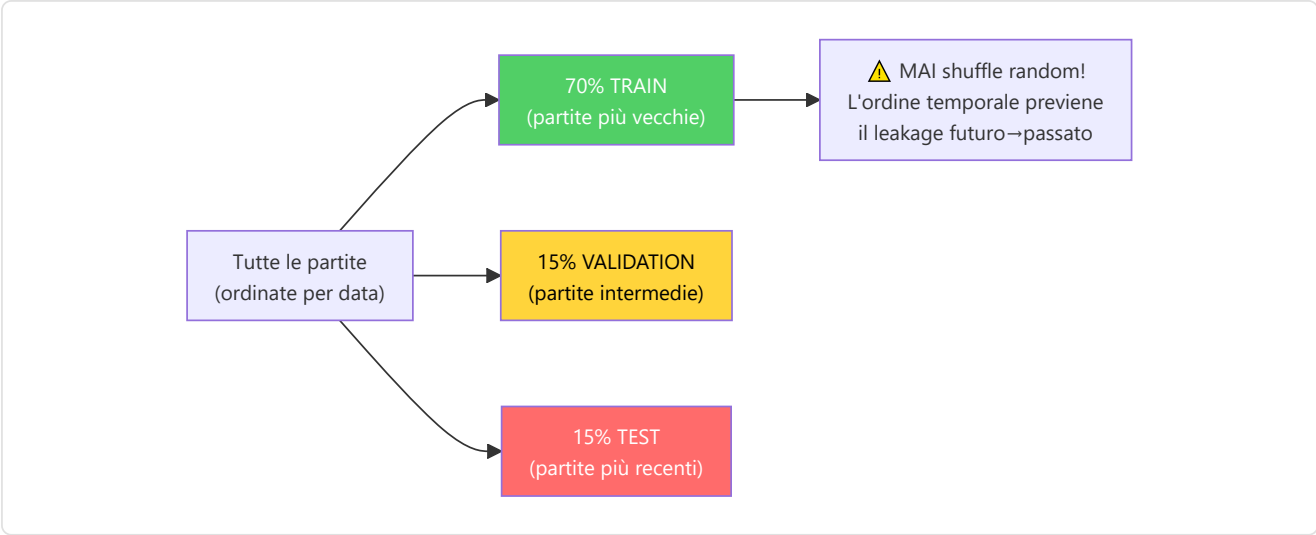
Dim	Feature	Tipo	Range	Descrizione
0	health	Float	[0, 1]	Salute normalizzata
1	armor	Float	[0, 1]	Armatura normalizzata
2	has_helmet	Binary	0/1	Casco equipaggiato
3	has_defuser	Binary	0/1	Kit defuse equipaggiato
4	equipment_value	Float	[0, 1]	Valore equipaggiamento normalizzato
5	is_crouching	Binary	0/1	Accucciato
6	is_scoped	Binary	0/1	Mirino attivo (scope)
7	is_blinded	Binary	0/1	Accecato da flash
8	enemies_visible	Float	[0, 1]	Nemici visibili (normalizzato, clamped)
9	pos_x	Float	[-1, 1]	Posizione X (normalizzata $\pm pos_{xy_extent}$)
10	pos_y	Float	[-1, 1]	Posizione Y (normalizzata $\pm pos_{xy_extent}$)
11	pos_z	Float	[0, 1]	Posizione Z (normalizzata, gestisce Nuke/Vertigo)
12	view_x_sin	Float	[-1, 1]	$\sin(yaw)$ — continuità ciclica angolo orizzontale
13	view_x_cos	Float	[-1, 1]	$\cos(yaw)$ — continuità ciclica angolo orizzontale
14	view_y	Float	[-1, 1]	Pitch normalizzato (angolo verticale)
15	z_penalty	Float	[0, 1]	Distinzione livello verticale (penalità piano)
16	kast_estimate	Float	[0, 1]	Stima KAST (rapporto partecipazione)
17	map_id	Float	[0, 1]	Hash deterministico della mappa
18	round_phase	Float	{0, 0.33, 0.66, 1}	Fase economica: pistol/eco/force/full_buy
19	weapon_class	Float	{0–1.0}	Classe arma: 0=coltello, 0.2=pistola, 0.4=SMG, 0.6=fucile, 0.8=sniper, 1.0=pesante
20	time_in_round	Float	[0, 1]	Secondi nel round / 115 (clamped)
21	bomb_planted	Binary	0/1	Bomba piazzata
22	teammates_alive	Float	[0, 1]	Compagni vivi (count / 4)
23	enemies_alive	Float	[0, 1]	Nemici vivi (count / 5)
24	team_economy	Float	[0, 1]	Media soldi squadra / 16000 (clamped)

Normalizzazione e bounds:

La normalizzazione è integrata direttamente nel FeatureExtractor, con bounds configurabili tramite `HeuristicConfig` (esternalizzata in JSON). La codifica ciclica degli angoli di vista ($\sin/\cos$ per lo yaw) previene discontinuità ai bordi $0^\circ/360^\circ$. Il `z_penalty` distingue automaticamente i piani in mappe multilivello (Nuke, Vertigo). La classe arma utilizza una mappatura categorica ordinale (6 classi) definita nella costante `WEAPON_CLASS_MAP` (include anche granate=0.1 e equipaggiamento speciale=0.05).

Dataset Split Temporale:

La suddivisione del dataset segue un rigido **ordinamento cronologico** per prevenire il data leakage temporale:



Enrich From Demo — Arricchimento post-parsing:

Dopo il parsing base, `enrich_from_demo()` aggiunge metriche avanzate calcolate dagli eventi:

Metrica arricchita	Calcolo	Fonte
Trade kills	<code>TradeKillDetector.detect()</code> con <code>TRADE_WINDOW_TICKS=192</code>	Eventi kill/death
Flash assists	Conteggio blind entro finestra temporale prima di un kill	Eventi blind + kill
Noscope kills	Kill con arma sniper senza scope attivo	Evento kill + weapon state
Wallbang kills	Kill attraverso superfici penetrabili	Evento kill con flag penetration
Through-smoke kills	Kill con fumo attivo nella linea di tiro	Evento kill + smoke position
Blind kills	Kill mentre il giocatore è flashato	Evento kill + flash state

Componenti specifici:

Componente	File	Ruolo
DemoLoader	<code>demo_loader.py</code>	Wrappa <code>demoparser2</code> , estrae frame e eventi dal file <code>.dem</code>
SteamLocator	<code>steam_locator.py</code>	Localizza automaticamente la cartella demo di CS2 via registro Steam / <code>libraryfolders.vdf</code>
IntegrityChecker	<code>integrity.py</code>	Verifica che i file demo siano validi, completi e non corrotti prima del parsing
UserIngestPipeline	<code>pipelines/user_ingest.py</code>	Pipeline completa per demo utente: parse → enrich → stats → coaching
JsonTournamentIngestor	<code>pipelines/json_tournament_ingestor.py</code>	Importa dati torneo da file JSON strutturati
Registry	<code>registry/registry.py</code>	<code>DemoRegistry</code> : traccia tutte le demo processate, previene duplicati
ResourceManager	<code>backend/ingestion/resource_manager.py</code>	Gestione risorse hardware: CPU/RAM throttling, spazio disco
RegistryLifecycle	<code>registry/lifecycle.py</code>	<code>DemoLifecycleManager</code> : cleanup demo vecchie (default 30 giorni)

SteamLocator (`ingestion/steam_locator.py`) — localizzazione automatica demo CS2:

Il SteamLocator implementa un algoritmo di **discovery cross-platform** per trovare automaticamente la cartella delle demo di CS2:

Piattaforma	Strategia	Percorso tipico
Windows	Registro di sistema → <code>libraryfolders.vdf</code>	<code>C:\Program Files (x86)\Steam\steamapps\common\Counter-Strike Global Offensive\game\csgo\replays</code>
Linux	<code>~/.steam/steam/</code> → <code>libraryfolders.vdf</code>	<code>~/.steam/steam/steamapps/common/ ...</code>
Fallback	Chiede all'utente via UI Settings	Percorso personalizzato

IntegrityChecker (`ingestion/integrity.py`):

Verifica preliminare di ogni file demo prima del parsing costoso:

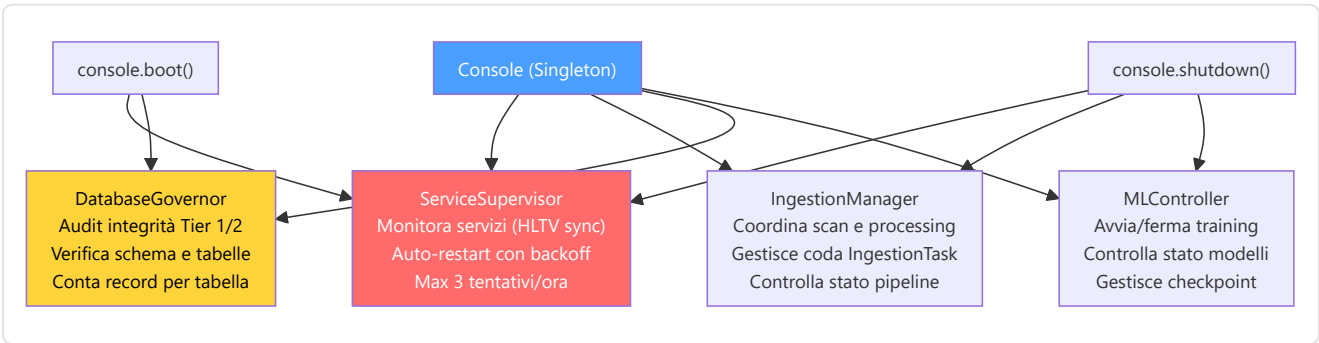
- **Hash:** `compute_sha256()` per identità e deduplicazione
- **Size bounds:** minimo 50KB, massimo 900MB (`validate_dem_file`)
- **Read test:** verifica che il file sia leggibile prima del parsing

(Livelli di validazione paralleli: `demo_format_adapter.py` usa bounds 10MB–5GB con magic bytes `PBDEMS2\0 / HL2DEMO\0` ; `validation/dem_validator.py` usa 100KB–800MB.)

12.7 Console di Controllo Unificata (`backend/control/`)

File: `Programma_CS2_RENAN/backend/control/console.py` , `ingest_manager.py` , `db_governor.py` , `ml_controller.py`

La Console è un **Singleton** che funge da punto di coordinamento centrale per tutti i sottosistemi backend. È il "quadro di comando" attraverso cui ogni parte del sistema può essere controllata.

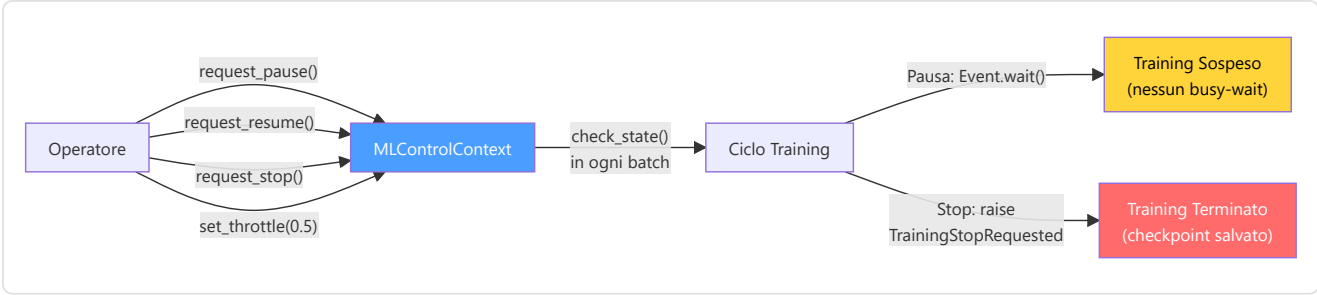


Sequenza di boot della Console:

1. **ServiceSupervisor** avvia il servizio "hunter" (HLTV sync) come processo monitorato
2. **DatabaseGovernor** esegue un audit di integrità: verifica tutte le tabelle, conta record, controlla schema
3. **MLController** resta in stato di attesa — il training è gestito dal daemon Teacher
4. **IngestionManager** resta idle — il lavoro attivo è gestito dai daemon Scanner/Digester

MLControlContext — Controllo Live dell'Addestramento:

L' **MLController** utilizza un token di controllo chiamato **MLControlContext** (`backend/control/ml_controller.py`) che viene passato ai cicli di addestramento per consentire **intervento in tempo reale** da parte dell'operatore. Questo sostituisce l'approccio precedente basato su **StopIteration** con un sistema thread-safe più robusto.

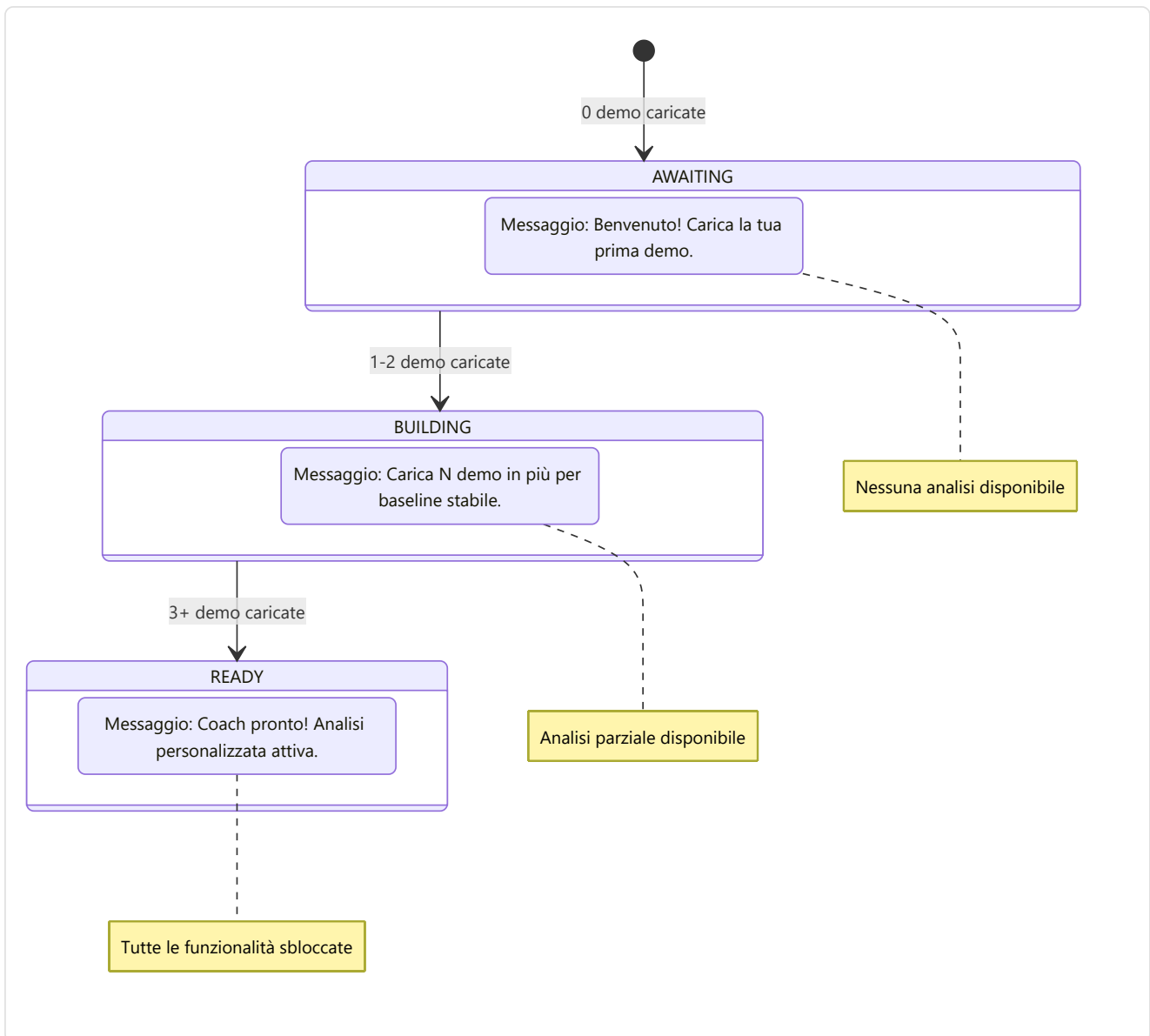


Comando	Metodo	Effetto
Pausa	<code>request_pause()</code>	<code>_resume_event.clear()</code> → blocca <code>check_state()</code>
Riprendi	<code>request_resume()</code>	<code>_resume_event.set()</code> → sblocca il training
Stop	<code>request_stop()</code>	Lancia <code>TrainingStopRequested</code> (eccezione custom)
Throttle	<code>set_throttle(factor)</code>	Aggiunge <code>time.sleep(factor)</code> dopo ogni batch

12.8 Onboarding e Flusso Nuovo Utente

File: `Programma_CS2_RENAN/backend/onboarding/new_user_flow.py`

L' `OnboardingManager` guida i nuovi utenti attraverso una **progressione a 3 fasi** che si adatta automaticamente alla quantità di dati disponibili.



Wizard Screen (Prima Configurazione):

Alla prima esecuzione (`SETUP_COMPLETED = False`), l'utente viene guidato attraverso il Wizard:

1. **Nome giocatore** — Il nome che apparirà nelle analisi
2. **Ruolo preferito** — Entry Fragger, AWPer, Lurker, Support o IGL
3. **Percorsi cartelle demo** — Dove il sistema cerca automaticamente le demo
4. Al completamento: `SETUP_COMPLETED = True` , redirect alla Home

Cache delle quote (Task 2.16.1): Il conteggio demo è cachato per 60 secondi per evitare query DB ripetute.

`invalidate_cache()` viene chiamato dopo ogni nuovo upload, garantendo che la UI mostri sempre il conteggio corretto senza sovraccaricare il database.

Help System (`backend/knowledge/help_system.py`):

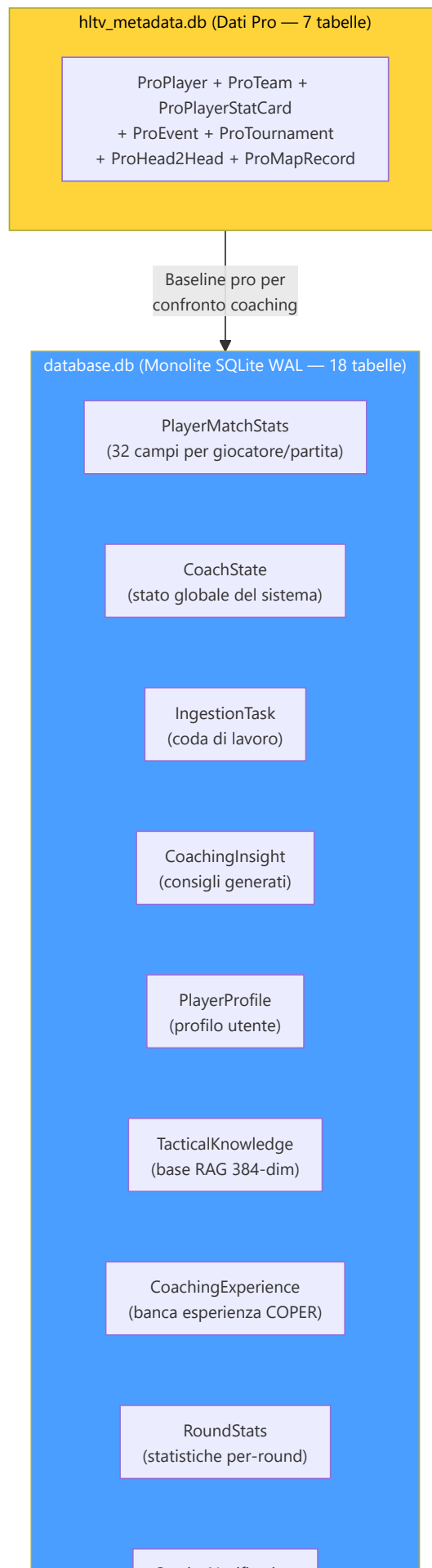
Il sistema di aiuto integrato fornisce supporto contestuale all'utente:

Funzionalità	Implementazione
Tutorial interattivo	Guide step-by-step per le funzionalità principali
FAQ contestuali	Domande frequenti filtrate per schermata corrente
Troubleshooting	Albero decisionale per problemi comuni (Steam path non trovato, demo non parsata, coaching vuoto)
Tooltips	Spiegazioni inline per metriche complesse (KAST, HLTV 2.0 Rating, ADR)

12.9 Architettura di Storage (`backend/storage/`)

Directory: `Programma_CS2_RENAN/backend/storage/` **File chiave:** `database.py` , `db_models.py` , `match_data_manager.py` , `storage_manager.py` , `maintenance.py` , `state_manager.py` , `stat_aggregator.py` , `backup_manager.py` , `db_backup.py` , `db_migrate.py` , `remote_file_server.py`

Il sistema di storage utilizza un'architettura **three-tier storage** basata su SQLite in modalità WAL (Write-Ahead Logging), che consente letture e scritture concorrenti senza blocchi.





Le 25 tabelle `SQLModel` di `db_models.py` (+ 3 per-match in `match_data_manager.py` = 28 totali):

#	Tabella	Database	Categoria	Descrizione
1	PlayerMatchStats	database.db	Core	Statistiche aggregate per giocatore/partita
2	PlayerTickState	database.db	Core	Stato per-tick, anche archiviato in DB per-match separati
3	PlayerProfile	database.db	Utente	Profilo utente (nome, ruolo, Steam ID, quota mensile)
4	RoundStats	database.db	Core	Statistiche isolate per round (uccisioni, valutazione, arricchimento)
5	CoachingInsight	database.db	Coaching	Consigli generati dal servizio di coaching
6	CoachingExperience	database.db	Coaching	Banca esperienze COPER (contesto, esito, efficacia, TrueSkill μ/σ , replay priority — KT-01)
7	IngestionTask	database.db	Sistema	Coda di lavoro per il daemon Digester
8	CoachState	database.db	Sistema	Stato globale (training metrics, heartbeat, status)
9	ServiceNotification	database.db	Sistema	Messaggi di errore/evento dei daemon → UI
10	TacticalKnowledge	database.db	Conoscenza	Base RAG (embedding 384-dim in JSON)
11	ProPlayer	hltv_metadata.db	Pro	Profili giocatori professionisti
12	ProTeam	hltv_metadata.db	Pro	Metadata squadre professionali
13	ProPlayerStatCard	hltv_metadata.db	Pro	Statistiche stagionali per giocatore pro
14	ProEvent	hltv_metadata.db	Pro	Eventi/competizioni HLTV
15	ProTournament	hltv_metadata.db	Pro	Tornei HLTV
16	ProHead2Head	hltv_metadata.db	Pro	Scontri diretti tra giocatori
17	ProMapRecord	hltv_metadata.db	Pro	Record per mappa
18	Ext_PlayerPlaystyle	database.db	Esterno	Dati stile di gioco da CSV (per NeuralRoleHead)
19	Ext_TeamRoundStats	database.db	Esterno	Statistiche torneo esterne
20	MatchResult	database.db	Partite	Esiti delle partite
21	MapVeto	database.db	Partite	Storico selezione mappe
22	CalibrationSnapshot	database.db	Sistema	Registro di calibrazione del modello di credenza (timestamp, campioni, risultato)
23	RoleThresholdRecord	database.db	Sistema	Soglie apprese per la classificazione dei ruoli (persistite tra i riavvii)
24	DataLineage	database.db	Provenienza	Registro append-only di provenienza dati: entity_type, entity_id, source_demo, pipeline_version, processing_step
25	DataQualityMetric	database.db	Provenienza	Metriche qualità append-only per run: run_id, run_type, metric_name, metric_value, sample_count

Enum di supporto (non tabelle):

Enum	Tipo	Descrizione
DatasetSplit	str, Enum	Categorie split (train/val/test/unassigned) — usato come constraint su <code>PlayerMatchStats.dataset_split</code>
CoachStatus	str, Enum	Stati del coach (Paused/Training/Idle/Error) — usato come constraint su <code>CoachState.status</code>

Componenti di Storage dettagliati:

MatchDataManager (`backend/storage/match_data_manager.py`) — il componente più grande dello storage layer:

Il MatchDataManager è responsabile della gestione dei dati per-partita ad alta densità (PlayerTickState con ~100.000 righe per partita). Per evitare che il database principale cresca in modo incontrollato, ogni partita ha il proprio database SQLite separato (`match_XXXX.db`).

Metodo	Descrizione
<code>get_match_session(match_id)</code>	Context manager transazionale sul DB per-match (engine da cache LRU)
<code>store_tick_batch(match_id, ticks)</code>	Bulk insert di <code>MatchTickState</code> nel DB dedicato
<code>store_event_batch(match_id, events)</code>	Bulk insert di <code>MatchEventState</code>
<code>store_metadata(match_id, metadata)</code>	Upsert dei metadati match
<code>list_available_matches()</code>	Elenca tutti i match con DB disponibili
<code>delete_match(match_id)</code>	Rimuove il DB per-match

Nota WR-14: il manager verifica il device-ID del volume per rilevare la disconnessione del drive esterno dei per-match DB.

StorageManager (`backend/storage/storage_manager.py`):

Il StorageManager è il **coordinatore di alto livello** dello storage che gestisce quote, upload e ciclo di vita dei dati:

Responsabilità	Implementazione
Quota mensile	<code>can_user_upload()</code> → verifica <code>MAX_DEMOS_PER_MONTH=10</code> e <code>MAX_TOTAL_DEMOS=100</code>
Upload flow	<code>handle_demo_upload(path)</code> → validazione → copia in working dir → crea IngestionTask
Pulizia	<code>cleanup_old_data(days)</code> → rimuove match DB e task vecchi
Spazio disco	<code>get_storage_usage()</code> → report dimensioni per ogni database e directory

StateManager (`backend/storage/state_manager.py`):

Il StateManager è un **Singleton** che mantiene lo stato runtime del sistema e lo persiste nel database tramite la tabella `CoachState` :

Metodo	Scopo
<code>update_status(daemon, text)</code>	Aggiorna lo stato di un daemon specifico
<code>heartbeat()</code>	Aggiorna il timestamp <code>last_heartbeat</code>
<code>get_state()</code>	Restituisce lo stato corrente come oggetto <code>CoachState</code>
<code>set_error(daemon, message)</code>	Registra un errore per un daemon con timestamp
<code>update_training_metrics(epoch, loss, val_loss, eta)</code>	Aggiorna le metriche di training in tempo reale
<code>get_belief_confidence()</code>	Restituisce il livello di fiducia del modello di credenza (0.0-1.0)

StatAggregator (`backend/storage/stat_aggregator.py`):

Calcola statistiche aggregate a partire dai dati grezzi per-round:

Aggregazione	Formula	Uso
avg_kills	mean(RoundStats.kills)	Dashboard, radar chart
avg_adr	mean(RoundStats.damage_dealt / rounds)	Confronto pro
avg_kast	mean(rounds_with_kast / total_rounds)	Metrica HLTV
accuracy	sum(hits) / sum(shots_fired)	Performance meccanica
trade_kill_rate	trade_kills / team_deaths	Lavoro di squadra

BackupManager (backend/storage/backup_manager.py):

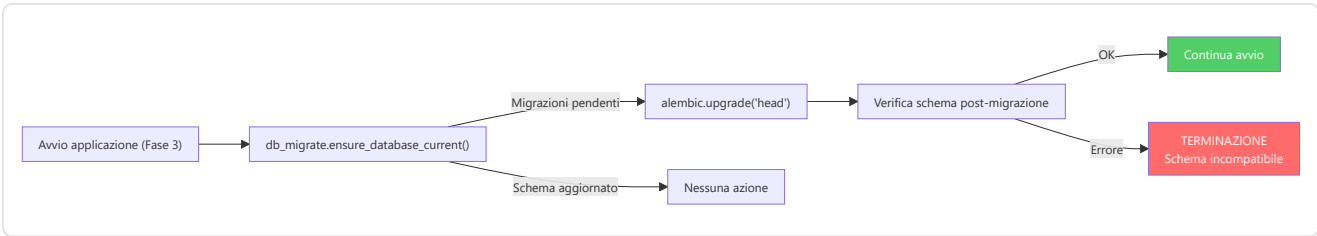
Caratteristica	Dettaglio
Meccanismo	Hot backup via SQLite Online Backup API (sqlite3.Connection.backup()), scelta al posto di VACUUM INTO per evitare interpolazione del path in SQL; sicura in WAL-mode)
Verifica	PRAGMA quick_check sulla copia (_verify_integrity , try/finally H-02)
Trigger automatico	All'avvio del Session Engine via should_run_auto_backup()
Trigger manuale	Via Console o UI Settings
Rotazione	_prune_backups() ; il modulo complementare db_backup.py (backup_monolith , backup_match_data , rotate_backups keep_count=5, restore_backup)
Etichettatura	Es. startup_auto , manual

Maintenance (backend/storage/maintenance.py):

Operazione	Frequenza	Scopo
vacuum()	Mensile	Compatta il database, recupera spazio da record eliminati
analyze()	Dopo ogni bulk insert	Aggiorna le statistiche dell'ottimizzatore query SQLite
wal_checkpoint()	All'avvio	Forza il merge del WAL nel database principale
integrity_check()	All'avvio	PRAGMA integrity_check — verifica coerenza strutturale

DbMigrate (backend/storage/db_migrate.py):

Wrapper attorno ad Alembic che automatizza l'esecuzione delle migrazioni:



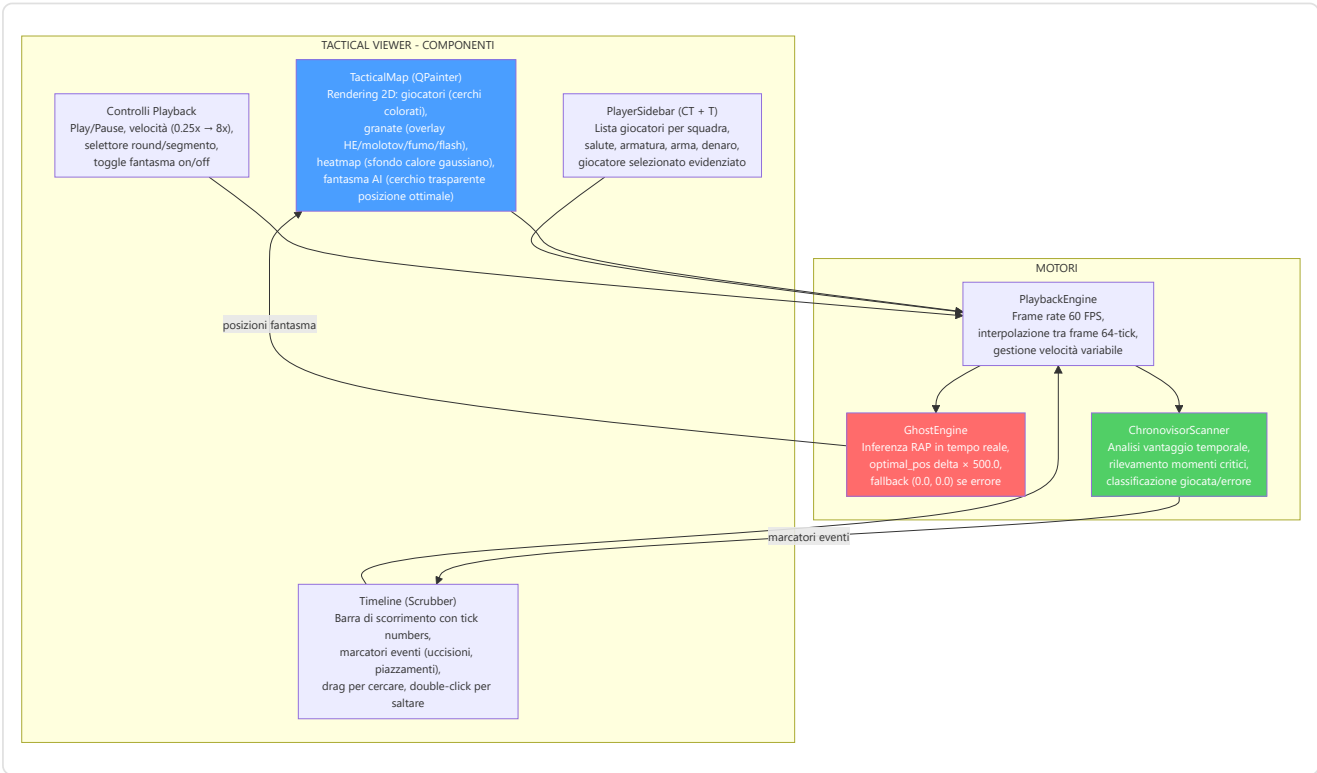
Connection Pooling e Concorrenza:

Parametro	Valore	Scopo
check_same_thread	False	Consente accesso multi-thread
timeout	30 secondi	Busy timeout per contesa WAL
pool_size	1	Singolo scrittore SQLite (sicurezza single-writer)
max_overflow	4	Connessioni overflow per picchi di carico
WAL mode	Abilitato	Letture concorrenti illimitate

12.10 Motore di Playback e Viewer Tattico

File: Programma_CS2_RENAN/core/playback_engine.py , apps/qt_app/screens/tactical_viewer_screen.py , apps/qt_app/widgets/tactical/map_widget.py , timeline_widget.py , player_sidebar.py

Il sistema di playback tattico consente all'utente di **rivivere le proprie partite** su una mappa 2D interattiva, con overlay AI (posizione fantasma ottimale), marcatori eventi (uccisioni, piazzamenti bomba) e controlli di riproduzione completi.



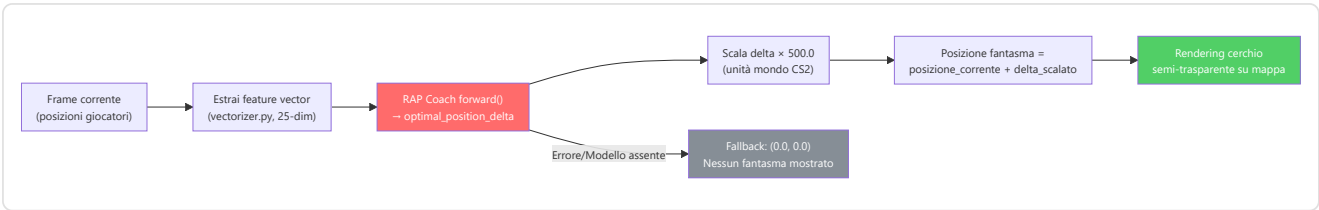
PlaybackEngine — Architettura interna:

Il PlaybackEngine gestisce la riproduzione frame-by-frame con interpolazione temporale. Il timer di aggiornamento utilizza `QTimer` , mantenendo la stessa logica di interpolazione e buffering:

Caratteristica	Dettaglio
Frame rate	60 FPS (interpolazione da 64-tick nativo del demo)
Velocità variabile	0.25x (slow-mo), 0.5x, 1x (normale), 2x, 4x, 8x (fast-forward)
Interpolazione	Lineare tra frame adiacenti per movimenti fluidi
Buffering	Pre-carica 120 frame in anticipo per evitare lag
Seek	Accesso diretto a qualsiasi tick via indice
Round selection	Salta direttamente all'inizio di un round specifico

GhostEngine — Inferenza in tempo reale:

Il GhostEngine è il componente che trasforma le predizioni del RAP Coach in **posizioni fantasma visibili sulla mappa**:



ChronovisorScanner — Rilevamento momenti critici:

Il ChronovisorScanner analizza l'intera partita e identifica i **momenti decisivi** basandosi sul cambio di vantaggio:

Tipo momento	Condizione	Colore marcatore
Errore critico	Morte in vantaggio numerico (es. 4v3 → 3v3)	● Rosso
Giocata eccellente	Uccisione in svantaggio numerico (es. 2v3 → 2v2)	● Verde
Piazzamento bomba	Evento bomb_planted con timer	● Giallo
Clutch	Vittoria 1vN (N ≥ 2)	★ Oro
Eco round win	Vittoria con equipaggiamento < \$2.000	● Blu

Ogni momento viene posizionato sulla Timeline come un marcatore cliccabile. Il click salta il playback direttamente a quel tick.

Flusso di caricamento del viewer (One-Click):

1. L'utente clicca "Tactical Viewer" dalla Home
2. Se nessuna demo è caricata → `trigger_viewer_picker()` apre il file picker automaticamente
3. L'utente seleziona un file `.dem`
4. Appare un dialogo "Ricostruzione Dinamica 2D in Corso..."
5. Un thread in background esegue `_execute_viewer_parse(path)` → `DemoLoader.load_demo()`
6. Al completamento: dismissione dialogo, caricamento frame in PlaybackEngine
7. La mappa viene renderizzata con i giocatori al frame 0

12.11 Dati Spaziali e Gestione Mappe

File: `Programma_CS2_RENAN/core/spatial_data.py`, `spatial_engine.py`, `data/map_config.json`

Il sistema di gestione mappe traduce le **coordinate mondo di CS2** (valori tipici: -2000 a +2000 su X/Y) in **coordinate pixel** sulla texture della mappa (0.0 a 1.0 normalizzato), e viceversa.

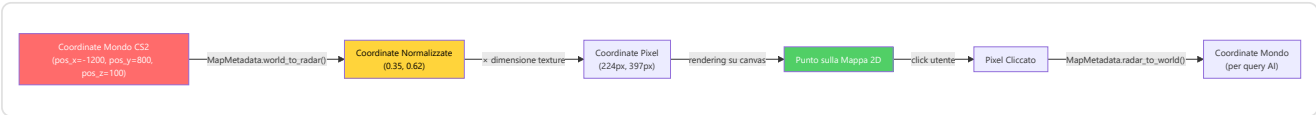
Prima della geometria, l'identità: quali mappe esistono. Un censimento dell'agosto 2026 ha trovato **dodici** elenchi di mappe note dichiarati in punti diversi del progetto, con contenuti divergenti: chi ne conosceva undici, chi nove, chi otto. La conseguenza non era teorica — la stessa demo poteva essere riconosciuta da uno strumento e ignorata da un altro, a seconda di quale elenco quel modulo si portava dietro.

`core/known_maps.py` è oggi l'autorità unica: `KNOWN_MAP_NAMES` (i nomi nudi), `KNOWN_MAP_IDS` (gli stessi con il prefisso `de_`), l'espressione regolare per riconoscere una mappa dentro un nome di file, e i tre helper `bare_name()`, `is_known_map()`, `sniff_map_from_text()`. L'insieme è deliberatamente un **soprainsieme** di tutti e dodici gli elenchi trovati: meglio riconoscere una mappa che non è nella rotazione competitiva che ignorarne una che c'è.

Va detto con precisione fin dove arriva, perché una SSOT dichiarata più ampia di quello che è vale meno di nessuna SSOT. **Sette consumatori** sono stati convertiti e sono presidiati dal test `test_known_maps_ssot.py`, che vieta di ridichiarare il trio mirage/inferno/nuke e verifica che ciascuno importi ancora il modulo:

`apps/qt_app/core/match_utils.py` · `apps/qt_app/screens/coach_screen.py` · `tools/d3_recover_shard_metadata.py` · `tools/mine_coaching_experience.py` · `tools/mine_shard_strategies.py` · `tools/populate_match_results.py` · `tools/rebuild_monolith.py`

Restano fuori quattro elenchi locali — in `coaching_dialogue.py`, `reporting/analytics.py`, `knowledge/pro_demo_miner.py` e in `Goliath_Hospital.py` — e due di questi sono esclusioni **volute**: `REQUIRED_MAPS` di Goliath verifica la presenza di file di asset, non l'identità di una mappa, e il registro spaziale porta geometrie radar calibrate a mano che non hanno senso fuori dal loro contesto. Gli altri due sono debito residuo, non progetto.



MapMetadata (dataclass immutabile per ogni mappa):

Campo	Esempio (Dust2)	Scopo
<code>pos_x</code>	-2476	X dell'angolo in alto a sinistra in unità mondo
<code>pos_y</code>	3239	Y dell'angolo in alto a sinistra in unità mondo
<code>scale</code>	4.4	Unità di gioco per pixel della texture
<code>z_cutoff</code>	<code>None</code>	Separatore di livello (solo mappe multilivello)
<code>level</code>	<code>"single"</code>	Tipo: <code>"single"</code> , <code>"upper"</code> , <code>"lower"</code>
<code>texture_width</code>	1024	Larghezza texture radar in pixel
<code>texture_height</code>	1024	Altezza texture radar in pixel

MapManager (`core/map_manager.py`):

Il MapManager è il componente di alto livello che coordina il caricamento delle mappe per l'interfaccia:

Metodo	Scopo
<code>get_map_metadata(map_name)</code>	Restituisce MapMetadata per la mappa richiesta
<code>load_radar_texture(map_name)</code>	Carica texture PNG della mappa radar da cache o disco
<code>get_available_maps()</code>	Lista delle mappe supportate con metadata
<code>world_to_pixel(pos, map_name)</code>	Shortcut per conversione coordinate mondo → pixel

Mappe multilivello supportate:

Mappa	z_cutoff	Livelli	Note
Nuke	-495	upper / lower	Due plant site su piani diversi
Vertigo	11700	upper / lower	Grattacielo con due aree giocabili
Tutte le altre	None	single	Mappe a livello singolo

SpatialEngine (`core/spatial_engine.py`):

Il SpatialEngine aggiunge capacità di **ragionamento spaziale** al sistema di coordinate base:

Metodo	Input	Output	Uso
<code>distance_2d(pos_a, pos_b)</code>	Due coordinate mondo	Float (unità mondo)	Calcolo distanza di ingaggio
<code>is_visible(pos_a, pos_b, obstacles)</code>	Due pos + ostacoli	Bool	Linea di vista (semplificata)
<code>get_zone(pos, map_name)</code>	Coordinata + mappa	Stringa (es. "A_site")	Classificazione zona tattica
<code>nearest_cover(pos, map_name)</code>	Coordinata + mappa	Coordinata copertura	Suggerimento posizionale

AssetManager (`core/asset_manager.py`):

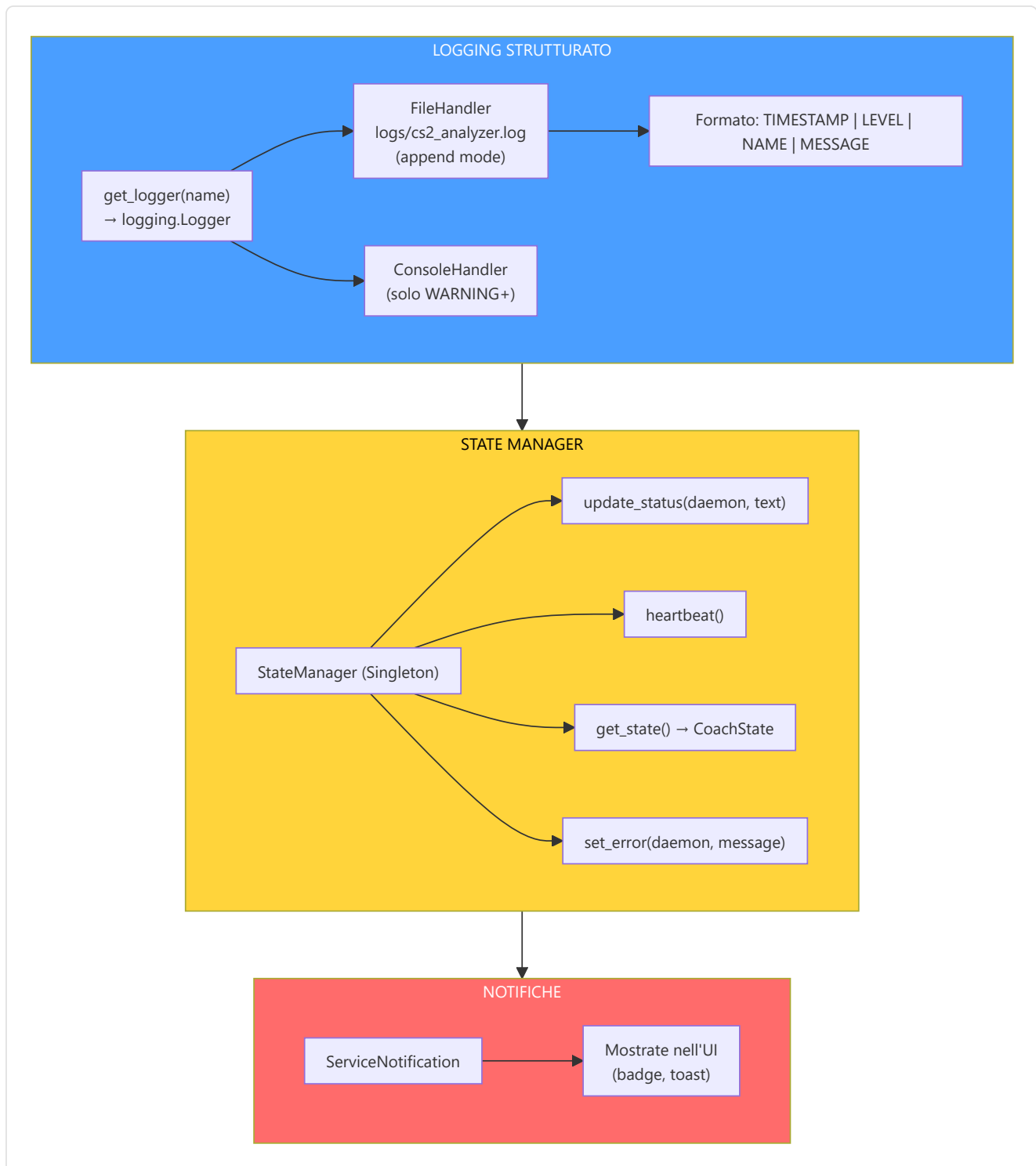
Gestisce il caricamento delle texture delle mappe e degli asset UI:

Asset	Formato	Dimensione tipica	Cache
Texture mappa (radar)	PNG 1024×1024	~500KB	Sì (in-memory)
Icone giocatore	PNG 32×32	~2KB	Sì
Font (JetBrains Mono, YUPIX)	TTF	~200KB	Sì (registrate via Qt)
Wallpaper temi	PNG 1920×1080	~2MB	Lazy load

12.12 Osservabilità e Logging

File: `Programma_CS2_RENAN/observability/logger_setup.py` **File correlati:** `backend/storage/state_manager.py` , `backend/services/telemetry_client.py`

Il sistema di osservabilità garantisce che ogni evento significativo sia **tracciabile, strutturato e persistente**. Il client di telemetria (`telemetry_client.py`) utilizza **httpx** (HTTP asincrono) per l'invio non bloccante di metriche e eventi, evitando che latenze di rete influiscano sulle prestazioni dell'applicazione.

**Daemon monitorati dallo StateManager:**

Daemon	Campo in CoachState	Valori tipici
Scanner	<code>hltv_status</code>	"Scanning", "Paused", "Error"
Digester (worker)	<code>ingest_status</code>	"Processing", "Idle", "Error"
Teacher (trainer)	<code>ml_status</code>	"Learning", "Idle", "Error"
Globale	<code>status</code>	"Paused", "Training", "Idle", "Error"

Sentry Integration (`observability/sentry_setup.py`):

Il sistema include integrazione opzionale con **Sentry** per error tracking remoto, con un approccio **double opt-in** per la privacy:

Caratteristica	Implementazione
Double opt-in	L'utente deve (1) impostare <code>SENTRY_DSN</code> come variabile d'ambiente E (2) attivare il flag nelle impostazioni
PII Scrubbing	Tutti i dati personali (nomi giocatori, Steam ID, percorsi) vengono rimossi prima dell'invio
Breadcrumb sanitization	I breadcrumb di navigazione vengono puliti da informazioni sensibili
Non bloccante	Se Sentry non è configurato, il sistema prosegue normalmente (Fase 4 dell'avvio)
Contesto arricchito	Ogni errore include: versione app, stato daemon, conteggio demo, sistema operativo

Logger Setup (`observability/logger_setup.py`):

Il sistema di logging centralizzato fornisce log strutturati con:

Caratteristica	Dettaglio
Formato	<code>`TIMESTAMP</code>
File handler	<code>logs/cs2_analyzer.log</code> (append mode, rotazione automatica)
Console handler	Solo <code>WARNING+</code> per non inquinare l'output
Naming convention	<code>get_logger("cs2analyzer.<module>")</code> — namespace gerarchico
Livelli usati	<code>DEBUG</code> (sviluppo), <code>INFO</code> (operazioni normali), <code>WARNING</code> (anomalie non critiche), <code>ERROR</code> (fallimenti), <code>CRITICAL</code> (terminazione)

Metriche di training esposte in CoachState: `current_epoch`, `total_epochs`, `train_loss`, `val_loss`, `eta_seconds`, `belief_confidence`, `system_load_cpu`, `system_load_mem`.

Registro Centralizzato dei Codici Errore (`observability/error_codes.py`):

Il sistema implementa un registro formale di **24 codici errore** classificati per severità e modulo. Ogni codice è definito come `ErrorCodeDef(NamedTuple)` con: `code`, `severity`, `module`, `description`, `remediation`.

Prefisso	Modulo	Esempio	Severità tipica
<code>LS</code>	Logger Setup	LS-01: RotatingFileHandler non disponibile	MEDIUM
<code>RP</code>	RASP Guard	RP-01: CS2_MANIFEST_KEY non impostato	HIGH
<code>DA</code>	Data Access	DA-01-03: JSON malformato da DB	LOW
<code>P</code>	Pipeline	P7-01/P7-02: Errori pipeline critica	HIGH
<code>F</code>	Feature/Fix	F6-SE: Errore session engine	HIGH
<code>SE</code>	Session Engine	SE-05: Errore critico sessione	HIGH
<code>IM</code>	Ingestion	IM-03: Errore ingestione	MEDIUM
<code>NN</code>	Neural Network	NN-02: Errore rete neurale	MEDIUM
<code>CO</code>	Console Control	CO-01/CO-03: Errori controllo critici	HIGH
<code>R1</code>	Release/Manifest	R1-12: Errore manifest rilascio	HIGH

Severità (`Severity(Enum)`): `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.

Utilità:

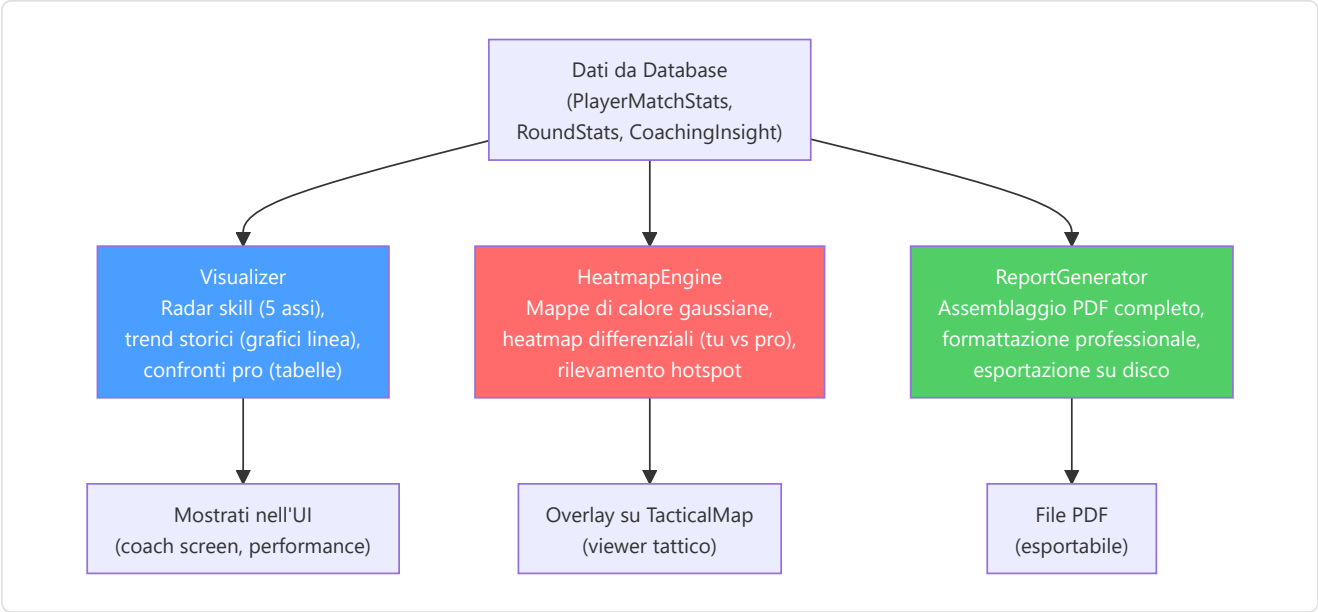
- `log_with_code(error_code, message) → str` — Prefixa il messaggio con il codice formale (es. `[LS-01] message`)
- `get_all_codes() → list[dict]` — Tutti i codici come lista di dizionari per accesso programmatico

Gerarchia Eccezioni (`observability/exceptions.py`): Classe base `CS2AnalyzerError(Exception)` con sottoclassi: `ConfigurationError`, `DatabaseError`, `IngestionError`, `TrainingError`, `IntegrationError`, `UIError`. Ogni eccezione può portare un `error_code: ErrorCode | None` per correlazione con il registro.

12.13 Reporting e Visualizzazione

File: `Programma_CS2_RENAN/reporting/visualizer.py`, `report_generator.py` **File correlati:** `backend/processing/heatmap_engine.py`

Il sistema di reporting trasforma i dati grezzi in **visualizzazioni comprensibili** per l'utente.



HeatmapEngine — Dettagli tecnici:

Caratteristica	Dettaglio
Thread-safety	<code>generate_heatmap_data()</code> gira in thread separato (non blocca UI)
Texture creation	Solo nel thread principale (requisito del thread Qt)
Tipo	Occupazione gaussiana 2D (blur kernel parametrico)
Differenziale	Sottrae heatmap pro da heatmap utente → rosso (troppo tempo), blu (troppo poco)
Hotspot	Identifica cluster di posizione per training posizionale

MatchVisualizer (`reporting/visualizer.py`) — metodi di rendering specializzati:

Il MatchVisualizer estende la capacità di reporting con 6 metodi di rendering ad alta qualità (cfr. sezione 12.26 per i dettagli algoritmici):

Metodo	Input	Output	Descrizione
<code>render_skill_radar(user, pro)</code>	Dict metriche	PNG radar chart	Pentagono 5 assi con overlay pro baseline
<code>render_trend_chart(history, metric)</code>	Lista storica	PNG line chart	Andamento temporale con media mobile
<code>render_heatmap(positions, map_name)</code>	Coordinate tick	PNG heatmap	Gaussiana 2D su texture mappa
<code>render_differential(user_pos, pro_pos)</code>	Due set posizioni	PNG heatmap diff	Rosso (eccesso) / Blu (deficit) / Verde (allineamento)
<code>render_critical_moments(events)</code>	Lista eventi round	PNG timeline	Marcatori colorati per kill/death/bomb
<code>render_comparison_table(user, pro)</code>	Due profili stats	HTML/PNG tabella	Delta percentuale per ogni metrica

ReportGenerator (`reporting/report_generator.py`):

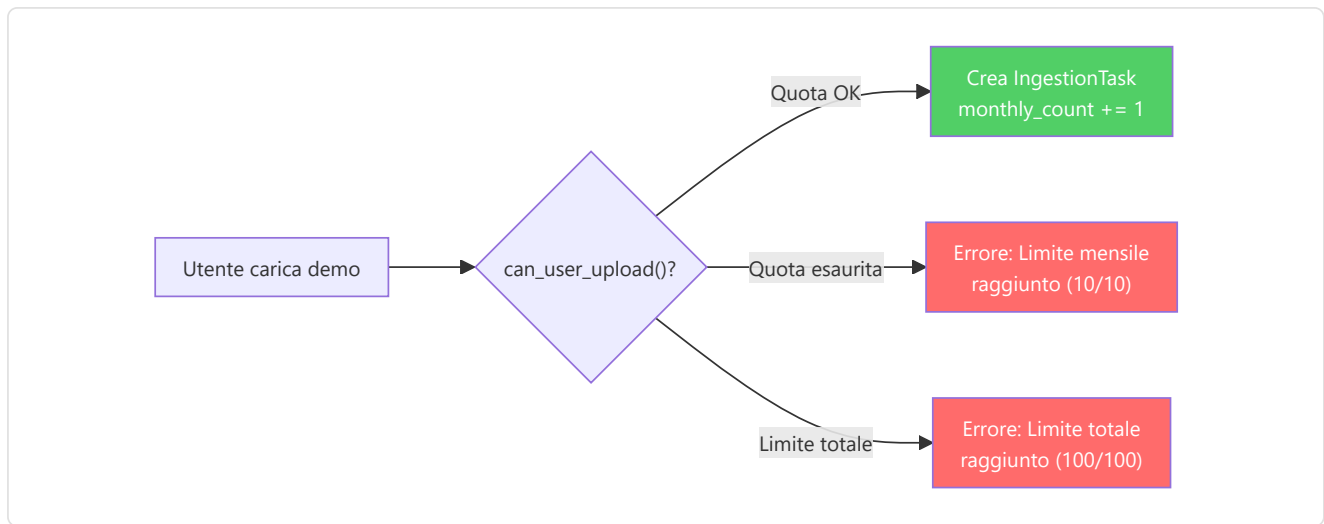
Assembla tutti gli elementi visuali in un **documento PDF completo**:

1. **Header**: Nome giocatore, data, demo analizzata
2. **Executive Summary**: Metriche chiave (KPR, ADR, KAST, Rating)
3. **Radar Chart**: Confronto 5 assi con pro baseline
4. **Trend Charts**: Tendenze delle ultime N partite
5. **Heatmap Differenziale**: Dove posizionarsi meglio
6. **Momenti Critici**: I 5 momenti decisivi della partita
7. **Consigli del Coach**: Top-3 insight prioritizzati
8. **Footer**: Generato da Macena CS2 Analyzer v.X.X

12.14 Gestione Quote e Limiti

Il sistema implementa un meccanismo di **quota mensile** per prevenire l'abuso delle risorse di elaborazione e garantire una distribuzione equa del carico.

Limite	Valore	Enforcement
MAX_DEMOS_PER_MONTH	10	<code>StorageManager.can_user_upload()</code>
MAX_TOTAL_DEMOS	100	<code>StorageManager.can_user_upload()</code>
MIN_DEMOS_FOR_COACHING	10	Soglia per coaching personalizzato completo
Reset mensile	Automatico	<code>PlayerProfile.last_upload_month</code> vs. mese corrente



12.15 Tolleranza ai Guasti e Recupero

Il sistema è progettato per **non perdere mai dati** e **riprendersi automaticamente** da quasi tutti i tipi di fallimento.

MECCANISMI DI TOLLERANZA AI GUASTI

Zombie Task Cleanup
 All'avvio: task 'processing'
 → reset a 'queued'
 Ripristino automatico senza perdita

Backup Automatico
 All'avvio: checkpoint 'startup_auto'
 Rotazione: 7 giornalieri + 4 settimanali
 Copia completa database

Service Supervisor
 Monitora servizi esterni
 Auto-restart, max 3 tentativi/ora
 delay 5s tra i tentativi

Resilienza HLTV
 Backoff adattivo sui fallimenti
 + modalità dormiente 6h
 se HLTV irraggiungibile

Connection Pooling
 pool_size=1 + max_overflow=4
 Timeout 30s per contesa WAL
 PRAGMA per-connessione

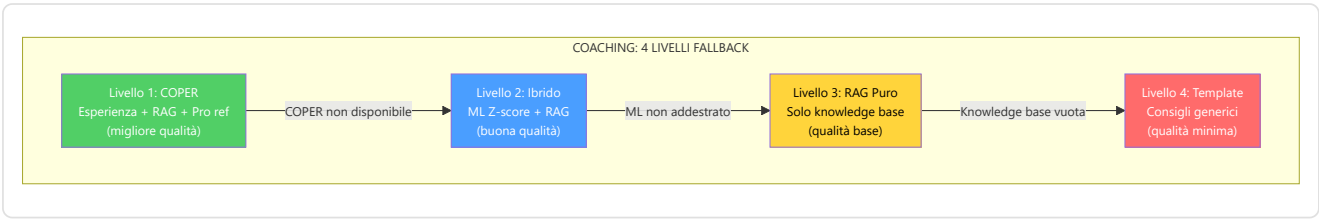
Degradazione Graduale
 Coaching: 4 livelli fallback
 GhostEngine: (0,0) se errore
 Ogni servizio ha un piano B

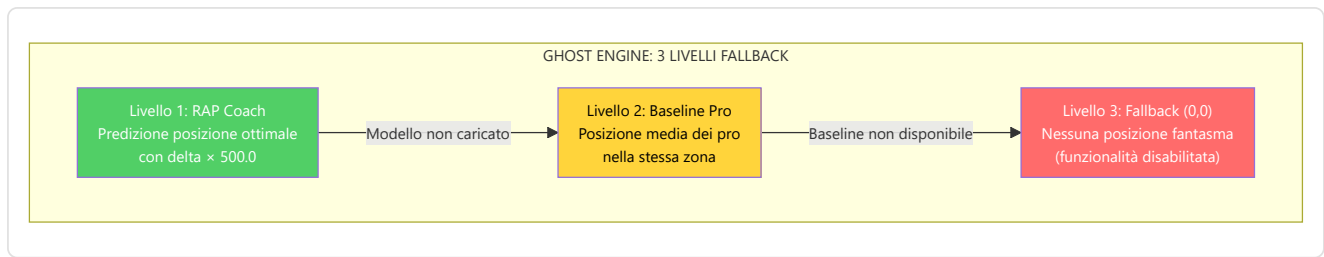
Matrice di fallimento e recupero:

Scenario di fallimento	Meccanismo di recupero	Perdita di dati
Crash dell'applicazione durante parsing	Zombie Task Cleanup al riavvio	Nessuna — task ricominciato
Corruzione database	Restore da backup automatico più recente	Massimo 24 ore di dati

Scenario di fallimento	Meccanismo di recupero	Perdita di dati
Servizio HLTV non raggiungibile	Backoff adattivo del fetcher (delay cresce con i fallimenti consecutivi) + modalità dormiente di 6 ore del sync service quando HLTV è irraggiungibile. Previene cascade failure su API esterne.	Nessuna — dati pro ritardati
Modello ML non caricabile	Fallback a pesi casuali (GhostEngine) o coaching base	Nessuna — qualità degradata
RAM insufficiente durante training	Early stopping automatico, checkpoint salvato	Nessuna — ultimo checkpoint valido
Disco pieno	Database Governor rileva e notifica via ServiceNotification	Prevenzione — nessun dato scritto
Demo corrotta/troncata	IntegrityChecker rifiuta prima del parsing	Nessuna — demo ignorata con warning
Ollama non disponibile	LLM fallback a RAG puro, poi coaching base	Nessuna — qualità narrativa degradata
API Steam/FACEIT timeout	Retry con backoff esponenziale (max 3 tentativi)	Nessuna — profilo non aggiornato
Container FlareSolvert non attivo	<code>DockerManager.ensure_flaresolvert()</code> : docker start → docker compose up, health poll 45s	Nessuna — scraping HLTV ritardato
Conflitto WAL (lock contention)	Timeout 30s con retry automatico	Nessuna — operazione ritardata
Checkpoint ML corrotto	Fallback a checkpoint precedente (versionamento)	Parziale — perde ultimo training
Vettore query norma-zero (M-07)	<code>VectorIndex.search()</code> ritorna <code>None</code> con warning — pipeline RAG continua senza crash	Nessuna — risultato RAG vuoto
File settings.json corrotto (M-08)	<code>load_user_settings()</code> valida struttura, fallback a default se non-dict	Nessuna — impostazioni predefinite
Logger non inizializzato (C-09)	Bootstrap anticipato usa <code>print()</code> prima dell'init del logger	Nessuna — messaggi su stdout
DB handle leak su integrity check (H-02)	<code>_verify_integrity()</code> usa <code>try/finally</code> + <code>PRAGMA quick_check</code>	Nessuna — connessione sempre chiusa
Errori batch mascherati in RAP training (H-03)	Rimosso <code>except (KeyError, TypeError): continue</code> — <code>train_step()</code> richiede dict con key <code>'loss'</code>	Nessuna — errori ora visibili

Degradazione graduale — La Catena di Fallback:

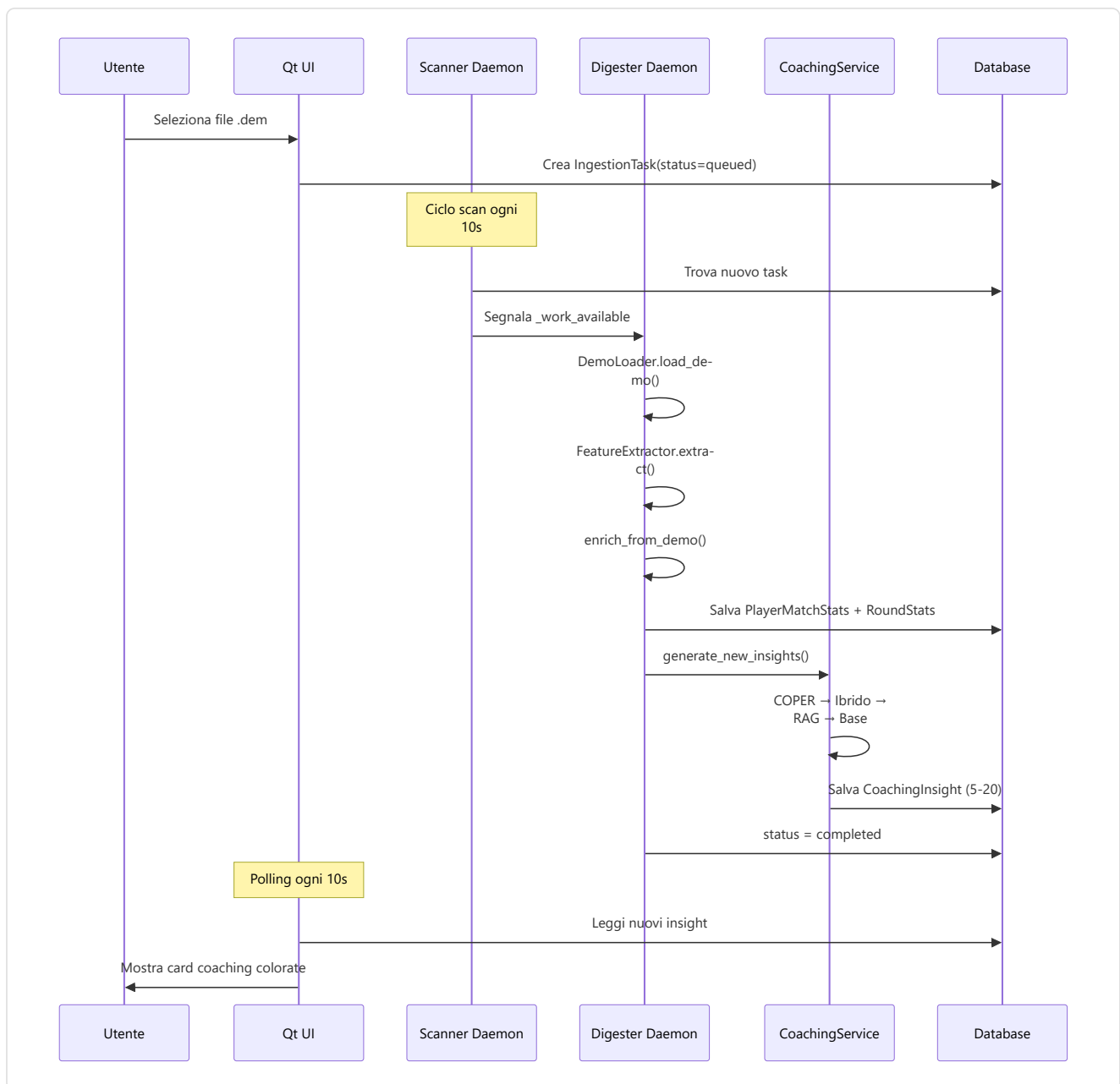




12.16 Viaggio Completo dell'Utente — 4 Flussi Principali

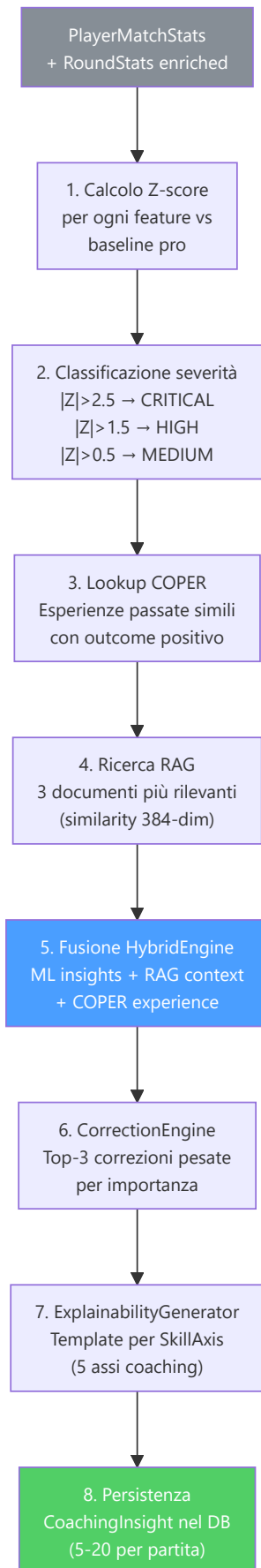
Questa sezione descrive i **4 flussi principali** che un utente attraversa durante l'uso di Macena CS2 Analyzer.

Flusso 1: Upload e Analisi di una Demo



Dettaglio: Pipeline CoachingService interna

Quando `generate_new_insights()` viene invocato, il CoachingService esegue internamente:

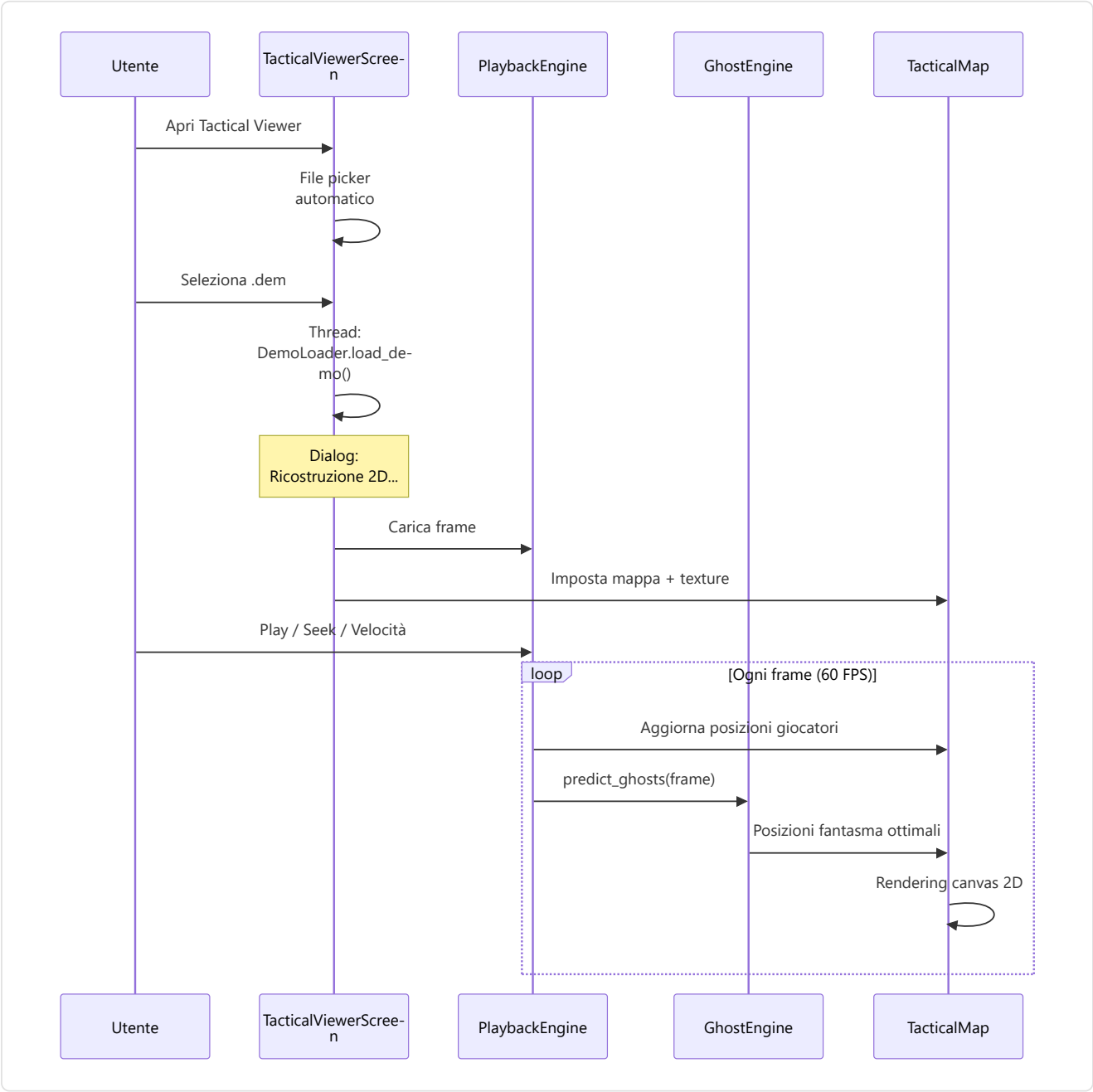


LessonGenerator — generazione lezioni strutturate da demo:

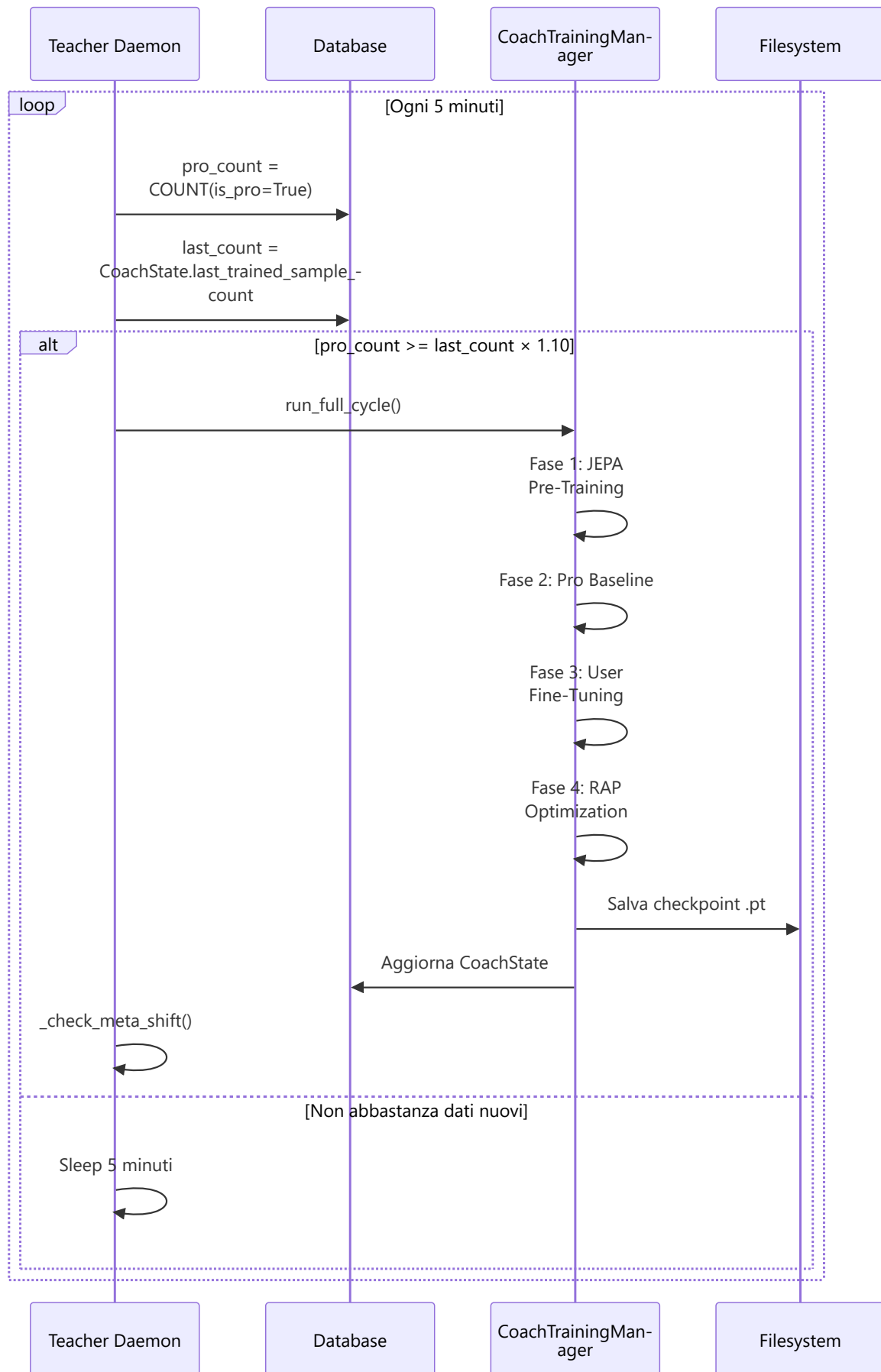
Parametro	Valore	Significato
ADR_STRONG	75	ADR ≥ 75 → "Buon danno medio"
HS_STRONG	0.40	HS% ≥ 40% → "Buona precisione al testa"
KAST_STRONG	0.70	KAST ≥ 70% → "Buona contribuzione al round"
Formato output	Sezione 1 (Overview) + Sezione 2 (Punti di forza) + Sezione 3 (Aree di miglioramento) + Sezione 4 (Pro tip)	Struttura lezione standard

Il LessonGenerator opera in modalità doppia: se Ollama è disponibile, genera lezioni in linguaggio naturale via `LLMService.generate_lesson()`. Se non è disponibile, usa template strutturati con i dati numerici inseriti in frasi preformate.

Flusso 2: Visualizzazione Tattica



Flusso 3: Riaddestramento ML Automatico



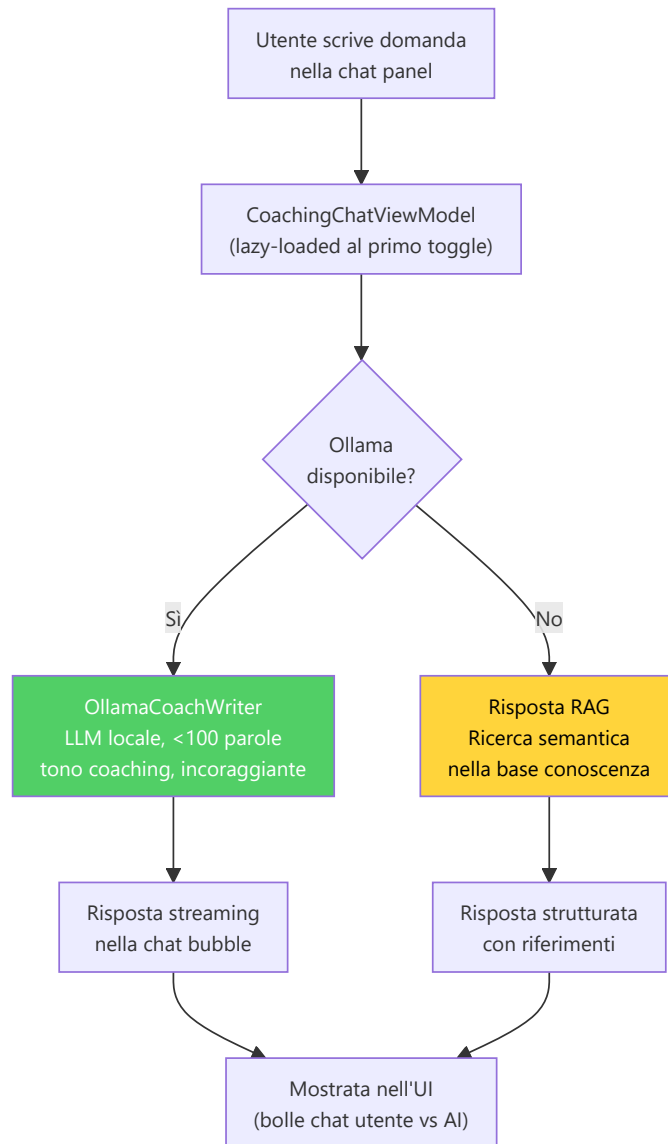
Flusso 4: Chat AI con il Coach

CoachingDialogueEngine — Pipeline Multi-Turno:

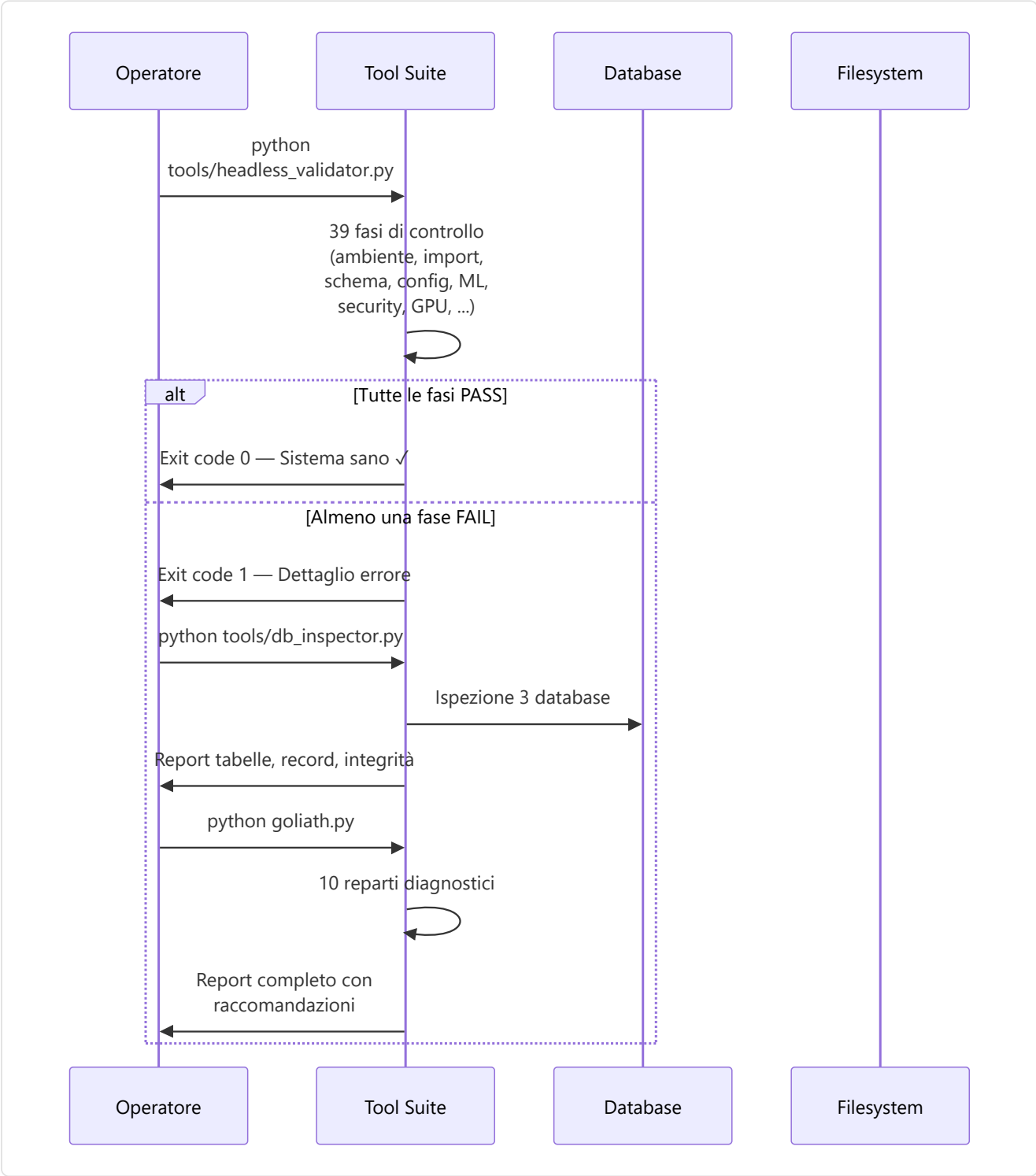
Il dialogo AI segue una pipeline a 5 fasi per ogni messaggio dell'utente:

Fase	Componente	Descrizione
1	Intent Classification	Classifica la domanda: coaching, stats query, comparison, general
2	Context Retrieval	Recupera le ultime N partite, insight recenti, profilo giocatore
3	RAG Augmentation	Cerca nella knowledge base i 3 documenti più rilevanti (similarity search 384-dim)
4	COPER Augmentation	Se disponibile, aggiunge esperienze di coaching passate con outcome positivo
5	Response Generation	Genera risposta via Ollama (se disponibile) o template RAG strutturato

Parametro	Valore	Scopo
MAX_CONTEXT_TURNS	6	Numero massimo di turni mantenuti nel contesto
MAX_RESPONSE_WORDS	100	Limite parole per risposta LLM
SIMILARITY_THRESHOLD	0.5	Soglia minima per risultati RAG
Tone	"coaching, encouraging"	Prompt system per tono positivo



Flusso 5: Diagnostica e Manutenzione



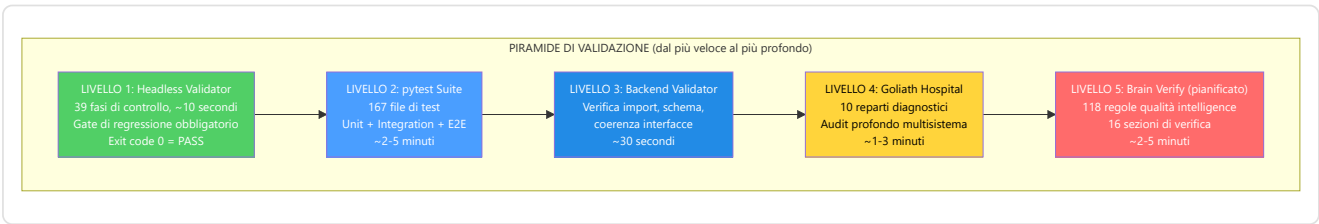
Matrice strumenti per scenario:

Scenario	Strumento consigliato	Tempo
Quick check post-deploy	<code>headless_validator.py</code>	~10s
Database sospetto	<code>db_inspector.py</code>	~30s
Demo non parsata	<code>demo_inspector.py</code>	~5s
Training diverge	<code>Ultimate_ML_Coach_Debugger.py</code>	~2min
Check-up completo	<code>Goliath_Hospital.py</code>	~3min
Audit ML profondo	<code>brain_verify.py</code> (pianificato, non implementato)	~5min

12.17 Suite di Strumenti — Validazione e Diagnostica (`tools/`)

Directory: `tools/` (root) + `Programma_CS2_RENAN/tools/` **File principali:** `headless_validator.py` , `db_inspector.py` , `demo_inspector.py` , `brain_verify.py` (pianificato, non ancora implementato), `Goliath_Hospital.py` , `Ultimate_ML_Coach_Debugger.py` , `_infra.py`

La suite di strumenti è una **raccolta di 67 script** distribuiti su due directory (`tools/` root con 49 script e `Programma_CS2_RENAN/tools/` con 18 script) che formano una piramide di validazione multi-livello. Ogni strumento ha uno scopo preciso e può essere eseguito indipendentemente, ma insieme formano un sistema di garanzia della qualità che copre ogni aspetto del progetto.



Headless Validator (`tools/headless_validator.py` , ~2.900 righe) — il gate di regressione obbligatorio. Eseguito dopo **ogni** task di sviluppo con **39 fasi tematiche di controllo** (funzioni `check()` / `warn()` con severità). Fasi principali:

Gruppo di fasi	Verifica
Ambiente / Deps / GPU / Platform	Python e dipendenze critiche presenti (torch, pyside6, sqlmodel, demoparser2), device CUDA
Core / NewImport / Structure	I moduli fondamentali (<code>config.py</code> , <code>spatial_data.py</code> , <code>lifecycle.py</code>) si caricano senza errori
NN / ML / ML-Deep / RAP / Training	Istanziamento modelli con pesi casuali — dimensioni e forward pass
DB / DB-Deep / Schema / Storage	Le tabelle SQLAlchemy (25 + 3 per-match) si creano correttamente
Config / Config-Deep / Features / Processing	<code>METADATA_DIM</code> , percorsi, costanti coerenti
Coaching / Knowledge / Services / Analysis / Belief / Baseline	Contratti dei sottosistemi di coaching e analisi
Ingestion / DataSrc / Adapter / Integrity / Security	Pipeline di ingestione, validazione demo, sicurezza
UI / Qt-Import / Design-Tokens / Reporting / Quality / Quality-Adv	Interfaccia, token di design, qualità

Infrastruttura Condivisa (`tools/_infra.py`):

Componente	Ruolo
<code>BaseValidator</code>	Classe base per tutti i validatori — output console unificato, conteggio pass/fail/warning
<code>path_stabilize()</code>	Normalizzazione <code>sys.path</code> per evitare import relativi inconsistenti
<code>Console</code> (Rich)	Output colorato con tabelle, progress bar, emoji di stato
<code>Severity</code> enum	4 livelli: <code>INFO</code> , <code>WARNING</code> , <code>ERROR</code> , <code>CRITICAL</code>

DB Inspector (`tools/db_inspector.py`) — ispezione profonda del database:

- Apre `database.db` e `hltv_metadata.db` separatamente
- Per ogni tabella: conta record, verifica schema, controlla integrità indici
- Mostra metriche di spazio (dimensione file, pagine WAL, frammentazione)
- Rileva anomalie: tabelle vuote inattese, record orfani, timestamp fuori range
- Output: tabella Rich colorata con stato per ogni tabella

Demo Inspector (`tools/demo_inspector.py`) — ispezione file demo:

- Verifica magic bytes (`PBDEMS2\0` per CS2, `HL2DEMO\0` per legacy)
- Controlla dimensione file (1KB min, 5GB max)
- Estrae metadata header senza parsing completo
- Rileva corruzione parziale (troncamento, header danneggiato)
- Output: report sulla validità della demo con dettagli errore se invalida

Brain Verify (`tools/brain_verify.py` + `brain_verification/`, 16 sezioni) — *(pianificato, non ancora implementato)*:

Il sistema di verifica dell'intelligenza è progettato con **16 sezioni tematiche** che copriranno **118 regole** di qualità:



BRAIN VERIFY — 16 SEZIONI (118 REGOLE)

§1 Dimensional Contracts
METADATA_DIM, latent_dim,
input shapes

§2 Forward Pass
Smoke test tutti i modelli
con input sintetico

§3 Loss Functions
Gradient flow, NaN check,
loss monotonicity

§4 Training Pipeline
DataLoader, optimizer,
scheduler config

§5 Feature Engineering
Vectorizer alignment,
normalization bounds

§6 Coaching Pipeline
COPER → Hybrid → RAG
fallback chain

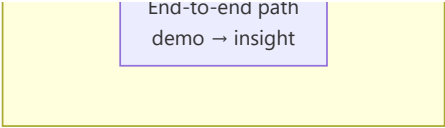
§7 Data Integrity
DB schema, FK constraints,
orphan detection

§8 Concept Alignment
16 coaching concepts,
label correctness

...

§16 Integration

Final integration test



End-to-end path
demo → insight

Goliath Hospital (tools/Goliath_Hospital.py) — diagnostica multi-dipartimento:

Il "Goliath Hospital" (Programma_CS2_RENAN/tools/Goliath_Hospital.py , invocabile anche via `python goliath.py doctor`) è il **sistema di diagnostica più completo** del progetto, organizzato come un ospedale con 11 reparti specializzati:

Reparto	Nome	Controlli
1	Pronto Soccorso (ER)	Sintassi, pattern proibiti, namespace
2	Radiologia	Asset, struttura file/directory
3	Patologia	Qualità dati, rilevamento dati mock
4	Cardiologia	Moduli core, DB, config, motori di analisi
5	Neurologia	ML/AI, forward pass
6	Oncologia	Debito tecnico
7	Pediatria	File modificati di recente
8	Terapia Intensiva (ICU)	Integrazione, import
9	Farmacia	Dipendenze, versioni, compatibilità
10	Clinica degli Strumenti	Validazione degli altri strumenti (meta-test)
11	Endocrinologia	Entry point, migrazioni, validazione JSON

Ultimate ML Coach Debugger (tools/Ultimate_ML_Coach_Debugger.py) — falsificazione delle credenze neurali:

Questo strumento esegue un audit a **3 fasi** sulla pipeline ML:

1. **Verifica Strutturale** — Dimensioni tensor, architettura modelli, parametri learnable
2. **Verifica Comportamentale** — Forward pass con dati reali, gradient flow, loss convergence
3. **Falsificazione Credenze** — Testa se il modello "crede" cose sbagliate: predizioni overconfident, bias sistematici, pattern degeneri

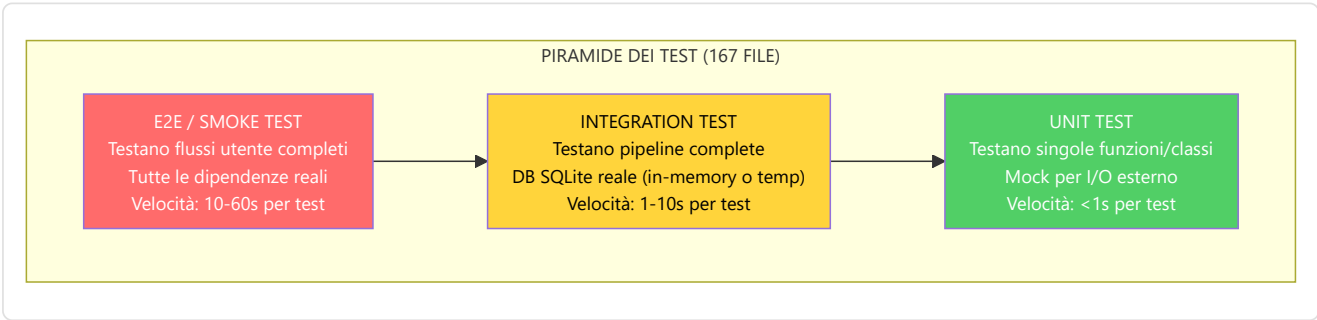
Altri strumenti:

Strumento	File	Scopo
backend_validator.py	tools/	Verifica coerenza import e interfacce backend
build_tools.py	tools/	Automazione build PyInstaller
user_tools.py	tools/	Utilità per l'utente finale (reset, export)
dev_health.py	tools/	Quick check salute sviluppo
context_gatherer.py	tools/	Raccoglie contesto per debug report
dead_code_detector.py	tools/	Identifica codice morto non referenziato
sync_integrity_manifest.py	tools/	Aggiorna integrity_manifest.json con hash SHA-256
project_snapshot.py	tools/	Snapshot completo dello stato del progetto
ui_diagnostic.py	tools/	Diagnostica specifica UI (Qt)
build_pipeline.py	Root tools/	Pipeline di build completa
Feature_Audit.py	Root	Audit del feature vector 25-dim
Sanitize_Project.py	Root	Pulizia file temporanei, cache, artifacts
audit_binaries.py	Root	Verifica integrità eseguibili e .pt

12.18 Architettura della Test Suite (tests/)

Directory: Programma_CS2_RENAN/tests/ (157 file test_*.py : 152 nella suite piatta + 5 in automated_suite/ , più conftest.py) e tests/ root (8 file test_*.py , 6 script verify_*.py e la cartella forensics/ con 10 script diagnostici, 2 dei quali test_*.py) — **167 file di test in totale**

La test suite è organizzata secondo il **principio della piramide dei test**: molti unit test (veloci, isolati), meno integration test (più lenti, con dipendenze reali), e pochi end-to-end test (completi ma costosi).



Conftest (tests/conftest.py) — fixture condivise:

Fixture	Scope	Descrizione
test_db	function	Database SQLite in-memory con tutte le tabelle create, auto-cleanup
sample_match_stats	function	PlayerMatchStats con dati realistici derivati da partite reali
mock_demo_data	function	DemoData mock con frame e eventi per test di parsing
tmp_models_dir	function	Directory temporanea per checkpoint .pt
coaching_service	function	CoachingService configurato con DB di test

Automated Suite (tests/automated_suite/):

Categoria	File	Copertura
Smoke	test_smoke_*.py	Import critici, istanziazione modelli, DB connection
Unit	test_unit_*.py	Funzioni pure, calcoli, trasformazioni
Functional	test_functional_*.py	Pipeline complete, flussi di lavoro
E2E	test_e2e_*.py	Demo upload → parsing → coaching → insight
Regression	test_regression_*.py	Bug fix specifici (G-01 label leakage, G-07 Bayesian, etc.)

Test per sottosistema:

Sottosistema	File di test	Cosa testano
NN Core	test_jepa_model.py, test_rap_model.py, test_advanced_nn.py	Forward pass, dimensioni tensor, gradient flow
Coaching	test_coaching_service.py, test_hybrid_engine.py, test_correction_engine.py	Pipeline coaching, prioritizzazione insight, fallback chain
Processing	test_vectorizer.py, test_heatmap.py, test_feature_eng.py	Feature extraction, normalizzazione, METADATA_DIM=25
Storage	test_database.py, test_models.py, test_backup.py	CRUD, schema, backup/restore, WAL mode
Data Sources	test_demo_parser.py, test_hltv_scraper.py, test_steam_api.py	Parsing, scraping, API timeout/retry
Analysis	test_game_theory.py, test_belief.py, test_role_engine.py	Motori analisi, calibrazione, euristica
Knowledge	test_rag.py, test_experience_bank.py, test_knowledge_graph.py	Retrieval, COPER efficacia, KG query
Ingestion	test_ingest_pipeline.py, test_registry.py	Pipeline completa, deduplicazione

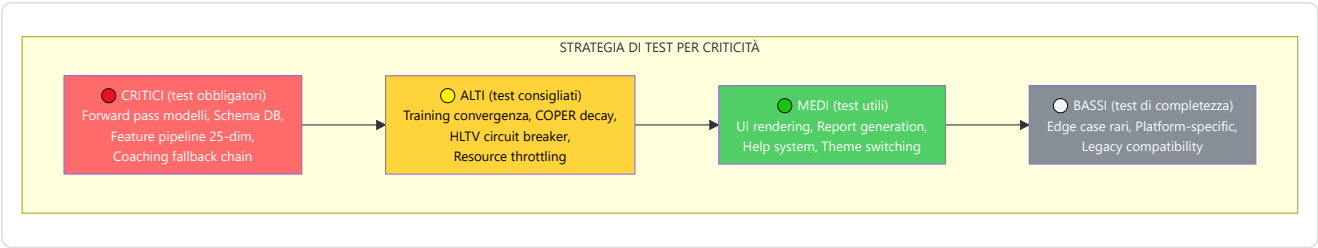
Forensics (tests/forensics/ nella root, 10 script):

Gli script forensics sono strumenti diagnostici per indagini post-mortem: check_db_status.py , check_failed_tasks.py , debug_env.py , debug_nade_cols.py , debug_parser_fields.py , probe_missing_tables.py , test_forensic_parser.py , test_skill_logic.py , verify_map_dimensions.py , verify_spatial_integrity.py .

Verification Scripts (6 file verify_*.py nella root tests/):

Script di verifica one-shot per validare specifici aspetti del sistema (coerenza spaziale, pipeline dati, integrità), affiancati dai test root test_d3_rederive.py , test_eval_harness.py , test_lock_files.py , test_rescrape_placeholder_pros.py , test_sync_pro_players.py .

Strategia di test per criticità:



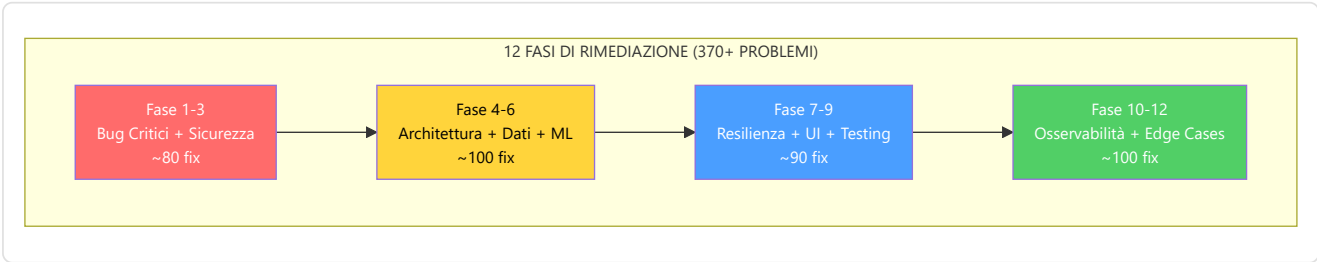
Gap di copertura identificati:

I seguenti moduli hanno alta complessità (>500 LOC) ma copertura test limitata:

Modulo	LOC	Copertura test	Rischio
training_orchestrator.py	733	Bassa	ALTO — orchestrazione multi-fase
session_engine.py	538	Media	MEDIO — testabile solo con subprocess
coaching_service.py	585	Media	MEDIO — 4 livelli fallback
experience_bank.py	751	Bassa	ALTO — logica COPER complessa
tensor_factory.py	686	Bassa	ALTO — DataLoader/Dataset
coach_manager.py	878	Bassa	ALTO — stato globale training

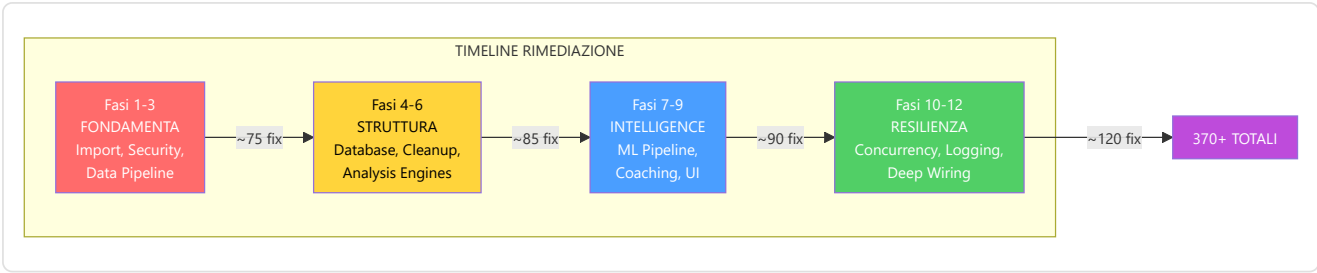
12.19 Le Fasi di Rimediazione Sistemática

Il progetto ha attraversato un processo di **rimediazione sistemática**: 12 fasi iniziali (**370+ problemi** risolti, dettagliate sotto), completate da una 13ª fase e da due ondate successive che hanno portato il totale a **610+ problemi risolti** (cfr. Riepilogo esecutivo in Parte 1A). Ogni fase si è concentrata su una categoria specifica di problemi, dalla correzione di bug critici alla ristrutturazione architetturale. A queste si aggiunge la campagna di audit dell'agosto 2026, che ha cambiato metodo — leggere tutto prima di decidere cosa sia un problema — ed è descritta in §12.19.1.



Dettaglio per fase:

Fase	Focus	Problemi risolti	Esempio chiave
1	Import e struttura base	~20	Circular imports, path resolution, missing <code>__init__.py</code>
2	Sicurezza e secrets	~25	API keys hard-coded → env vars/keyring, input validation
3	Data pipeline e feature	~30	G-01 label leakage, G-02 normalizzazione bounds, feature alignment
4	Database e schema	~30	WAL mode enforcement, missing indici, schema migration safety
5	Dead code e cleanup	~25	G-06 eliminazione <code>nn/advanced/</code> , import inutilizzati, file duplicati
6	Analysis engines	~30	Graceful degradation per tutti gli 11 motori, edge case handling
7	ML pipeline	~35	G-07 Bayesian calibration, gradient clipping, checkpoint versioning
8	Coaching e COPER	~30	G-08 experience decay, RAG index validation, fallback chain
9	UI e UX	~25	F8-XX feedback visivo, state consistency, error prevention
10	Resilienza e concorrenza	~40	F5-35 threading.Event, timeout enforcement, circuit breaker
11	Osservabilità e logging	~35	F5-33 structured logging, correlation IDs, Sentry integration
12	Deep debug e wiring	~45	18 issues: wiring verification, integration testing, edge case audit



Directory **reports/** :

Ogni fase di rimediazione ha prodotto un report dettagliato salvato nella directory **reports/** . Ogni report include:

- Lista numerata dei problemi trovati con codice (G-XX o F-XX)
- Descrizione del problema con file e righe interessate
- Soluzione implementata con diff concettuale
- Verifica: conferma che il fix non ha introdotto regressioni

Correzioni chiave per codice (G-XX):

Codice	Fase	Descrizione	Impatto
G-01	3	Label leakage in <code>ConceptLabeler.label_tick()</code> — feature future leaked nei label di training	CRITICO — invalidava l'addestramento VL-JEPA
G-02	4	Danger zone in <code>vectorizer.py</code> — normalizzazione senza bounds check	ALTO — NaN propagation in training
G-06	5	Dead code in <code>backend/nn/advanced/</code> — 3 file non referenziati	MEDIO — confusione e import accidentali
G-07	7	Bayesian death estimator non calibrato — modello statico senza auto-calibration	ALTO — predizioni inaccurate
G-08	8	COPER experience bank senza decay — esperienze obsolete mai rimosse	MEDIO — coaching basato su dati stantii

Correzioni per feature (F-XX):

Codice	Area	Descrizione
F3-29	Processing	ResNet resize inconsistente — dimensioni immagine non normalizzate
F5-19	Services	Matplotlib rendering senza error handling — crash su stats vuote
F5-21	Storage	<code>session.commit()</code> esplicito dentro context manager — doppio commit
F5-22	Security	API keys hard-coded in source — migrate a env vars + keyring
F5-33	Observability	<code>print()</code> debugging — migrato a structured logging
F5-35	Control	<code>time.sleep()</code> loop per stop — migrato a <code>threading.Event</code>
F6-XX	Analysis	Motori analisi senza graceful degradation — crash su input incompleti
F7-XX	Knowledge	RAG senza index validation — embedding dimensioni incoerenti
F8-XX	UI	Widget Qt senza feedback visivo — azioni silenti confondono l'utente

12.19.1 La campagna di audit integrale (agosto 2026)

Le ondate precedenti partivano da un sintomo: qualcosa si rompeva, si cercava la causa, si correggeva. La campagna dell'agosto 2026 ha invertito il metodo — ha letto **tutti** i file del repository, in due passaggi, prima di decidere cosa fosse un problema.

Il primo passaggio è stato per file: 618 file letti in 76 lotti, ciascuno con il proprio dossier in `docs/audit/dossiers/`. Il secondo è stato trasversale, per lente: dieci contratti che tagliano il codice di traverso invece che per cartella — tick e tensori, sicurezza dei thread Qt, ciclo di vita delle sessioni DB, gestione degli errori, risorse, configurazione e percorsi, internazionalizzazione, correttezza numerica e ML, sicurezza, codice morto (`docs/audit/CONTRACTS.md`).

Ne sono usciti **44 finding**: nessun P0, 12 P1 (correttezza, threading o risorse sotto uso reale), 32 P2 (deriva di contratto e codice morto). Trentuno sono stati corretti, **ciascuno con il proprio test di regressione nello stesso commit**; tredici sono stati differiti, ognuno con la condizione bloccante scritta per esteso — dati di riferimento assenti, verità visiva mancante, o una domanda di ricerca che non si risolve con una patch.

La parte che sopravvive alla campagna non sono però le 31 correzioni: sono i **test di dottrina**, che non verificano un comportamento ma vietano il ritorno di un'intera classe di errore.

Invariante	Test	Come morde
Nessun accesso al DB dal thread della GUI	<code>test_screens_no_gui_thread_db.py</code>	Ispeziona il sorgente delle schermate: la funzione che tocca il DB deve essere una <code>staticmethod</code> eseguita da un <code>Worker</code>
Nessun tick rate scritto a mano	<code>test_tick_rate_ssot.py</code>	Scansione AST (immune a commenti e docstring) con un test-esca che fallisce se lo scanner smette di mordere
Nessuna lista di mappe ridichiarata	<code>test_known_maps_ssot.py</code>	Vieta il trio mirage/inferno/nuke nei consumatori convertiti e verifica che importino la SSOT
Una sola configurazione pytest	<code>test_single_pytest_config.py</code>	Verifica l'assenza del file ombra e conferma con un sottoprocesso quale configurazione viene risolta
Ogni tool mutante è protetto	<code>test_verify_all_safe_gate.py</code>	Censimento: nessuno strumento distruttivo può essere invocato nudo
Ogni token citato esiste davvero	<code>test_design_token_references.py</code>	Confronta ogni <code>tokens.<nome></code> nel codice Qt con i campi reali della dataclass
I chip si ricolorano al cambio tema	<code>test_theme_live_restyle.py</code>	Passa dal relay di modulo
Il timeout di parsing è reale	<code>test_parse_timeout_real.py</code>	Con un worker appeso 30 s, il chiamante deve tornare in meno di 5
Un solo runner reclama ogni demo	<code>test_ingestion_atomic_claim.py</code>	Claim esclusivo e task già reclamato che viene saltato
Il classificatore rifiuta dizionari senza vocabolario	<code>test_role_vocabulary_guard.py</code>	Vedi Parte 2, motori di analisi
Zero import di QtCharts	<code>test_charts.py</code>	Gate di licenza

Nella stessa campagna la soglia minima di copertura è salita da 33% a **50%** (`pyproject.toml`), è entrato `ruff` con un insieme di regole di partenza, ed è stato aggiunto un gate `pip check` sulle dipendenze.

Le cifre del cancello di test riportate dalla campagna — 2.574 test verdi, zero falliti, zero errori — sono quelle registrate in `docs/audit/FINAL_REPORT.md` al 14 agosto 2026. Ciò che è verificabile leggendo il repository, senza eseguire nulla, è la sua dimensione: **167 file di test** e **2.470 funzioni test_**.

12.20 Pre-commit Hooks e Quality Gates

File: `.pre-commit-config.yaml`

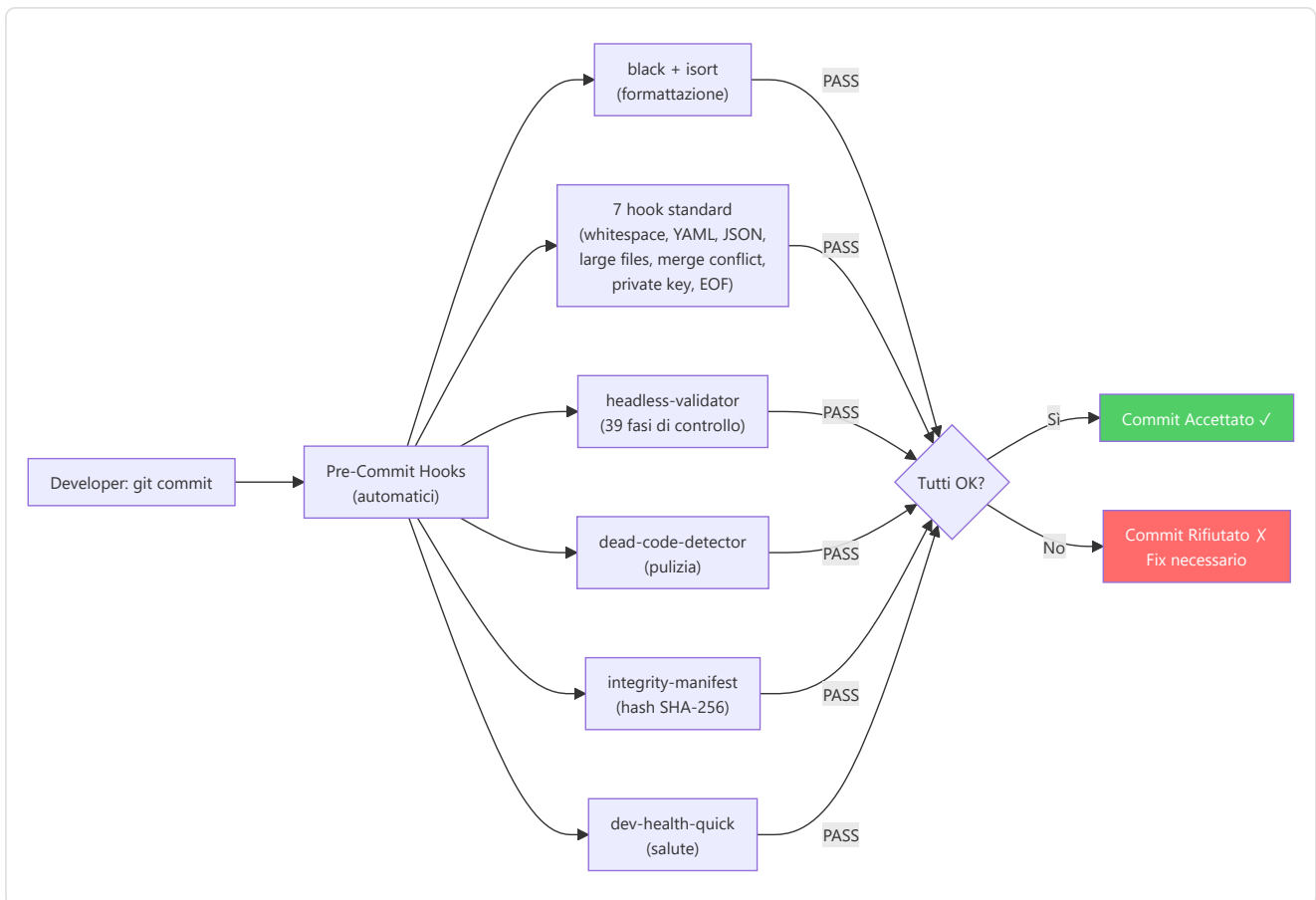
Il progetto utilizza un sistema di **pre-commit hooks** che si attivano automaticamente prima di ogni commit, impedendo che codice non conforme raggiunga il repository.

4 Hook Locali (custom del progetto):

Hook	Script	Timeout	Descrizione
<code>headless-validator</code>	<code>tools/headless_validator.py</code>	20s	39 fasi di controllo, regression gate — il più importante
<code>dead-code-detector</code>	<code>tools/dead_code_detector.py</code>	15s	Identifica import e funzioni non referenziati
<code>integrity-manifest-check</code>	<code>tools/sync_integrity_manifest.py</code>	10s	Verifica coerenza hash SHA-256 del manifesto
<code>dev-health-quick</code>	<code>tools/dev_health.py</code>	10s	Quick check salute progetto

Hook Standard (7 da `pre-commit/pre-commit-hooks` + 2 Python):

Hook	Sorgente	Descrizione
<code>trailing-whitespace</code>	<code>pre-commit/pre-commit-hooks</code>	Rimozione spazi bianchi finali
<code>end-of-file-fixer</code>	<code>pre-commit/pre-commit-hooks</code>	Newline finale obbligatorio
<code>check-yaml</code>	<code>pre-commit/pre-commit-hooks</code>	Validazione sintassi YAML
<code>check-json</code>	<code>pre-commit/pre-commit-hooks</code>	Validazione sintassi JSON
<code>check-added-large-files</code>	<code>pre-commit/pre-commit-hooks</code>	Blocca file >1MB (previene <code>.pt</code> accidentali)
<code>check-merge-conflict</code>	<code>pre-commit/pre-commit-hooks</code>	Rileva marcatori di conflitto merge non risolti
<code>detect-private-key</code>	<code>pre-commit/pre-commit-hooks</code>	Blocca commit di chiavi private
<code>black</code>	<code>psf/black</code>	Formattazione automatica Python (line length 100, target py3.12)
<code>isort</code>	<code>pycqa/isort</code>	Ordinamento automatico import (profilo black, line length 100)



12.21 Build, Packaging e Deployment

File chiave: `packaging/windows_installer.iss`, `setup_new_pc.bat`, `export_env.bat`

Il progetto include un sistema di build e packaging per la distribuzione dell'applicazione desktop su Windows.

3 File Requirements:

File	Scopo	Dipendenze
<code>requirements.txt</code>	Base — dipendenze principali per l'esecuzione, pin esatti (DEP-1; torch con range pin per varianti piattaforma)	26 pacchetti (torch, pyside6, sqlmodel, demoparser2, playwright, fastapi, uvicorn, httpx, etc.)
<code>requirements-ci.txt</code>	CI/CD — aggiunge strumenti di test e analisi	pytest, coverage, mypy, black, isort, pre-commit
<code>requirements-lock.txt</code>	Lock — versioni esatte per build riproducibili	Pin esatto di ogni dipendenza e sotto-dipendenza

Packaging Windows (`packaging/windows_installer.iss`):

- Installer Inno Setup con wizard grafico
- Icona personalizzata CS2 Analyzer
- Collegamento sul desktop e nel menu Start
- Dimensione stimata: ~500MB (include PyTorch e modelli)
- Target: Windows 10/11 64-bit

Script di Setup:

Script	Scopo
<code>setup_new_pc.bat</code>	Configurazione ambiente da zero: Python, pip, venv, dipendenze
<code>export_env.bat</code>	Esportazione variabili d'ambiente necessarie
<code>run_full_training_cycle.py</code>	Orchestrazione ciclo completo di addestramento ML

12.22 Sistema Migrazioni Alembic

Directory: `alembic/` (root) + `Programma_CS2_RENAN/backend/storage/migrations/`

Il progetto utilizza **due setup Alembic separati** per gestire le migrazioni dello schema del database in modo organizzato.

Alembic Root (`alembic/versions/` , **18 versioni** — **catena lineare** da `add_missing_profile_fields` a `drop_connect_state_from_ext_playerplaystyle`):

Contiene le migrazioni principali dello schema, dalla creazione iniziale delle tabelle alle modifiche più recenti. Ogni migrazione:

- Ha un hash univoco come identificatore
- Contiene `upgrade()` (applica modifica) e `downgrade()` (annulla modifica)
- È idempotente — può essere ri-applicata senza errori
- È testata su schema production-like prima del deploy

Scaffolding secondario (`Programma_CS2_RENAN/migrations/`):

Contiene solo lo scaffolding Alembic (`env.py` , `script.py.mako`) senza directory `versions/` — le migrazioni reali vivono tutte in `alembic/versions/` alla root.

Esecuzione automatica: `backend/storage/db_migrate.py::ensure_database_current()` viene chiamata all'avvio dell'applicazione: confronta `current_rev` vs `head_rev` e, se differiscono, esegue `alembic.command.upgrade(cfg, "head")` , garantendo che lo schema sia sempre aggiornato prima di qualsiasi operazione DB.

12.23 Orchestratore Ingestione Principale (`run_ingestion.py`)

File: `Programma_CS2_RENAN/run_ingestion.py`

L' `run_ingestion.py` è il **cuore orchestratore** dell'intera pipeline di ingestione. È il file più grande dedicato all'ingestione e coordina tutte le fasi dal discovery delle demo alla persistenza dei risultati nel database.

Funzioni principali (verificate nel codice):

Funzione	Ruolo
<code>_check_duplicate_demo()</code>	Deduplicazione SHA-256 su 3 archivi: IngestionTask (path esatto), PlayerMatchStats (per stem), esistenza DB per-match con <code>match_id = sha256(stem) % 2⁶³-1</code> (DA-03)
<code>_ingest_single_demo()</code>	Pipeline completa per una demo: parse aggregato → <code>persist_round_stats_and_enrichment()</code> → salvataggio stats
<code>_save_player_stats()</code>	Persiste PlayerMatchStats con sanitizzazione NaN/Inf (<code>_sanitize_value</code>)
<code>_save_sequential_data()</code>	Estrazione tick chunked: <code>BATCH_SIZE = 10000</code> se <code>HP_MODE=1</code> , altrimenti <code>2000</code> ; dual-write su DB per-match + monolite
<code>enrich_tick_data()</code>	(da <code>backend/processing/tick_enrichment.py</code>) calcola le feature cross-player 20-24 per tick
<code>_EventExtractor</code> / <code>_extract_and_store_events()</code>	Estrae weapon_fire/hurt/death/granate/bomba → MatchEventState nel DB per-match
<code>_build_match_tick_dataframe()</code>	DataFrame tick per tutti i giocatori (POV completo)
<code>_finalize_match_record()</code>	Chiusura del record match e metadati
<code>run_ml_pipeline()</code> / <code>_save_insights()</code>	Follow-on ML e persistenza CoachingInsight

Il driver batch parallelo è il root `batch_ingest.py` (`ingest_one_demo` delega a `run_ingestion._ingest_single_demo`; discovery ricorsiva `rglob("*.dem")` con esclusione symlink; `--no-train` via `ingest.sh`). Il worker `run_worker.py` fa claim atomico dei task (`status=processing` nella stessa transazione) e recupera i task stale (`_recover_stale_tasks`).

ResourceManager (`ingestion/resource_manager.py`):

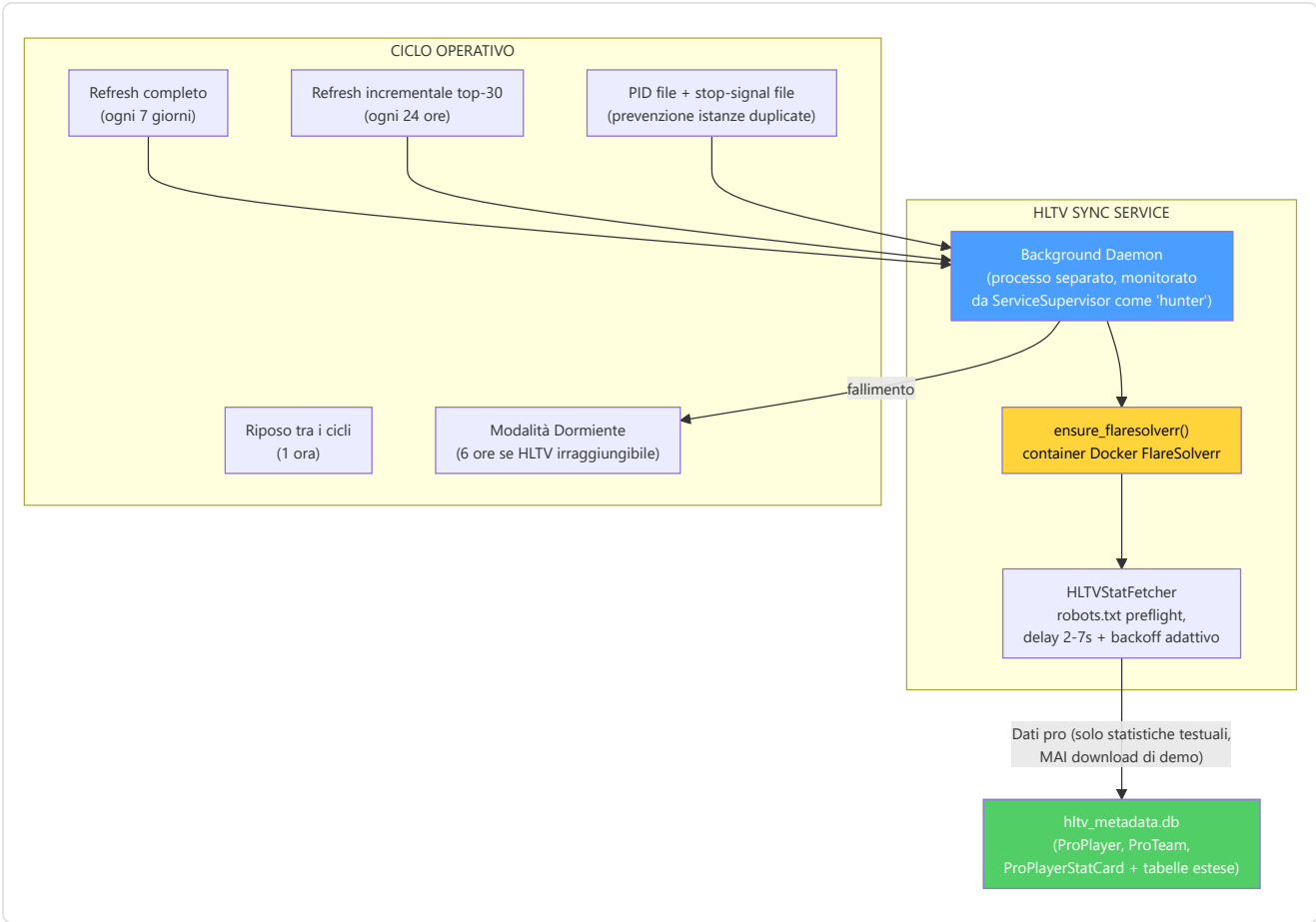
Il ResourceManager gestisce le **risorse hardware** durante l'ingestione per evitare il sovraccarico del sistema:

Parametro	Valore Default	Scopo
CPU threshold	80%	Pausa ingestione se CPU supera questa soglia
RAM threshold	85%	Pausa ingestione se RAM supera questa soglia
Disk check	Sì	Verifica spazio disco sufficiente prima del processing
Throttle delay	2s	Delay tra task consecutivi per raffreddamento

12.24 HLTV Sync Service e Background Daemon

File: `Programma_CS2_RENAN/hltv_sync_service.py` **File correlati:** `backend/data_sources/hltv/`, `backend/services/telemetry_client.py`

L'HLTV Sync Service è un **daemon in background** che sincronizza automaticamente i dati dei giocatori professionisti da HLTV.org. Opera come un servizio monitorato dal **ServiceSupervisor** della Console.



Politica di refresh: sincronizzazione **completa ogni 7 giorni, incrementale dei top-30 ogni 24 ore**, riposo di 1 ora tra i cicli, **modalità dormiente di 6 ore** quando HLTV non è raggiungibile. Il servizio scrive esclusivamente **statistiche testuali** (Rating 2.0, K/D, ADR, KAST, HS%) — non scarica mai file demo. Gira come processo detached (`subprocess.Popen`) con **PID file** e **stop-signal file** (`start_detached` / `stop_service`), separato dal session engine per evitare contesa WAL sul monolite.

12.25 RASP Guard — Integrità Runtime del Codice

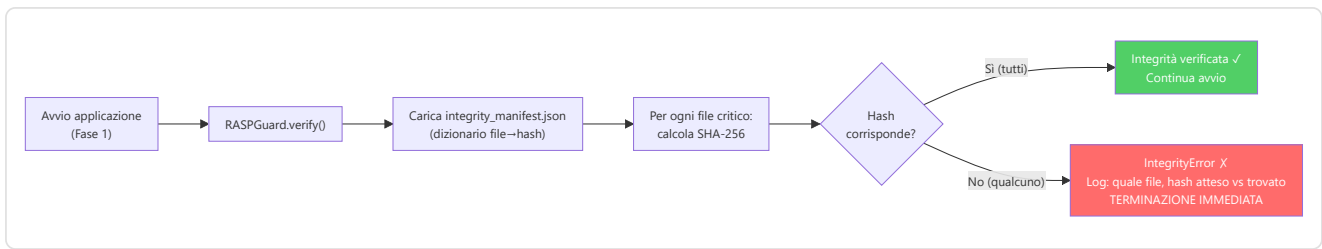
File: `Programma_CS2_RENAN/observability/rasp.py` **File dati:** `data/integrity_manifest.json`

Il RASP (Runtime Application Self-Protection) Guard è il **primo controllo** eseguito all'avvio dell'applicazione (Fase 1 della sequenza di boot). Verifica che nessun file sorgente sia stato modificato rispetto al manifesto di integrità.

Componenti del RASP Guard:

Componente	Descrizione
<code>RASPGuard</code>	Classe principale — carica il manifesto, verifica hash, solleva eccezione
<code>integrity_manifest.json</code>	File JSON con hash SHA-256 di ogni file sorgente critico
<code>IntegrityError</code>	Eccezione custom — terminazione immediata con log dettagliato
<code>sync_integrity_manifest.py</code>	Tool per aggiornare il manifesto dopo modifiche legittime

Flusso di verifica:



12.26 MatchVisualizer — Rendering Avanzato

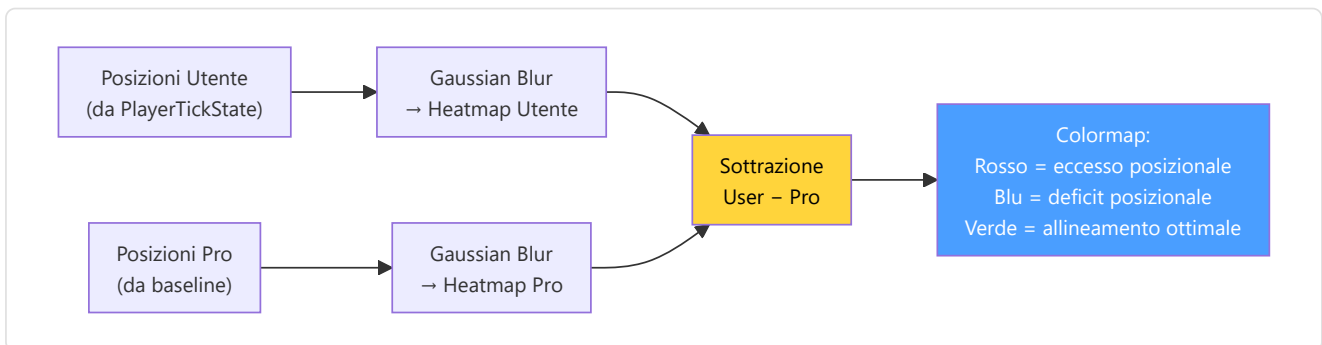
File: `Programma_CS2_RENAN/reporting/visualizer.py`

Il `MatchVisualizer` estende il sistema di reporting (sezione 12.13) con **6 metodi di rendering** specializzati per la generazione di grafici e visualizzazioni.

6 Metodi di rendering:

Metodo	Output	Descrizione
<code>render_skill_radar()</code>	Radar chart PNG	Pentagono a 5 assi (Meccanica, Posizionamento, Utility, Timing, Decisione) con overlay pro
<code>render_trend_chart()</code>	Line chart PNG	Tendenza storica di metriche chiave (KPR, ADR, KAST) su N partite
<code>render_heatmap()</code>	Heatmap PNG	Occupazione gaussiana 2D sulla mappa tattica
<code>render_differential_heatmap()</code>	Heatmap diff PNG	Sottrazione user-pro: rosso=troppo, blu=troppo poco, verde=ottimale
<code>render_critical_moments()</code>	Annotated timeline PNG	Timeline con marcatori colorati per momenti chiave (uccisioni, morti, piazzamenti)
<code>render_comparison_table()</code>	Table PNG/HTML	Tabella confronto user vs pro con delta percentuale per ogni metrica

Heatmap differenziale — algoritmo:



12.27 File di Dati e Configurazione Runtime

Il progetto include diversi file di dati che vengono letti o scritti durante l'esecuzione. Questi file non sono codice Python ma sono essenziali per il funzionamento del sistema.

`data/map_config.json` — Configurazione delle mappe CS2:

Contiene i parametri di trasformazione coordinate per ogni mappa supportata (Dust2, Mirage, Inferno, Nuke, Vertigo, Overpass, Ancient, Anubis):

```
{
  "de_dust2": {
    "pos_x": -2476, "pos_y": 3239, "scale": 4.4,
    "z_cutoff": null, "level": "single"
  },
  "de_nuke": {
    "pos_x": -3453, "pos_y": 2887, "scale": 7.0,
    "z_cutoff": -495, "level": "multi"
  }
}
```

data/integrity_manifest.json — Manifesto di integrità RASP:

Dizionario `{percorso_file: hash_sha256}` per tutti i file Python critici. Generato/aggiornato da `tools/sync_integrity_manifest.py`. Verificato all'avvio da **RASPGuard** (sezione 12.25).

data/training_progress.json — Progresso di addestramento:

Persiste lo stato del training tra i riavvii: epoca corrente, miglior loss, learning rate, ultima demo processata. Consultato dal Teacher daemon per riprendere il training dal punto esatto in cui si era fermato.

user_settings.json — Impostazioni utente:

File di configurazione livello 2 (cfr. sezione 12.3). Salvato nella directory del progetto, contiene preferenze personalizzabili: percorsi demo, tema UI, lingua, flag funzionalità.

12.28 Entry Point Root-Level

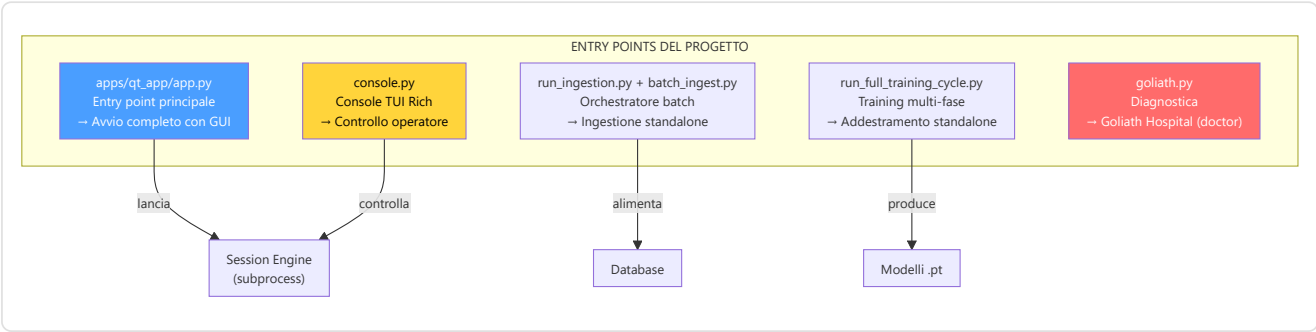
Il progetto include diversi **script eseguibili** a livello root (più gli entry point del pacchetto) che servono come punti di ingresso per operazioni specifiche:

Script	Posizione	Scopo	Invocazione
console.py	root	"MACENA UNIFIED CONSOLE v3.0": TUI Rich + CLI argparse, CommandRegistry con 10 categorie (ml, ingest, build, test, sys, set, svc, maint, tool, help)	python console.py
run_ingestion.py	Programma_CS2_RENAN/	Orchestratore ingestione standalone (cfr. 12.23)	python -m Programma_CS2_RENAN.run_ingestion
batch_ingest.py	root	Driver batch parallelo di ingestione (delega a run_ingestion)	python batch_ingest.py / ./ingest.sh
goliath.py	root	"MACENA GOLIATH": orchestratore Rich con subcomandi build, sanitize, integrity, audit, db, doctor (Goliath Hospital), baseline	python goliath.py <subcomando>
schema.py	root	SchemaSuite: CLI sqlite3 raw (non ORM) di ispezione/migrazione del DB — subcomandi inspect, migrate, import, fix, reset	python schema.py <subcomando>
run_full_training_cycle.py	root	Addestramento completo standalone (JEPA→Pro→User→RAP→RoleHead) con flag CLI (cfr. Parte 1A)	python run_full_training_cycle.py
run_worker.py	Programma_CS2_RENAN/	Worker di ingestione con claim atomico dei task e recupero stale	python -m Programma_CS2_RENAN.run_worker

Console operatore (console.py root, ≈66KB):

La console è il **punto di ingresso più potente** per gli operatori esperti. Offre una TUI (Terminal User Interface) basata su Rich con:

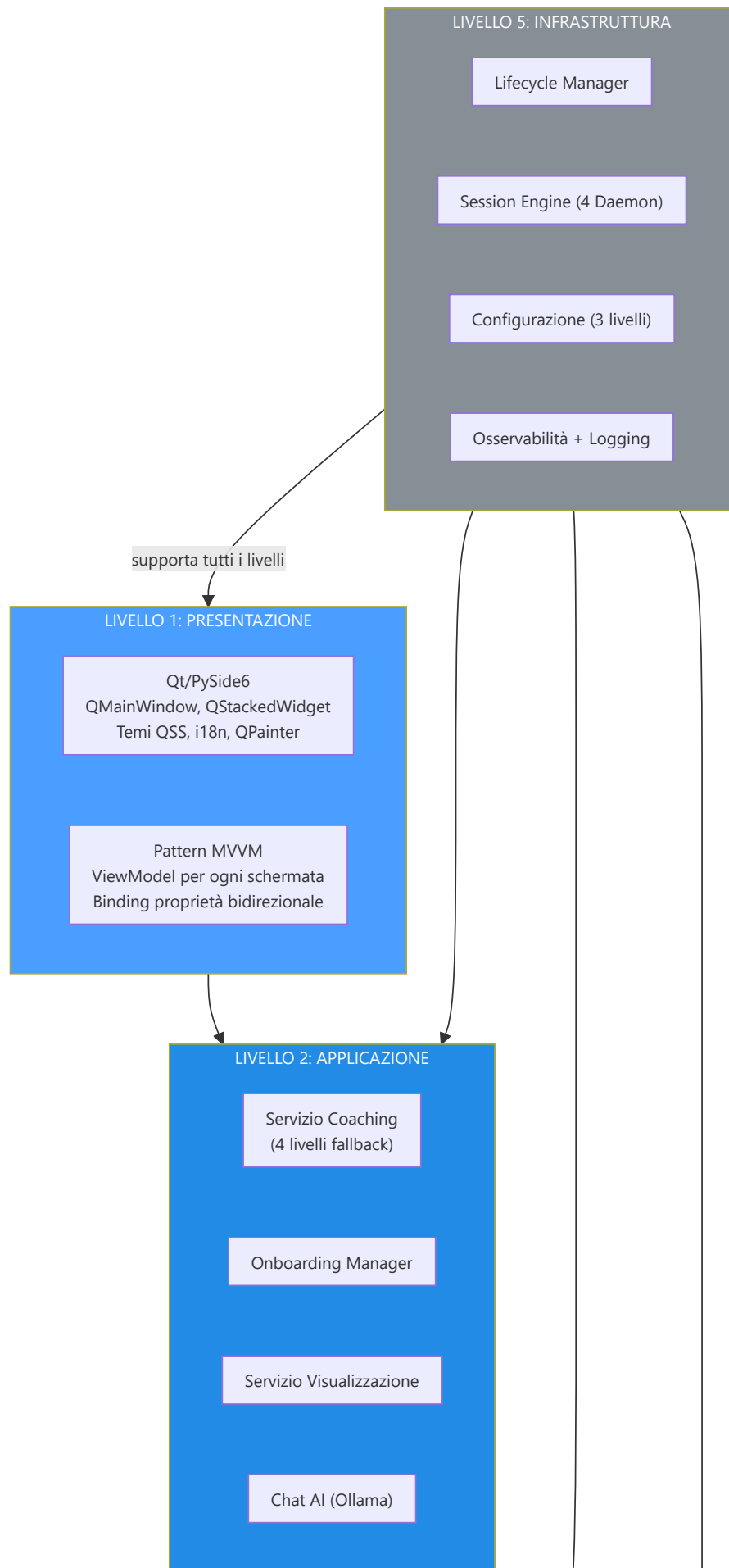
- **CommandRegistry**: Sistema registrato di comandi in 10 categorie
- **Modalità CLI**: Esecuzione single-shot di comandi (run_cli_mode)
- **Modalità TUI**: Dashboard interattiva (run_tui_mode) con TUIRenderer e StatusPoller
- **Guardia venv**: verifica l'ambiente virtuale all'avvio

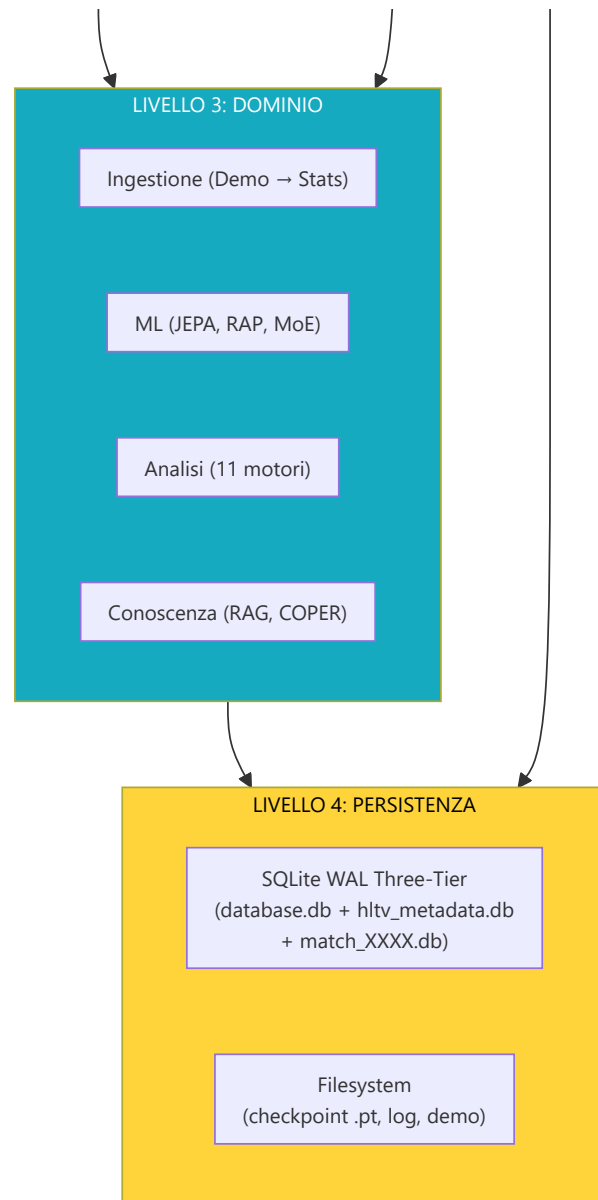


Riepilogo Architettuale

Macena CS2 Analyzer è un'applicazione **stratificata e modulare** organizzata su 5 livelli architetturali con 3 processi separati.







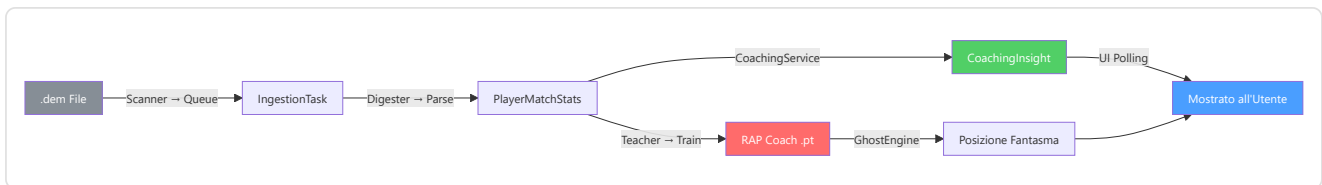
Stack tecnologico (26 dipendenze pinnate in `requirements.txt`):

Categoria	Libreria	Versione	Ruolo nel progetto
ML Core	PyTorch	2.x	Reti neurali (JEPA, RAP, MoE, NeuralRoleHead, WinProb)
ML Ext	ncps	latest	Liquid Time-Constant Networks (LTC) per memoria RAP
ML Ext	hflayers	latest	Hopfield layers per attenzione associativa
UI	PySide6	6.x	Binding Qt 6 per Python — framework UI desktop primario
DB	SQLModel	latest	ORM (Pydantic + SQLAlchemy)
DB	SQLAlchemy	2.x	Engine database sottostante
DB	Alembic	1.x	Migrazioni schema
HTTP	requests	2.x	API Steam/FACEIT (sincrono)
HTTP	httpx	latest	Telemetria (asincrono)
Scraping	playwright	1.58.0	Browser headless (dipendenza disponibile; lo scraping HLTV passa per FlareSolvrr)
Parsing	demoparser2	latest	Parser demo CS2 (Rust-based, veloce)
Data	pandas	2.x	Manipolazione dati tabellari
Data	numpy	1.x	Calcoli numerici
Viz	matplotlib	3.x	Grafici e visualizzazioni
NLP	sentence-transformers	latest	Embedding 384-dim per RAG
TUI	rich	latest	Console colorata, tabelle, progress bar
Logging	logging (stdlib)	—	Logging strutturato
Security	keyring	latest	Gestione sicura API keys
Monitoring	sentry-sdk	latest	Error tracking remoto (opzionale)
Testing	pytest	latest	Framework test
Formatting	black	latest	Formattazione codice
Import	isort	latest	Ordinamento import
QA	pre-commit	latest	Hook pre-commit
Crypto	hashlib (stdlib)	—	SHA-256 per RASP manifesto
Concurrency	threading (stdlib)	—	Daemon thread, MLControlContext
IPC	subprocess (stdlib)	—	Session Engine come subprocess
Config	json (stdlib)	—	user_settings.json, map_config.json

I 3 processi dell'applicazione:

Processo	Tipo	Responsabilità	Comunicazione
Main	Qt GUI	Interfaccia utente, rendering, interazione	Polling DB ogni 10-15s
Daemon	Subprocess (Session Engine)	Scanner, Digester, Teacher, Pulse	stdin pipe (IPC) + DB condiviso
Servizi Opzionali	Processi esterni	HLTV sync, Ollama LLM locale	HTTP/API + supervisione Console

Flussi dati principali:



Nota sulla Rimediazione — Codice Eliminato (G-06)

Durante il processo di rimediazione (cfr. sezione 12.19), il contenuto della directory `backend/nn/advanced/` è stato **eliminato** in quanto codice morto non referenziato (oggi la directory contiene solo un `__init__.py` segnaposto):

- `superposition_net.py` — Una rete di sovrapposizione sperimentale mai integrata nel flusso di addestramento o inferenza. La funzionalità di sovrapposizione attiva è implementata in `layers/superposition.py` (utilizzata dal livello Strategia RAP).
- `brain_bridge.py` — Un ponte tra modelli sperimentale mai chiamato da nessun modulo.
- `feature_engineering.py` — Feature engineering duplicato; la versione canonica risiede in `backend/processing/feature_engineering/`.

Motivazione (G-06): Mantenere codice morto crea rischi di confusione (quale `superposition` è quello vero?), aumenta la superficie di manutenzione e può introdurre import accidentali. L'eliminazione è stata verificata tramite analisi statica delle dipendenze: nessun file nel progetto importava da `backend/nn/advanced/`.

Utilità Condivise — `round_utils.py`

File: `Programma_CS2_RENAN/backend/knowledge/round_utils.py`

La funzione `infer_round_phase(equipment_value)` è un'**utilità condivisa** utilizzata sia dal servizio di coaching che dal sistema di conoscenza per classificare la fase economica di un round. Risiede in `backend/knowledge/` ed è importata da moduli in `services/` e `processing/`.

Valore equipaggiamento	Fase restituita
< \$1.500	"pistol"
\$1.500 – \$2.999	"eco"
\$3.000 – \$3.999	"force"
≥ \$4.000	"full_buy"

Utilità Core — Lock Files e Platform

Lock Files (`core/lock_files.py`):

Sistema di lock file per la concorrenza tra processi D-track / HLTV-track. Utilizza una directory `.locks/` locale al repository (sopravvive alle sessioni, non ai riavvii).

Componente	Descrizione
Formato lock file	<code><pid> <iso_timestamp></code> — identifica il processo proprietario
Lock stale	Recuperati automaticamente se il PID proprietario è morto (<code>os.kill(pid, 0)</code>)
<code>LockConflict(RuntimeError)</code>	Sollevata quando il lock è detenuto da un processo vivo
<code>acquire(name) → Path</code>	Crea lock, controlla conflitti, recupera stale, scrive PID + timestamp
<code>release(name)</code>	Rimuove lock file. Idempotente
<code>lock(name)</code> (context manager)	Acquire all'ingresso, release all'uscita
Signal handlers	<code>install_signal_handlers()</code> registra handler SIGTERM/SIGINT che rilasciano tutti i lock prima della terminazione
<code>_held_locks: Set[str]</code>	Stato module-level che traccia i lock attualmente detenuti

Punti di Forza dell'Architettura

- Contratto di funzionalità unificato a 25 dimensioni** — `METADATA_DIM = 25` impone la parità di addestramento/inferenza a livello di sistema.
- Gating di maturità a 3 livelli** — Previene l'implementazione prematura del modello con dati insufficienti.
- Fallback di coaching a 4 livelli** — COPER → Ibrido → RAG → Base garantisce sempre la fornitura di insight.
- Diversità multi-modello** — JEPA, VL-JEPA, LSTM+MoE, RAP e NeuralRoleHead contribuiscono a bias induttivi complementari.
- Suddivisione temporale** — Previene la perdita di dati garantendo l'ordinamento cronologico.
- Ciclo di feedback COPER** — Monitoraggio dell'efficacia basato su EMA con decadimento dell'esperienza obsoleta.
- Suite di analisi di Fase 6** — 11 motori di analisi (ruolo, probabilità di vittoria, albero di gioco, convinzione, inganno, momentum, entropia, punti ciechi, utilità ed economia, distanza di ingaggio, qualità movimento).
- Persistenza della soglia** — Le soglie di ruolo sopravvivono ai riavvii tramite la tabella DB `RoleThresholdRecord`.
- Euristica configurabile** — `HeuristicConfig` esternalizza i limiti di normalizzazione in JSON.
- Polishing LLM** — Integrazione opzionale con Ollama per narrazioni di coaching in linguaggio naturale.
- Training Observatory** — Introspezione a 4 livelli (Callback, TensorBoard, Maturity State Machine, Embedding Projector) con impatto zero quando disabilitato e callback isolate dagli errori.
- Neural Role Consensus** — Doppia classificazione euristica + NeuralRoleHead MLP con protezione cold-start, che garantisce un'assegnazione dei ruoli affidabile anche con dati parziali.
- Per-Round Statistical Isolation** — Il modello `RoundStats` impedisce la contaminazione tra round, consentendo un coaching granulare a livello di round e valutazioni HLTV 2.0 per round.
- Architettura Quad-Daemon** — Separazione completa tra GUI e lavoro pesante, con shutdown coordinato e zombie task cleanup automatico.
- Degradazione graduale pervasiva** — Ogni componente ha un piano di fallback: il sistema non crasha mai, degrada sempre in modo controllato.
- Architettura Three-Tier Storage** — Separazione di `database.db` (core + conoscenza, 18 tabelle), `hlvtv_metadata.db` (dati pro, 3 tabelle) e `match_data/{id}.db` (telemetria per-match) per eliminare la contesa WAL e prevenire la crescita incontrollata del monolite.
- Calibrazione Bayesiana Live (G-07)** — Lo stimatore di morte si auto-calibra con `extract_death_events_from_db()` → `auto_calibrate()`, trasformandosi da modello statico a sistema adattivo.
- Controllo Live Addestramento (MLControlContext)** — Pause/resume/stop/throttle in tempo reale del training via `threading.Event`, con eccezione custom `TrainingStopRequested` al posto di `StopIteration`.

19. **Resilienza HLTV Adattiva** — backoff adattivo del fetcher sui fallimenti consecutivi + modalità dormiente di 6 ore del sync service quando HLTV è irraggiungibile: previene cascade failure sulle API esterne.
20. **Piramide di Validazione a 5 Livelli** — Headless Validator → pytest → Backend Validator → Goliath Hospital → Brain Verify: ogni livello progressivamente più profondo, dal quick smoke test (10s) all'audit completo di 118 regole.
21. **RASP Guard** — Runtime Application Self-Protection con manifesto SHA-256: verifica integrità del codice sorgente all'avvio, previene manomissioni accidentali e intenzionali.
22. **Pre-commit Gate a 13 Hook** — 4 hook custom + 7 standard + 2 Python impediscono che codice non conforme raggiunga il repository. Formattazione, validazione, dead code, integrità, merge conflict e private key verificati automaticamente.
23. **ResourceManager Hardware-Aware** — Throttling automatico CPU/RAM durante ingestione: il sistema rallenta autonomamente se le risorse hardware sono sotto pressione, senza intervento umano.
24. **Test Forensics** — 10 script di indagine post-mortem per diagnosticare training failure, weight anomalies, coaching path traces e DB consistency — la "scatola nera" del sistema.

PUNTI DI FORZA ARCHITETTURALI - I 24 PILASTRI

1. Contratto unificato 25-dim - Tutti parlano la stessa lingua

2. Gate maturità 3 livelli - Nessun rilascio prematuro

3. Fallback coaching 4 livelli - Mai a mani vuote

4. Diversità multi-modello - 5 cervelli > 1 cervello

5. Divisione temporale - Nessun imbroglio viaggi nel tempo

6. Loop feedback COPER - Impara dai propri consigli

7. Analisi Fase 6 (11 mot.) - 11 detective specializzati

8. Persistenza soglie - Sopravvive ai riavvii

9. Euristiche configurabili - Override via JSON

10. Rifinitura LLM (Ollama) - Consigli suonano naturali

11. Osservatorio Addestramento - Pagella per il cervello

12. Consenso Neurale Ruoli - Due insegnanti confrontano note

13. Isolamento Per-Round - Valuta ogni domanda, non solo il test

14. Architettura Quad-Daemon - GUI reattiva, lavoro pesante in background

15. Degradazione Graduale - Il sistema non crasha mai

16. Three-Tier Storage - Nessuna contesa tra scrittura e lettura

17. Calibrazione Bayesiana Live - Si auto-calibra con i dati

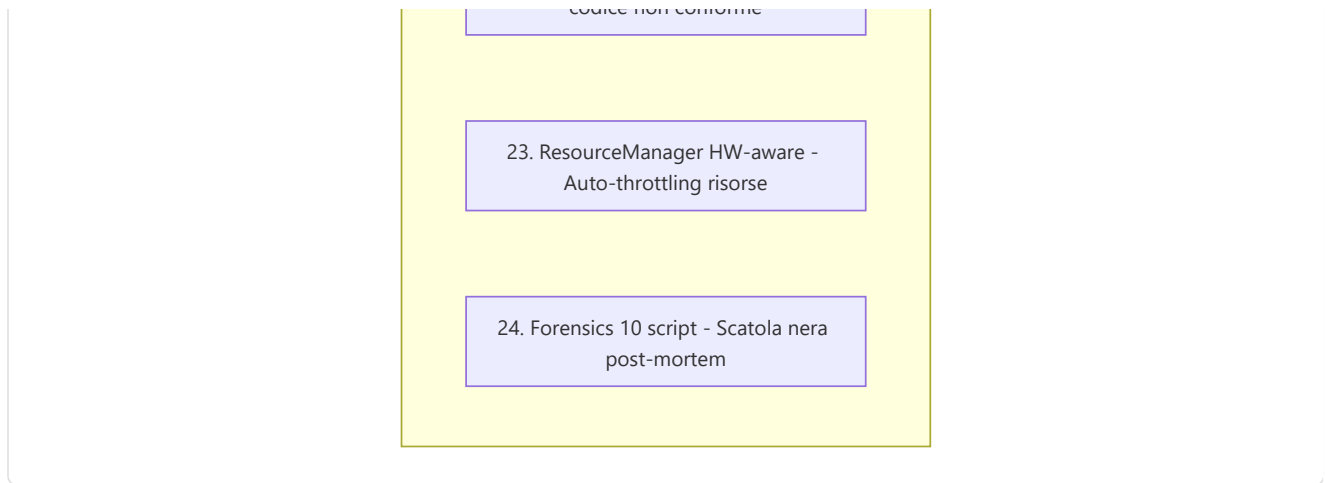
18. Controllo Live Training - Pausa/Stop senza perdita

19. Resilienza HLTV - Backoff adattivo + dormienza

20. Piramide Validazione 5 livelli - Dal quick test al deep audit

21. RASP Guard SHA-256 - Integrità runtime garantita

22. Pre-commit 13 hook - Zero codice non conforme

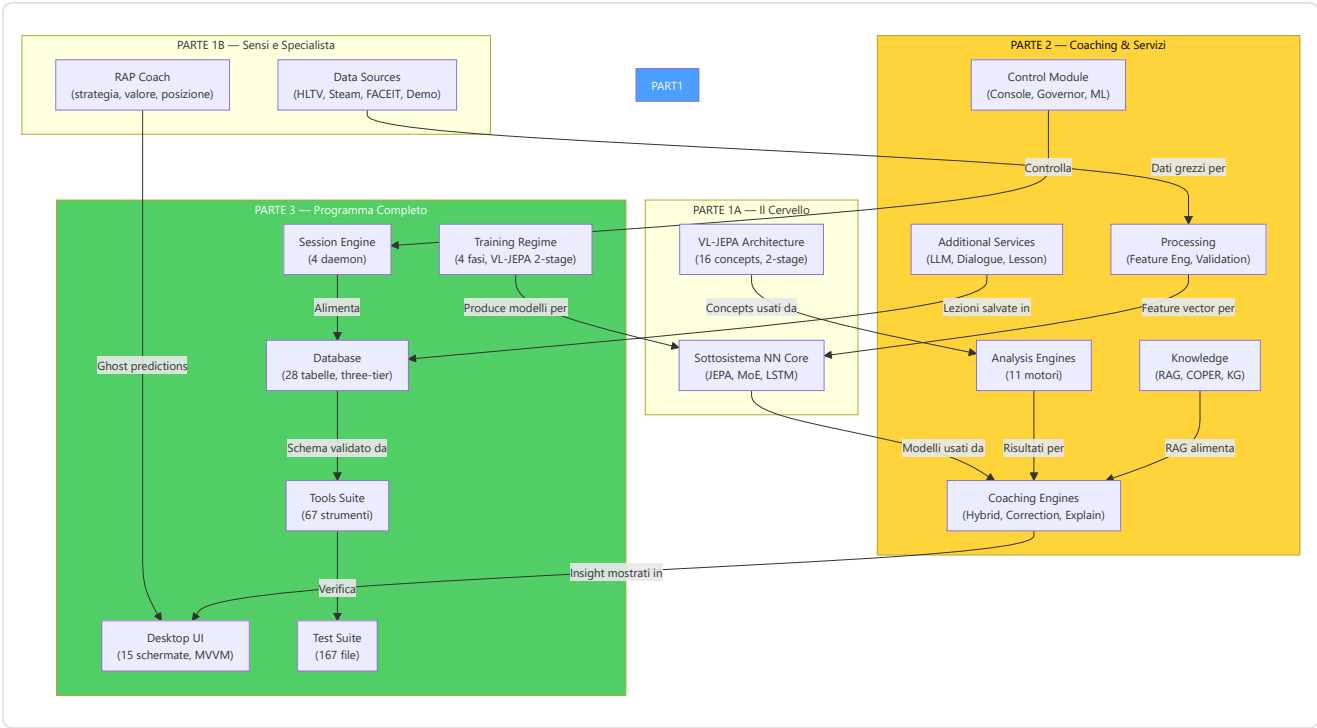


Fine documento — Guida completa di Macena CS2 Analyzer

Totale file `.py` nel progetto: **493** (in `Programma_CS2_RENAN/`) Totale righe di codice Python: **≈ 126.400** Sottosistemi AI coperti: **8** (NN Core, VL-JEPA, RAP Coach, Servizi di Coaching, Motori di Coaching, Conoscenza, Analisi, Elaborazione + Osservatorio Addestramento) Sottosistemi programma coperti: **18** (Avvio, Lifecycle, Configurazione, Session Engine, UI Desktop, Ingestione, Storage, Osservabilità, Console di Controllo, RASP Guard, HLTV Sync, Orchestratore Ingestione, ResourceManager, Tools Suite, Test Suite, Pre-commit, Build/Packaging, Migrazioni Alembic) Modelli documentati: **6** (AdvancedCoachNN/TeacherRefinementNN, JEPA, VL-JEPA, RAPCoachModel, NeuralRoleHead, WinProbabilityNN) Motori di analisi documentati: **11** (Ruolo, WinProb, GameTree, Credenza, Inganno, Momentum, Entropia, Punti Ciechi, Utilità ed Economia, Distanza di Ingaggio, Qualità Movimento) Motori di coaching documentati: **8** (HybridEngine, CorrectionEngine, ExplainabilityGenerator, NNRefinement, ProBridge, TokenResolver, LongitudinalEngine, JEPAINsightAdapter) Servizi aggiuntivi documentati: **8** (CoachingDialogue, LessonGenerator, LLMSERVICE, VisualizationService, ProfileService, AnalysisService, TelemetryClient, PlayerLookupService) Tabelle di database documentate: **28** (18 monolite + 7 HLTV + 3 per-match, su architettura three-tier storage: `database.db`, `hlty_metadata.db`, `match_data/{id}.db`) Schermate UI documentate: **15** (Wizard, Home, Coach, Tactical Viewer, Settings, Help, Match History, Match Detail, Performance, User Profile, Profile, Pro Comparison, Pro Player Detail, Steam Config, FACEIT Config) — Qt/PySide6 Daemon documentati: **4** (Scanner, Digester, Teacher, Pulse) Strumenti di validazione documentati: **71** (53 root + 18 nel pacchetto: Headless Validator, Goliath Hospital, DB Inspector, Demo Inspector, ML Coach Debugger, Backend Validator, Dead Code Detector, Dev Health, Feature Audit, etc.) File di test documentati: **130** in `Programma_CS2_RENAN/tests/` (+ `conftest.py`; nella root: 7 test, 6 verify script, 10 file forensics) Fasi headless validator: **39** fasi tematiche di controllo Pre-commit hooks documentati: **13** (4 locali custom + 7 standard + 2 Python) Pilastri architetturali: **24** (inclusi Three-Tier Storage, Calibrazione Bayesiana Live, Controllo Live Training, Resilienza HLTV, Piramide Validazione, RASP Guard, Pre-commit Gate, ResourceManager HW-aware, Forensics) Problemi risolti tramite rimediazione: **610+** (12 fasi iniziali con codici G-XX e F-XX + ondate successive)

Mappa delle Interconnessioni tra le 3 Parti

Le tre parti della documentazione formano un sistema interconnesso. Questa mappa mostra le principali dipendenze tra le sezioni:



Riferimenti incrociati chiave:

Da (Parte)	A (Parte)	Collegamento
Part 1 §VL-JEPA	Part 3 §10 (Training)	Il protocollo two-stage descritto in Part 1 è implementato dal training regime in Part 3
Part 1 §Data Sources	Part 3 §12.6 (Ingestion)	Le sorgenti dati alimentano la pipeline di ingestione
Part 2 §Coaching Engines	Part 3 §12.16 Flusso 1	I motori di coaching generano gli insight mostrati nell'UI
Part 2 §Control Module	Part 3 §12.4 (Session Engine)	Il modulo di controllo gestisce i 4 daemon
Part 2 §Processing	Part 1 §NN Core	Il vectorizer produce il METADATA_DIM=25 consumato dai modelli
Part 3 §9 (Database)	Part 2 §Knowledge	Le tabelle RAG/COPER sono usate dal sistema di conoscenza
Part 3 §11 (Loss)	Part 1 §VL-JEPA	Le loss functions documentate implementano l'addestramento VL-JEPA
Part 3 §12.17 (Tools)	Part 3 §12.18 (Tests)	La piramide di validazione include sia tools che test suite

Glossario Tecnico

Termine	Definizione
ADR	Average Damage per Round — danno medio inflitto per round
BCE	Binary Cross-Entropy — loss per classificazione binaria
COPER	Coaching through Past Experiences and References — coaching basato su esperienze passate
EMA	Exponential Moving Average — media mobile esponenziale usata per target encoder JEPA
HLTV 2.0	Sistema di rating di HLTV.org basato su KPR, survival, impact, damage
InfoNCE	Noise Contrastive Estimation — loss contrastiva per allineare rappresentazioni
JEPA	Joint-Embedding Predictive Architecture — architettura che predice embedding target da contesto
KAST	Kill/Assist/Survive/Trade — percentuale di round con contributo positivo
KG	Knowledge Graph — grafo di conoscenza con entità e relazioni
KPR	Kills Per Round — uccisioni medie per round
LTC	Liquid Time-Constant — reti neurali con costanti temporali apprese
METADATA_DIM	Dimensione del feature vector unificato (25 nel progetto)
MoE	Mixture of Experts — rete con esperti specializzati e gating
MVVM	Model-View-ViewModel — pattern architetturale per separazione UI/logica
RAG	Retrieval-Augmented Generation — generazione arricchita da retrieval
RAP	Reasoning Action Planning — modello per coaching strategico
RASP	Runtime Application Self-Protection — protezione runtime dell'applicazione
TUI	Terminal User Interface — interfaccia utente nel terminale
VICReg	Variance-Invariance-Covariance Regularization — regolarizzazione per diversità embedding
VL-JEPA	Vision-Language JEPA — estensione con concetti linguistici
WAL	Write-Ahead Logging — modalità SQLite per letture/scritture concorrenti
Z-score	Numero di deviazioni standard dalla media — misura quanto un valore è lontano dalla norma

Autore: Renan Augusto Macena